



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
École Supérieure Privée d'Ingénierie et de Technologie
TEK-UP



Rapport de Projet de Fin d'Études

Présenté en vue de l'obtention du

Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques

Spécialité : **Génie Logiciel et Systèmes d'Information**

Conception et développement d'un portail juridique intelligent basé sur une architecture microservices

Réalisé par

Moetez BENFREDJ

Encadrant professionnel

Mohamed Sadok Mouha

Ingénieur Informatique

Progress Engineering

Encadrant académique

Siwar Zrelli

Ingénieur Informatique

TEK-UP

Année Universitaire 2025 - 2026

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **M. Mohamed Sadok Mouha**

Signature et cachet



J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **Mm. Siwar Zrelli**

Signature

Dédicace

Je dédie ce modeste travail à :

Mes chers parents, pour leur amour inconditionnel, leur soutien indéfectible et les sacrifices consentis tout au long de mon parcours. Ce diplôme est le fruit de leurs efforts autant que des miens.

Ma famille tout entière, pour les encouragements et la bienveillance qui m'ont accompagné à chaque étape.

Mes amis et collègues, pour leur présence, leur solidarité et les moments partagés qui rendent la vie estudiantine précieuse.

Tous mes enseignants, depuis la première classe jusqu'aux professeurs d'ingénierie, pour le savoir transmis, la passion insufflée et les valeurs inculquées.

À toute personne qui croit que la technologie peut servir le droit et rendre la justice accessible à tous les citoyens.

Moetez BENFREDJ

Remerciements

Il est particulièrement agréable, avant de présenter ce travail, d'exprimer toute ma gratitude envers les personnes qui, de près ou de loin, m'ont apporté leur aide précieuse lors de la réalisation de ce projet de fin d'études.

Je tiens tout d'abord à exprimer ma profonde reconnaissance à mon encadrant professionnel au sein de

Progress Engineering,

M. Mohamed Sadok Mouha,

pour sa disponibilité, ses conseils avisés, son encadrement bienveillant et sa confiance accordée tout au long de ce stage.

J'adresse également mes sincères remerciements à mon encadrant académique,

Mm. Siwar Zrelli,

Ingénieur Informatique à l'École Supérieure Privée d'Ingénierie et de Technologie, pour son suivi rigoureux, ses orientations méthodologiques et son implication dans la réussite de ce projet.

*Mes remerciements vont aussi à l'ensemble du corps enseignant de la spécialité **Génie Logiciel et Systèmes d'Information** pour la formation de qualité dispensée au fil de ces trois années d'études.*

Je remercie chaleureusement les membres du jury qui ont bien voulu consacrer leur temps et leur expertise à l'évaluation de ce travail.

Enfin, un grand merci à ma famille et à mes amis pour leur soutien moral constant et leurs encouragements qui m'ont donné la force d'aller jusqu'au bout.

Table des matières

Liste des figures	x
Liste des tableaux	xii
Liste des abréviations	xiv
Introduction générale	1
1 Cadre du projet	3
Introduction	4
1.1 Cadre général du projet	4
1.1.1 Présentation de l'organisme d'accueil	4
1.1.2 Contexte et objectifs du projet	5
1.2 Étude de l'existant	6
1.2.1 Étude du portail JORT en ligne	7
1.2.2 Étude de la plateforme Legislation.tn	7
1.2.3 Étude de la plateforme Casetext	8
1.3 Problématique	9
1.4 Solution proposée	10
1.5 Méthodologie de gestion de projet	11
1.5.1 Méthodologie Scrum	11
1.5.2 Rôles Scrum	11
1.5.3 Cérémonies Scrum	11
Conclusion	12
2 Analyse et spécification des besoins	13
Introduction	14
2.1 Spécification des besoins	14
2.1.1 Identification des acteurs	14
2.1.2 Spécification des besoins fonctionnels	14
2.1.3 Spécification des besoins non fonctionnels	15

2.2	Diagramme de cas d'utilisation global	16
2.3	Modèle de données	17
2.3.1	Diagramme de classes global	17
2.3.2	Description des entités principales	18
2.3.3	Schéma de la base de données OpenSearch	20
2.4	Pilotage du projet avec Scrum	20
2.4.1	Équipe et rôles du projet	20
2.4.2	Product Backlog global	21
2.4.3	Planification des releases	25
2.5	Architecture du système	26
2.5.1	Vue d'ensemble de l'architecture microservices	26
2.5.2	Description des microservices	26
2.5.3	Flux de communication inter-services	27
2.5.4	Infrastructure	27
2.6	Environnement de travail	28
2.6.1	Notre environnement matériel	28
2.6.2	Langages de programmation	28
2.6.3	Pile backend	29
2.6.4	Pile frontend	31
2.6.5	Intelligence artificielle	32
2.6.6	Bases de données, moteurs et infrastructure	35
2.6.7	Outils de développement	37
	Conclusion	38
3	Release 1 : Authentification et Moteur de Recherche	39
	Introduction	40
3.1	Sprint 1 : Service d'authentification et gestion des utilisateurs	40
3.1.1	Objectifs du Sprint 1	40
3.1.2	Backlog du Sprint 1	40
3.1.3	Planification des tâches du Sprint 1	41
3.1.4	Conception du Sprint 1	42
3.1.5	Réalisation du Sprint 1	43

3.1.6	Tests du Sprint 1	45
3.1.7	Revue et rétrospective du Sprint 1	46
3.2	Sprint 2 : Moteur de recherche hybride et signets	46
3.2.1	Objectifs du Sprint 2	46
3.2.2	Backlog du Sprint 2	46
3.2.3	Planification des tâches du Sprint 2	47
3.2.4	Conception du Sprint 2	48
3.2.5	Réalisation du Sprint 2	49
3.2.6	Tests du Sprint 2	51
3.2.7	Revue et rétrospective du Sprint 2	52
	Conclusion	52
4	Release 2 : Assistant RAG et Pipeline d'Ingestion	53
	Introduction	54
4.1	Sprint 3 : Assistant conversationnel RAG	54
4.1.1	Objectifs du Sprint 3	54
4.1.2	Backlog du Sprint 3	54
4.1.3	Planification des tâches du Sprint 3	55
4.1.4	Conception du Sprint 3	55
4.1.5	Réalisation du Sprint 3	57
4.1.6	Tests du Sprint 3	59
4.1.7	Revue et rétrospective du Sprint 3	60
4.2	Sprint 4 : Pipeline d'ingestion documentaire	60
4.2.1	Objectifs du Sprint 4	60
4.2.2	Backlog du Sprint 4	60
4.2.3	Planification des tâches du Sprint 4	61
4.2.4	Conception du Sprint 4	62
4.2.5	Réalisation du Sprint 4	64
4.2.6	Tests du Sprint 4	66
4.2.7	Revue et rétrospective du Sprint 4	66
	Conclusion	66

5	Release 3 : Notifications, Alertes et Tableau de Bord Analytique	67
	Introduction	68
5.1	Sprint 5 : Notifications et alertes thématiques	68
5.1.1	Objectifs du Sprint 5	68
5.1.2	Backlog du Sprint 5	68
5.1.3	Planification des tâches du Sprint 5	69
5.1.4	Conception du Sprint 5	70
5.1.5	Réalisation du Sprint 5	71
5.1.6	Tests du Sprint 5	73
5.1.7	Revue et rétrospective du Sprint 5	74
5.2	Sprint 6 : Tableau de bord analytique et administration	74
5.2.1	Objectifs du Sprint 6	74
5.2.2	Backlog du Sprint 6	74
5.2.3	Planification des tâches du Sprint 6	75
5.2.4	Conception du Sprint 6	76
5.2.5	Réalisation du Sprint 6	76
5.2.6	Contrôle d'accès basé sur les rôles	79
5.2.7	Tests du Sprint 6	83
5.2.8	Revue et rétrospective du Sprint 6	83
5.3	Revue et rétrospective globale	84
	Conclusion	84
6	Pipeline d'extraction et d'indexation du corpus juridique	85
	Introduction	86
6.1	Audit préalable du corpus JORT	86
6.2	Première itération : la chaîne texte classique	87
6.2.1	Vue d'ensemble du <i>pipeline</i>	87
6.2.2	Quatre obstacles que nous avons documentés	87
6.2.3	Bilan de la première itération	88
6.3	Deuxième itération : nettoyage par modèle de langage textuel	88
6.3.1	Notre motivation	88
6.3.2	Choix du modèle et instrumentation	89

6.3.3	Résultats RAGAS observés	89
6.3.4	Trois limites que nous avons identifiées	89
6.3.5	Bilan de la deuxième itération	90
6.4	Troisième itération : modèle vision-langage Qwen2.5-VL	90
6.4.1	Pourquoi l'extraction d'actes juridiques est un problème difficile	90
6.4.2	Notre choix : un modèle vision-langage exécuté en local	91
6.4.3	Notre nouveau <i>pipeline</i>	91
6.4.4	Une démarche expérimentale : des erreurs observées aux règles du <i>prompt</i>	93
6.4.5	Le tournant : séparer la lecture de la structuration	94
6.4.6	Robustesse face aux réponses dégradées du modèle	95
6.4.7	Post-traitement : fiabiliser et consolider les champs	95
6.4.8	Structure des données extraites et analyse des résultats	96
6.4.9	Gains et limites de notre chaîne V3	97
6.5	Validation sur des cas réels et difficultés résiduelles	98
6.5.1	Acte réparti sur deux pages	98
6.5.2	Tableau de nominations sur une page entière	99
6.5.3	Difficulté résiduelle : tableau à cheval sur deux pages	100
6.5.4	Page mixte : décrets, arrêtés et tableau encadré	100
6.5.5	Validation de bout en bout par interrogation RAG	101
6.5.6	Bilan des difficultés d'extraction	102
6.6	Synthèse comparative de nos trois itérations	102
6.7	Indexation OpenSearch	103
6.7.1	Notre <i>pipeline</i> en quatre commandes	103
6.7.2	Notre <i>mapping</i> bilingue à analyseurs spécialisés	103
6.7.3	Multi-fields et types imbriqués	104
6.7.4	Du JSONL au <i>mapping</i> : notre étape de normalisation	104
6.7.5	Vérification après indexation	105
6.7.6	Diagramme d'ensemble de notre indexation	105
	Conclusion	105
7	Conteneurisation, déploiement et intégration continue	106
	Introduction	107

7.1	Notre stratégie de conteneurisation	107
7.1.1	Construction multi-étages pour les microservices Java	107
7.1.2	Frontend Angular servi par Nginx	108
7.1.3	Image Python pour l’agent conversationnel	108
7.2	Orchestration locale avec Docker Compose	108
7.2.1	Topologie déployée	108
7.2.2	Gestion des dépendances de démarrage	109
7.2.3	Mode développement contre mode production	109
7.3	Notre <i>pipeline</i> GitLab CI/CD	110
7.3.1	Trois étapes pour une livraison maîtrisée	110
7.3.2	Factorisation des travaux Docker	110
7.3.3	Construction sélective par service	110
7.3.4	Déploiement automatique sur la branche principale	111
7.3.5	Notre exécuteur GitLab	111
7.4	Externalisation des paramètres sensibles	111
7.5	Cycle de déploiement type d’une modification	112
	Conclusion	112
	Conclusion générale et perspectives	113
	Bibliographie	115
	Webographie	116

Liste des figures

1.1	Logo de la société Progress Engineering	4
2.4	Logo de Java 21 (LTS)	28
2.5	Logo de TypeScript 5.9	29
2.6	Logo de Python 3.11	29
2.7	Logo de Spring Boot 3	29
2.8	Logo de Spring Cloud (Gateway + Eureka)	30
2.9	Logo de Hibernate / Spring Data JPA	30
2.10	Logo de Spring Security + JWT	30
2.11	Logo de Apache Tika 2.9	31
2.12	Logo de Angular 21	31
2.13	Logo de Angular Material + PrimeNG	31
2.14	Logo de NgRx Signals	32
2.15	Logo de RxJS 7.8	32
2.16	Logo de LM Studio	32
2.17	Logo de Ollama	33
2.18	Logo de Mistral	33
2.19	Logo de Qwen2.5-VL-7B	33
2.20	Logo de LangChain 0.3	33
2.21	Logo de FastAPI 0.115	34
2.22	Logo de OpenCV 4.13	34
2.23	Logo de Tesseract OCR 5	34
2.24	Logo de MySQL 8.4	35
2.25	Logo de OpenSearch 2.13	35
2.26	Logo de Redis 7	35
2.27	Logo de MinIO	36
2.28	Logo de RabbitMQ 3	36
2.29	Logo de Docker + Compose	36
2.30	Logo de GitLab CI/CD	36

2.31	Logo de Nginx	37
2.32	Logo de IntelliJ IDEA	37
2.33	Logo de Visual Studio Code	37
2.34	Logo de PyCharm	38
2.35	Logo de Postman	38
2.36	Logo de Git + Maven + npm	38
3.2	Interface de connexion — Saisie de l’email	44
3.3	Interface de vérification OTP — Saisie du code à 6 chiffres	44
3.4	E-mail de réception du code OTP — vue depuis la boîte de réception	45
3.6	Interface de recherche	50
4.2	Interface de l’assistant conversationnel RAG	58
4.5	Interface d’ingestion de documents — Zone de glisser-déposer	65
5.3	Interface de gestion des alertes thématiques	73
5.5	Tableau de bord administrateur — KPIs et graphiques	78
5.6	Tableau de bord utilisateur — Historique et signets	78
5.7	Interface de gestion des utilisateurs	81
5.8	Interface de gestion des rôles et permissions	82
5.9	Interface de gestion du profil utilisateur	82
6.3	Fusion inter-pages d’un acte réparti sur deux pages du JORT.	99
6.4	Extraction complète d’un tableau de nominations occupant une page entière.	99
6.5	Tableau réparti sur deux pages : donnée intégrale mais structure linéarisée.	100
6.6	Segmentation d’une page mixte combinant plusieurs actes et un tableau.	101
6.7	Validation RAG : réponse correcte et sourcée à une question dont la donnée provient d’un tableau du JORT.	102

Liste des tableaux

1.1	Points forts et points faibles du portail JORT	7
1.2	Points forts et points faibles de Legislation.tn	8
1.3	Points forts et points faibles de Casetext / CoCounsel	9
1.4	Réponses de notre solution aux limites des plateformes existantes	10
2.1	Description des entités principales du portail juridique	18
2.2	Composition de l'équipe et attribution des rôles Scrum	20
2.3	Product Backlog global du portail juridique intelligent	21
2.4	Planification des releases du projet	25
2.5	Description des microservices du portail juridique	26
3.1	Backlog du Sprint 1 — Authentification	40
3.2	Tâches du Sprint 1	41
3.3	Cas de test — Sprint 1 : Authentification	45
3.4	Backlog du Sprint 2 — Moteur de recherche	47
3.5	Tâches du Sprint 2	47
3.6	API REST du search-service	50
3.7	Cas de test — Sprint 2 : Moteur de recherche	51
4.1	Backlog du Sprint 3 — Assistant RAG	54
4.2	Tâches du Sprint 3	55
4.3	API REST du chat-service	58
4.4	Cas de test — Sprint 3 : Assistant RAG	59
4.5	Backlog du Sprint 4 — Pipeline d'ingestion	60
4.6	Tâches du Sprint 4	61
4.7	API REST du ingest-service	64
4.8	Cas de test — Sprint 4 : Pipeline d'ingestion	66
5.1	Backlog du Sprint 5 — Notifications et alertes	68
5.2	Tâches du Sprint 5	69

5.3	API REST du notification-service	72
5.4	Cas de test — Sprint 5 : Notifications et alertes	73
5.5	Backlog du Sprint 6 — Tableau de bord analytique	74
5.6	Tâches du Sprint 6	75
5.7	API REST de l’analytics-service	77
5.8	Matrice d’affectation des permissions par rôle.	80
5.9	Points d’accès d’administration et permissions requises.	81
5.10	Cas de test — Sprint 6 : Tableau de bord analytique	83
5.11	Bilan global des sprints	84
6.1	Indicateurs d’audit que nous avons mesurés sur le corpus JORT 2025.	86
6.2	Métriques RAGAS que nous avons obtenues sur notre version V2.	89
6.3	Erreurs observées et règles ajoutées au fil de nos tests d’extraction.	93
6.4	Principaux champs extraits, leur signification et leur utilité pour la recherche juridique.	96
6.5	Synthèse de nos trois itérations successives.	103
7.1	Paramètres externalisés que nous avons identifiés.	111

Liste des abréviations

- **GLSI** = Génie Logiciel et Systèmes d'Information
- **UML** = Unified Modeling Language
- **HTTP** = Hypertext Transfer Protocol
- **HTTPS** = Hypertext Transfer Protocol Secure
- **API** = Application Programming Interface
- **REST** = Representational State Transfer
- **JWT** = JSON Web Token
- **JSON** = JavaScript Object Notation
- **XML** = eXtensible Markup Language
- **OTP** = One-Time Password
- **RAG** = Retrieval-Augmented Generation
- **LLM** = Large Language Model
- **NLP** = Natural Language Processing
- **OCR** = Optical Character Recognition
- **kNN** = k-Nearest Neighbors
- **BM25** = Best Match 25
- **RRF** = Reciprocal Rank Fusion
- **TF** = Term Frequency
- **IDF** = Inverse Document Frequency
- **RTL** = Right To Left
- **DTO** = Data Transfer Object
- **JPA** = Java Persistence API
- **ORM** = Object-Relational Mapping
- **CI** = Continuous Integration
- **CD** = Continuous Deployment
- **SOA** = Service-Oriented Architecture
- **SMTP** = Simple Mail Transfer Protocol
- **AMQP** = Advanced Message Queuing Protocol
- **S3** = Simple Storage Service
- **KPI** = Key Performance Indicator

- **PFE** = Projet de Fin d'Études
- **JORT** = Journal Officiel de la République Tunisienne
- **DI** = Dependency Injection
- **DAO** = Data Access Object
- **PDF** = Portable Document Format
- **IT** = Information Technology

Introduction générale

L'accès à l'information juridique constitue un pilier fondamental de tout État de droit. En Tunisie, le corpus législatif est vaste, en constante évolution, et rédigé dans deux langues — le français et l'arabe — ce qui complexifie davantage la recherche documentaire pour les professionnels du droit, les chercheurs, et les simples citoyens. La dispersion des textes de loi sur différentes plateformes institutionnelles, combinée à l'absence d'outils de recherche intelligents, engendre une perte de temps considérable et limite l'accès équitable à la justice.

L'essor des technologies d'intelligence artificielle, et plus particulièrement des grands modèles de langage (*Large Language Models* — LLM) et de la recherche sémantique par vecteurs denses, ouvre aujourd'hui de nouvelles perspectives pour rendre l'information juridique accessible, compréhensible et exploitable par tous. Ces technologies, couplées à une architecture logicielle moderne basée sur les microservices, permettent de concevoir des systèmes robustes, évolutifs et maintenables.

C'est dans ce contexte que s'inscrit le présent projet de fin d'études, réalisé au sein de l'École Supérieure Privée d'Ingénierie et de Technologie (TEK-UP) en vue de l'obtention du Diplôme National d'Ingénieur en spécialité Génie Logiciel et Systèmes d'Information. Le projet consiste en la conception et le développement d'un **portail juridique intelligent** dédié aux textes de loi tunisiens, reposant sur une architecture microservices et intégrant un moteur de recherche hybride ainsi qu'un assistant conversationnel basé sur le paradigme *Retrieval-Augmented Generation* (RAG).

Le système proposé s'articule autour de huit microservices indépendants orchestrés via Spring Boot 4.0.5 et Spring Cloud 2025.1.0, avec Eureka pour la découverte de services. La recherche documentaire exploite OpenSearch 2.13.0 en combinant la recherche lexicale BM25 et la recherche sémantique par vecteurs denses (kNN). L'assistant conversationnel utilise le modèle de langage Mistral via Ollama selon le paradigme RAG. L'interface utilisateur, développée avec Angular 21, propose une expérience bilingue français-arabe avec support RTL natif.

La gestion de ce projet a été conduite selon la méthodologie agile **Scrum**, décomposée en six sprints répartis sur trois releases successives.

Le présent rapport est organisé de la manière suivante :

- **Chapitre 1 — Cadre du projet** : Ce chapitre présente l'organisme d'accueil, le contexte et les objectifs du projet, l'étude critique de solutions existantes similaires, ainsi que la méthodologie agile Scrum et l'environnement de développement adoptés.
- **Chapitre 2 — Analyse et spécification des besoins** : Ce chapitre identifie les acteurs du système, détaille les besoins fonctionnels et non fonctionnels, présente le diagramme de classes global, l'architecture microservices, et établit la planification des releases et des sprints avec le diagramme de Gantt.
- **Chapitre 3 — Release 1 : Authentification et Moteur de Recherche** : Ce chapitre

couvre le Sprint 1 consacré à la gestion des comptes utilisateurs et à l'authentification OTP, ainsi que le Sprint 2 dédié au moteur de recherche hybride, aux suggestions et aux signets.

- **Chapitre 4 — Release 2 : Assistant RAG et Pipeline d'Ingestion** : Ce chapitre traite du Sprint 3, portant sur l'assistant conversationnel RAG basé sur Mistral, et du Sprint 4, consacré au pipeline d'ingestion documentaire avec OCR Apache Tika et stockage MinIO.
- **Chapitre 5 — Release 3 : Notifications, Alertes et Tableau de Bord** : Ce chapitre présente le Sprint 5, couvrant le service de notifications et d'alertes, et le Sprint 6, dédié au tableau de bord analytique et au module d'administration avancé.
- **Chapitre 6 — Pipeline d'extraction et d'indexation du corpus juridique** : Ce chapitre retrace l'évolution de notre chaîne d'extraction documentaire en trois itérations successives, depuis l'extraction texte classique jusqu'au modèle vision-langage, et détaille la stratégie d'indexation bilingue dans OpenSearch.
- **Chapitre 7 — Conteneurisation, déploiement et intégration continue** : Ce chapitre décrit la stratégie de conteneurisation Docker de l'ensemble des composants, l'orchestration via Docker Compose, et le pipeline d'intégration et de déploiement continu mis en place avec GitLab CI/CD.
- **Conclusion générale** : La conclusion dresse le bilan du travail réalisé, met en évidence les principales contributions et les résultats obtenus, puis expose les limites du système ainsi que les perspectives d'évolution envisagées pour le portail.

Cadre du projet

Introduction

Ce premier chapitre situe notre projet dans son cadre général. Nous commençons par présenter l'organisme d'accueil, puis nous exposons le contexte et les objectifs du projet. Nous conduisons ensuite une étude critique de trois solutions existantes dans le domaine de la recherche juridique en ligne. Enfin, nous présentons la méthodologie de gestion de projet Scrum adoptée, le langage de modélisation UML choisi, ainsi que l'environnement technologique de développement.

1.1 Cadre général du projet

1.1.1 Présentation de l'organisme d'accueil

1.1.1.1 Progress Engineering

Le présent projet de fin d'études a été réalisé au sein de **Progress Engineering**, une société de services informatiques (ESN) située au Parc Technologique El Ghazala (BP 9, Cedex 2088, Tunisie). Fondée en 2000 par trois ingénieurs, l'entreprise s'est spécialisée dans une large gamme de prestations de développement d'applications Web et mobiles, ainsi que dans le conseil et le développement informatique. Elle s'appuie aujourd'hui sur une équipe de développeurs qualifiés (codeurs, concepteurs, etc.) disposant d'une vaste expérience dans la conception d'applications Web et mobiles de haute qualité.



Figure 1.1 : Logo de la société Progress Engineering

Les services proposés couvrent l'ensemble du cycle de vie d'un projet informatique : conseil, développement logiciel et Web, test, développement de solutions SMS et MMS, maintenance et support. Parmi les produits phares conçus par la société, on peut citer :

- **ORegMX** : système d'inscription universitaire à distance ;
- **SMSLink** : plateforme SMS multi-opérateurs ;
- **OneSign** : serveur de signature électronique.

En 2007, Progress Engineering a remporté, en partenariat avec le Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de la Technologie, le deuxième prix e-gov dans le cadre du concours *Tunisian Best e-Contents' 2007*, pour la réalisation du système d'inscription universitaire en ligne **ORegMx** (<https://www.inscription.tn>). La société est également à l'origine de l'application *eHowwiya*. Son site officiel est accessible à l'adresse <https://www.progress.com.tn/>.

1.1.1.2 Spécialité Génie Logiciel et Systèmes d’Information

La filière **Génie Logiciel et Systèmes d’Information** (GLSI) prépare des ingénieurs capables de concevoir, développer et maintenir des systèmes logiciels complexes. La formation couvre notamment :

- Les architectures logicielles modernes (microservices, SOA, cloud-native).
- Le développement full-stack (backend Java/Spring, frontend Angular/React).
- La gestion de bases de données relationnelles et NoSQL.
- L’intelligence artificielle appliquée au traitement du langage naturel.
- Les méthodes agiles de gestion de projet (Scrum, Kanban).

1.1.2 Contexte et objectifs du projet

1.1.2.1 Contexte

Le système juridique tunisien produit chaque année des centaines de textes législatifs et réglementaires (lois organiques, décrets-lois, arrêtés ministériels, circulaires), publiés principalement dans le Journal Officiel de la République Tunisienne (JORT). Ces textes, rédigés en arabe et parfois en français, sont aujourd’hui difficilement accessibles : les moteurs de recherche des portails officiels sont rudimentaires, ne supportent pas la recherche sémantique, et ne permettent aucune interaction conversationnelle avec le corpus.

Par ailleurs, la numérisation des documents juridiques tunisiens est partielle : une large portion est disponible uniquement sous forme de PDF scannés, nécessitant une reconnaissance optique de caractères (OCR) avant toute indexation. Face à ces lacunes, le présent projet propose une solution complète intégrant :

- Un pipeline d’ingestion documentaire basé sur Apache Tika et Tesseract OCR.
- Un moteur de recherche hybride (BM25 lexical + kNN sémantique) via OpenSearch.
- Un assistant conversationnel basé sur le paradigme RAG avec le LLM Mistral.
- Une interface bilingue français-arabe avec support RTL.
- Une architecture microservices garantissant évolutivité et maintenabilité.

1.1.2.2 Objectifs du projet

L’objectif général de ce projet est de concevoir et de développer un portail juridique intelligent qui rende le corpus du Journal Officiel réellement accessible, compréhensible et exploitable par les citoyens, les professionnels du droit et les chercheurs. Cet objectif se décline en plusieurs objectifs spécifiques, exprimés en termes de livrables fonctionnels :

- **Constituer un fonds documentaire structuré** : extraire, découper et indexer les textes du JORT (PDF natifs et scannés) afin de disposer d’une base juridique bilingue exploitable.
- **Offrir une recherche hybride pertinente** : combiner la recherche lexicale (BM25) et la recherche sémantique vectorielle, complétées par le filtrage à facettes, les suggestions de complétion et l’export des résultats.
- **Fournir un assistant conversationnel** : permettre à l’utilisateur d’interroger le corpus en langage naturel au moyen d’un assistant RAG citant systématiquement ses sources.
- **Enrichir l’expérience utilisateur** : proposer des services complémentaires — signets, historique de recherche, alertes thématiques et tableau de bord analytique.
- **Livrer une solution évolutive** : fournir un socle logiciel déployable de manière reproductible, capable d’accueillir de nouveaux corpus juridiques.

Les exigences de qualité associées (performance, disponibilité, sécurité, scalabilité, maintenabilité) ne constituent pas des objectifs à part entière : elles sont traitées comme *besoins non fonctionnels* dans le chapitre consacré à l’analyse et à la spécification des besoins.

1.2 Étude de l’existant

L’étude de l’existant est une étape indispensable qui nous permet d’identifier les forces et les faiblesses des solutions actuelles, et d’en déduire les axes d’amélioration à intégrer dans notre projet. Nous analysons ci-après trois plateformes de référence dans le domaine de la recherche juridique en ligne.

- Le portail **JORT en ligne** (Journal Officiel de la République Tunisienne)
- La plateforme **Legislation.tn**
- Le moteur **Casetext** (international, basé sur l’IA)

1.2.1 Étude du portail JORT en ligne

1.2.1.1 Description

Le Journal Officiel de la République Tunisienne (JORT) est la publication officielle de l'État tunisien qui contient tous les textes législatifs et réglementaires promulgués. Son portail en ligne permet de consulter et de télécharger les numéros du JORT en format PDF depuis 1956 jusqu'à nos jours [W12]. C'est la source primaire de référence pour tout professionnel du droit tunisien.

1.2.1.2 Étude fonctionnelle

- **Visiteur/Citoyen :**
 - Rechercher un numéro du JORT par date ou numéro.
 - Télécharger le PDF du journal officiel.
 - Consulter les suppléments et éditions spéciales.

1.2.1.3 Points forts et points faibles

Le tableau ci-dessous synthétise les points forts et les points faibles de cette plateforme, que nous avons identifiés au cours de notre étude.

Tableau 1.1 : Points forts et points faibles du portail JORT

Points forts	Points faibles
• Source officielle et exhaustive. • Archives complètes depuis 1956. • Accès gratuit et ouvert.	• Recherche uniquement par date/numéro, pas par contenu. • Pas de recherche sémantique ni lexicale. • Documents en PDF scannés, non indexés. • Interface vieillissante, non responsive. • Pas d'assistant conversationnel. • Pas de support RTL cohérent.

1.2.2 Étude de la plateforme Legislation.tn

1.2.2.1 Description

Legislation.tn est une plateforme privée tunisienne proposant une base de données de textes juridiques tunisiens. Elle offre une interface de recherche par mots-clés dans les titres des textes, avec possibilité de filtrage par type de texte et par date de promulgation.

1.2.2.2 Étude fonctionnelle

- **Utilisateur authentifié :**
 - Rechercher des textes par mots-clés dans le titre.
 - Filtrer par type (loi, décret, arrêté, circulaire).
 - Filtrer par plage de dates.
 - Consulter le texte intégral en ligne.
 - Télécharger les textes en PDF.

1.2.2.3 Points forts et points faibles

Le tableau ci-dessous synthétise les points forts et les points faibles de cette plateforme, que nous avons identifiés au cours de notre étude.

Tableau 1.2 : Points forts et points faibles de Legislation.tn

Points forts	Points faibles
• Base de données structurée. • Recherche par mots-clés dans le titre. • Filtres par type et date. • Textes intégraux disponibles.	• Pas de recherche sémantique vectorielle. • Recherche limitée aux titres, pas au contenu. • Plateforme payante (abonnement). • Pas d'assistant conversationnel. • Couverture incomplète du corpus. • Pas de gestion de signets/historique.

1.2.3 Étude de la plateforme Casetext

1.2.3.1 Description

Casetext est une plateforme américaine de recherche juridique basée sur l'intelligence artificielle, récemment rachetée par Thomson Reuters. Son assistant CoCounsel utilise GPT-4 pour répondre aux questions juridiques en citant des sources issues de la jurisprudence et de la législation américaine. C'est l'une des premières applications commerciales du paradigme RAG dans le domaine juridique.

1.2.3.2 Étude fonctionnelle

- **Avocat/Juriste :**
 - Poser des questions juridiques en langage naturel.
 - Obtenir des réponses avec citations de sources vérifiées.
 - Rechercher dans la jurisprudence par embeddings sémantiques.
 - Rédiger des mémos juridiques assistés par IA.
 - Analyser des contrats et identifier les clauses problématiques.

1.2.3.3 Points forts et points faibles

Le tableau ci-dessous synthétise les points forts et les points faibles de cette plateforme, que nous avons identifiés au cours de notre étude.

Tableau 1.3 : Points forts et points faibles de Casetext / CoCounsel

Points forts	Points faibles
• Recherche sémantique avancée par LLM. • Interface conversationnelle intuitive. • Citations des sources automatiques. • Corpus juridique américain complet.	• Uniquement en anglais, corpus américain. • Très coûteux (abonnement professionnel). • Pas applicable au droit tunisien. • Pas de support arabe/RTL. • Dépendant d’APIs propriétaires (OpenAI).

1.3 Problématique

L’analyse des trois plateformes existantes met en évidence les manques suivants qu’aucune solution actuelle ne comble simultanément pour le contexte tunisien :

- Recherche sémantique vectorielle combinée à la recherche lexicale sur le corpus juridique tunisien.
- Assistant conversationnel RAG en français et arabe avec citation des sources.
- Pipeline d’ingestion automatisé pour les PDF scannés via OCR.
- Interface bilingue français-arabe avec support RTL natif.
- Gratuité d’accès pour tous les citoyens.
- Architecture évolutive capable d’accueillir de nouveaux services.

Notre portail juridique intelligent se positionne comme la première solution open-source dédiée au corpus juridique tunisien intégrant toutes ces fonctionnalités dans une architecture microservices moderne.

1.4 Solution proposée

Au regard des limites identifiées lors de l'étude de l'existant, et afin de répondre à la problématique posée, nous proposons un **portail juridique intelligent** : une plateforme open-source qui rend le corpus du Journal Officiel tunisien réellement accessible, compréhensible et fiable. Notre démarche part directement des points faibles relevés chez les trois plateformes étudiées : à chacun, nous apportons une réponse concrète, comme le résume le tableau 1.4.

Tableau 1.4 : Réponses de notre solution aux limites des plateformes existantes

Limite des solutions existantes	Réponse apportée par notre solution
Recherche limitée aux mots-clés exacts, sans compréhension du sens (JORT en ligne, Legislation.tn).	Moteur de recherche hybride combinant BM25 (lexical) et kNN (sémantique), fusionnés par l'algorithme RRF.
Aucun assistant capable de répondre à une question juridique en citant ses sources.	Assistant conversationnel RAG qui répond en langage naturel uniquement à partir des textes officiels, avec des sources vérifiables.
Numérisation partielle : de nombreux PDF scannés restent non indexables.	Pipeline d'ingestion associant OCR (Tesseract) et extraction par modèle vision-langage (Qwen2.5-VL), qui restitue la structure réelle des documents.
Outils avancés réservés à l'anglais et inadaptés au droit tunisien (Casetext).	Solution bilingue arabe-français avec support RTL natif, dédiée au corpus juridique tunisien.
Solutions propriétaires et coûteuses.	Plateforme open-source et gratuite , hébergeant ses modèles d'IA en local pour garantir la souveraineté des données juridiques.
Architectures peu extensibles.	Architecture microservices modulaire et évolutive, conçue pour accueillir de nouveaux services.

Sur le plan technique, cette solution se concrétise par quatre briques complémentaires : un **pipeline d'extraction et d'indexation** du corpus, un **moteur de recherche hybride bilingue** fondé sur OpenSearch, un **assistant RAG souverain** s'appuyant sur le LLM Mistral exécuté localement, et une **interface web bilingue** avec support RTL. L'ensemble est déployé sur une architecture microservices industrialisée (Docker, CI/CD GitLab). Les chapitres suivants détaillent la conception puis la réalisation de chacune de ces briques.

1.5 Méthodologie de gestion de projet

1.5.1 Méthodologie Scrum

La méthodologie **Scrum** [B2][B1] est un cadre agile de gestion de projet largement adoptée dans l'industrie du logiciel. Elle repose sur des cycles de développement courts et itératifs appelés **Sprints**, généralement de 2 à 4 semaines, au terme desquels un incrément fonctionnel du produit est livré.

Nous avons choisi la méthodologie Scrum pour les raisons suivantes :

- **Adaptabilité** : Les besoins d'un portail juridique sont susceptibles d'évoluer en cours de projet ; Scrum permet d'intégrer ces changements sans déstabiliser l'ensemble.
- **Visibilité** : Le *backlog* produit offre une vision claire de l'état d'avancement à tout moment.
- **Livraison continue** : Chaque sprint produit une version fonctionnelle et testable du logiciel.
- **Qualité** : La *Definition of Done* (DoD) garantit que chaque fonctionnalité est développée, testée et documentée avant d'être intégrée.

1.5.2 Rôles Scrum

- **Product Owner** : Responsable de la valeur du produit, il définit et priorise le *Product Backlog* et représente les parties prenantes auprès de l'équipe.
- **Scrum Master** : Garant de la bonne application de la méthodologie Scrum, il facilite les cérémonies et lève les obstacles rencontrés par l'équipe.
- **Équipe de développement** : Équipe pluridisciplinaire et auto-organisée, chargée de concevoir, développer, tester et livrer un incrément fonctionnel à la fin de chaque sprint.

1.5.3 Cérémonies Scrum

- **Sprint Planning** : Réunion de planification au début de chaque sprint pour sélectionner les *user stories* du *Product Backlog* à réaliser.
- **Daily Scrum** : Point quotidien de 15 minutes sur l'avancement, les obstacles et le plan du jour.
- **Sprint Review** : Démonstration des fonctionnalités développées au Product Owner à la fin de chaque sprint.
- **Sprint Retrospective** : Analyse des points positifs et négatifs du sprint écoulé pour améliorer le processus.

Conclusion

Dans ce premier chapitre, nous avons présenté le cadre général du projet : l'organisme d'accueil, le contexte et les objectifs, une étude critique des solutions existantes, la problématique qui en découle, et la méthodologie Scrum adoptée. Nous passons maintenant au chapitre suivant consacré à l'analyse et à la spécification des besoins du système.

Analyse et spécification des besoins

Introduction

Ce chapitre présente l'analyse et la spécification des besoins du portail juridique intelligent. Nous commençons par identifier les acteurs du système et leurs besoins fonctionnels et non fonctionnels. Nous présentons ensuite le diagramme de cas d'utilisation global, le modèle de données (diagramme de classes), l'architecture microservices retenue, et enfin la planification Scrum complète avec le *Product Backlog* global et le diagramme de Gantt.

2.1 Spécification des besoins

2.1.1 Identification des acteurs

Le portail juridique intelligent implique cinq catégories d'acteurs principaux, organisés selon une hiérarchie de permissions correspondant aux cinq rôles RBAC du système (USER, JURIST, MANAGER, ANALYSTE, ADMIN) :

- **Citoyen (USER)** : Tout utilisateur authentifié disposant d'un compte de base. Il peut accéder à la recherche hybride, consulter les détails des textes juridiques, gérer ses signets et son historique, et interagir avec l'assistant conversationnel RAG.
- **Juriste (JURIST)** : Professionnel du droit bénéficiant de permissions étendues : en plus des fonctions du citoyen, il peut déposer des documents juridiques, créer des alertes thématiques, exporter en JSON/XML et consulter les retours RAG.
- **Gestionnaire (MANAGER)** : Responsable de la gestion documentaire : en plus des fonctions du juriste, il peut modérer et supprimer des documents, gérer les retours RAG et consulter la liste des utilisateurs.
- **Analyste (ANALYSTE)** : Profil orienté supervision : en plus des fonctions du citoyen, il accède au tableau de bord analytique (KPIs, statistiques) et à la consultation des retours RAG, sans droit de dépôt documentaire.
- **Administrateur (ADMIN)** : Superutilisateur qui gère l'ensemble de la plateforme : en plus des fonctions précédentes, il gère les comptes utilisateurs (CRUD), les rôles et permissions, et consulte le journal d'audit.

2.1.2 Spécification des besoins fonctionnels

2.1.2.1 Besoins fonctionnels par acteur

- **Citoyen** :
 - S'inscrire avec email et mot de passe.
 - Se connecter avec email + mot de passe (avec OTP en second facteur optionnel).
 - Gérer son profil (consulter et modifier ses informations).

- Rechercher des textes juridiques par mots-clés (mode lexical BM25).
- Rechercher des textes juridiques par similarité sémantique (mode vectoriel kNN).
- Effectuer une recherche hybride (BM25 + kNN combinés).
- Obtenir des suggestions de complétion automatique.
- Consulter le détail complet d'un texte juridique.
- Ajouter/supprimer des signets sur des textes.
- Consulter son historique de recherches.
- Ouvrir une session de chat avec l'assistant juridique RAG.
- Envoyer des questions en langage naturel à l'assistant.
- Recevoir des réponses avec citations des sources juridiques.
- Évaluer la qualité des réponses de l'assistant (pouce haut/bas).
- Consulter ses sessions de chat et l'historique des messages.
- Créer et gérer des alertes thématiques.
- Recevoir des notifications par email lors d'un nouvel index documentaire.
- **Juriste (en sus du Citoyen) :**
 - Exporter les résultats de recherche en format JSON ou XML.
 - Accéder à des filtres de recherche avancés (type, date, juridiction).
 - Créer des annotations sur les textes juridiques.
- **Administrateur :**
 - Téléverser des documents PDF/Word pour ingestion.
 - Suivre l'état du pipeline d'ingestion (en cours, réussi, échoué).
 - Gérer les comptes utilisateurs (liste, activation/désactivation).
 - Consulter le tableau de bord analytique (KPIs, top requêtes, feedback).
 - Modérer les retours utilisateurs sur les réponses RAG.
 - Consulter les journaux d'audit du système.
 - Configurer les paramètres des alertes et notifications.

2.1.3 Spécification des besoins non fonctionnels

- **Performance :** Le temps de réponse d'une recherche hybride ne doit pas dépasser 2 secondes pour 95% des requêtes. L'assistant RAG doit fournir une première réponse en moins de 10 secondes.
- **Disponibilité :** Le système doit garantir une disponibilité de 99.5% en dehors des fenêtres de maintenance planifiées.
- **Sécurité :** Toutes les communications sont chiffrées via HTTPS. L'authentification repose sur

le couple email + mot de passe (haché en bcrypt), avec un OTP en second facteur optionnel, et délivre des tokens JWT à durée de vie limitée (1 heure) avec blacklist Redis et refresh token.

- **Scalabilité** : L'architecture microservices permet le déploiement horizontal indépendant de chaque service. OpenSearch supporte l'ajout de nœuds de données sans interruption.
- **Maintenabilité** : Chaque microservice est indépendant avec son propre dépôt Git, sa base de données et son pipeline CI/CD. Le couplage entre services est assuré uniquement via API REST et messages asynchrones RabbitMQ.
- **Interopérabilité** : Les APIs sont documentées via OpenAPI 3.0 (Swagger). Les exports sont disponibles en JSON et XML.
- **Accessibilité** : L'interface supporte le bilinguisme français-arabe avec permutation dynamique et support RTL complet pour la langue arabe.

2.2 Diagramme de cas d'utilisation global

La figure 2.1 présente le diagramme de cas d'utilisation global du portail juridique intelligent, regroupant l'ensemble des interactions entre les acteurs et le système.

Les cinq acteurs humains sont organisés en une **hiérarchie de généralisation** : chaque acteur hérite des cas d'utilisation de son parent et en ajoute de nouveaux. Le **Citoyen** constitue l'acteur de base (authentification, recherche hybride, gestion des signets, dialogue avec l'assistant RAG et consultation de l'historique). Le **Juriste** en hérite et ajoute le dépôt de documents JORT et la configuration d'alertes thématiques. Le **Gestionnaire** étend le juriste avec la modération documentaire et la gestion des retours RAG. L'**Administrateur** dispose en outre de la gestion des utilisateurs, des rôles et permissions, et de l'audit. Enfin, l'**Analyste** hérite directement du citoyen et se spécialise dans la supervision analytique (tableau de bord, statistiques), sans droit de dépôt documentaire. Toute fonctionnalité exigeant une authentification préalable, le cas « *S'authentifier* » est placé à droite et partagé par l'ensemble des cas d'utilisation au moyen de la relation « *include* » ; le cas « *Consulter les sources citées* » étend quant à lui, de façon optionnelle, le dialogue avec l'assistant RAG (relation « *extend* »). Les relations « *extend* » (cas optionnels tels que la validation OTP ou le filtrage de recherche) sont détaillées dans les diagrammes de cas d'utilisation propres à chaque sprint.

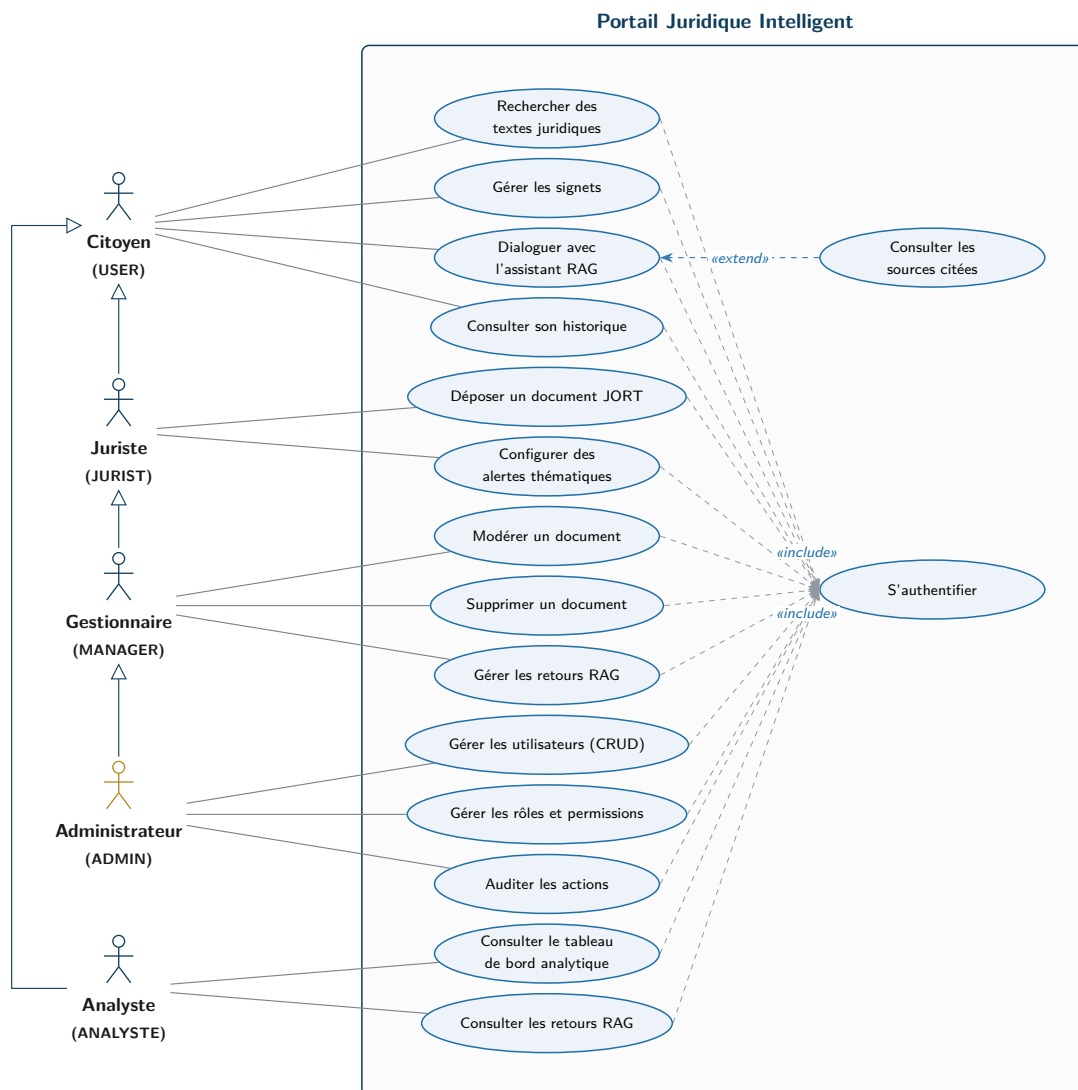


Figure 2.1 : Diagramme de cas d'utilisation global du portail juridique

2.3 Modèle de données

2.3.1 Diagramme de classes global

La figure 2.2 présente les entités principales du domaine métier et leurs relations, telles qu'elles sont réellement définies dans les microservices.

Le modèle s'organise autour de l'entité **UserAccount**, qui joue le rôle de pivot (*hub*). Le rattachement d'un utilisateur à ses données n'est pas dupliqué sous forme d'attribut **userId** dans chaque table : il est porté directement par des **associations un-à-plusieurs**. Chaque utilisateur possède ainsi ses signets (**Bookmark**), son historique de recherche (**SearchHistory**), ses sessions de chat (**ChatSession**), ses alertes (**Alert**), ses traces d'audit (**AuditLog**) et ses retours (**RagFeedback**). La relation entre **ChatSession** et **ChatMessage** est une **composition** (losange plein) : un message n'existe pas en dehors de sa session, d'où une cardinalité toujours égale à 1 du côté composite. Le contrôle d'accès s'appuie sur les entités **Role** et **Permission** :

chaque compte est rattaché à *un* rôle (**UserAccount** 0..* — 1 **Role**), et chaque rôle agrège *plusieurs* permissions (**Role** 1 — 0..* **Permission**). Enfin, trois énumérations (**SearchMode**, **MessageRole**, **FeedbackRating**) typent explicitement les attributs concernés, garantissant l'intégrité des valeurs métier.

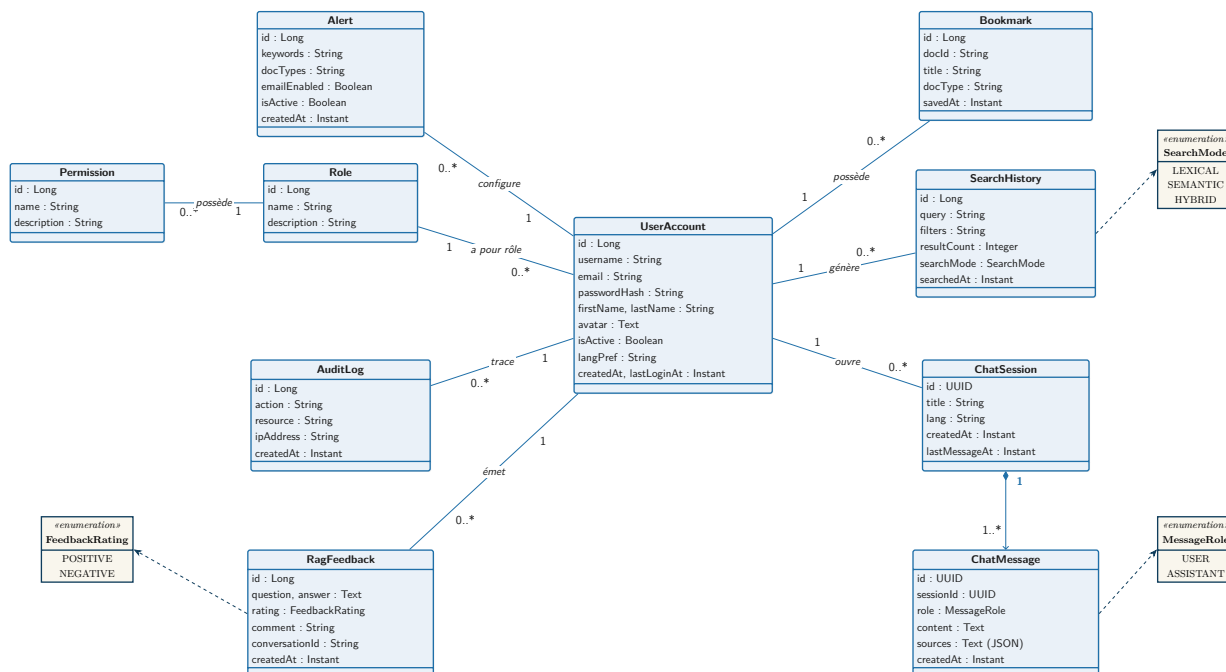


Figure 2.2 : Diagramme de classes du domaine métier du portail juridique

2.3.2 Description des entités principales

Le tableau ci-dessous décrit les entités principales du modèle de données, leurs attributs essentiels et leur rôle dans le système.

Tableau 2.1 : Description des entités principales du portail juridique

Entité	Attributs principaux	Description
UserAccount	id, email, passwordHash, firstName, lastName, avatar, role, enabled, createdAt	Compte utilisateur rattaché à un rôle RBAC (ADMIN, MANAGER, JURIST, ANALYSTE, USER). Authentification par email + mot de passe haché bcrypt, avec OTP en second facteur optionnel ; champs de profil dont l'avatar.
Role	id, name, description	Rôle RBAC persisté en base, associé à un ensemble de permissions.

Entité	Attributs principaux	Description
Permission	id, name, description	Permission atomique (ex. : <code>USER_READ</code> , <code>DOCUMENT_UPLOAD</code>) rattachée à un rôle via une association <i>un-à-plusieurs</i> (un rôle agrège plusieurs permissions).
OtpToken	id, email, code (6 chiffres), expiresAt, used	Token OTP à usage unique, valide 10 minutes, envoyé par email.
LegalDocument	id (UUID), title, content, type, datePublication, source, language	Document juridique indexé dans OpenSearch. Inclut le vecteur d'embedding dense (768 dimensions).
Bookmark	id, documentId, createdAt	Signet créé par un utilisateur sur un document.
SearchHistory	id, query, searchMode, resultCount, duration, createdAt	Entrée d'historique pour chaque recherche effectuée.
ChatSession	id (UUID), title, language, createdAt	Session de conversation avec l'assistant RAG.
ChatMessage	id, sessionId, role (USER/ASSISTANT), content, sources, createdAt	Message dans une session. Les sources citent les documents pertinents.
RagFeedback	id, messageId, rating (UP/DOWN), comment, createdAt	Évaluation de la qualité d'une réponse RAG par l'utilisateur.
Alert	id, keyword, email, active, createdAt	Alerte thématique : notifie l'utilisateur quand un nouveau document correspond au mot-clé.
AuditLog	id, action, resource, details, ipAddress, createdAt	Journal d'audit des actions administratives.

2.3.3 Schéma de la base de données OpenSearch

L'index OpenSearch `legal-documents` est défini avec le mapping suivant :

- **title** : champ `text` avec analyzer BM25 (fr/ar).
- **content** : champ `text` avec analyzer BM25 (fr/ar), stocké pour le RAG.
- **type** : champ `keyword` (LAW, DECREE, ORDER, CIRCULAR).
- **datePublication** : champ `date` ISO-8601.
- **language** : champ `keyword` (fr, ar).
- **embedding** : champ `knn_vector` de dimension 768 (modèle `multilingual-e5-base`), espace L2.
- **source** : champ `keyword` (JORT, legislation.tn, upload).
- **s3Key** : champ `keyword`, clé du fichier original dans MinIO.

2.4 Pilotage du projet avec Scrum

2.4.1 Équipe et rôles du projet

Le projet a été piloté selon la méthodologie Scrum présentée au chapitre précédent. Dans le cadre de ce projet de fin d'études, l'organisation a été adaptée à un contexte mono-développeur encadré : le tableau 2.2 précise l'attribution concrète des rôles.

Tableau 2.2 : Composition de l'équipe et attribution des rôles Scrum

Rôle Scrum	Affectation
Product Owner	M. Mohamed Sadok Mouha (encadrant professionnel) et M ^{me} Siwar Zrelli (encadrante académique), qui expriment et priorisent les besoins.
Scrum Master	Moetez BENFREDJ, garant de l'application de la méthode et de la tenue des cérémonies.
Équipe de développement	Moetez BENFREDJ, assurant l'ensemble des rôles techniques : <i>backend</i> Spring Boot, <i>frontend</i> Angular, agent IA Python, et DevOps (Docker, CI/CD).

2.4.2 Product Backlog global

Le *Product Backlog* regroupe l'**ensemble des *user stories* du projet** — les mêmes que celles détaillées dans le backlog de chaque sprint aux chapitres suivants —, organisées par sprint et priorisées (1 = plus haute). Chaque story est exprimée du point de vue d'un **acteur** (visiteur, utilisateur, juriste ou administrateur). Le tableau 2.3 les présente sur une pleine page pour rester lisible.

Tableau 2.3 : Product Backlog global du portail juridique intelligent

Id	Description	User Story	Prio.	Estim. (j)
Sprint 1 — Authentification et gestion des comptes (total : 22 j)				
1	Inscription utilisateur	En tant que visiteur, je veux m'inscrire avec mon email et mon nom afin de créer un compte sur le portail.	1	3
2	Envoi du code OTP	En tant que visiteur, je veux recevoir un code OTP à 6 chiffres par email afin de valider mon identité.	1	3
3	Vérification OTP + JWT	En tant que visiteur, je veux soumettre le code OTP reçu afin de finaliser ma connexion et recevoir un token JWT.	1	4
4	Consultation du profil	En tant qu'utilisateur connecté, je veux consulter mon profil afin de vérifier mes informations.	2	2
5	Modification du profil	En tant qu'utilisateur connecté, je veux modifier mon profil (nom, langue préférée) afin de personnaliser mon expérience.	2	3
6	Attribution d'un rôle	En tant qu'administrateur, je veux attribuer le rôle JURIST à un utilisateur afin de lui accorder des permissions étendues.	2	3
7	Révocation de session	En tant qu'utilisateur, je veux que mon jeton soit révoqué à la déconnexion (blacklist Redis) afin de sécuriser ma session.	1	4
Sprint 2 — Moteur de recherche hybride (total : 22 j)				
8	Recherche lexicale (BM25)	En tant qu'utilisateur, je veux rechercher des textes par mots-clés (mode BM25) afin de trouver des lois contenant des termes précis.	1	4

Id	Description	User Story	Prio.	Estim. (j)
9	Recherche sémantique (kNN)	En tant qu'utilisateur, je veux effectuer une recherche sémantique (mode kNN) afin de retrouver des textes de même signification.	1	4
10	Recherche hybride	En tant qu'utilisateur, je veux effectuer une recherche hybride (BM25+kNN) afin d'obtenir les résultats les plus pertinents.	1	3
11	Auto-complétion	En tant qu'utilisateur, je veux voir des suggestions de complétion automatique en tapant ma requête.	2	3
12	Ajout d'un signet	En tant qu'utilisateur, je veux ajouter un texte juridique en signet afin de le retrouver facilement.	2	2
13	Consultation des signets	En tant qu'utilisateur, je veux consulter mes signets depuis mon tableau de bord.	2	2
14	Suppression d'un signet	En tant qu'utilisateur, je veux supprimer un signet dont je n'ai plus besoin.	2	2
15	Export des résultats	En tant que juriste, je veux exporter les résultats de recherche en JSON ou XML afin de les utiliser dans mes outils.	3	2
Sprint 3 — Assistant conversationnel RAG (total : 19 j)				
16	Création de session	En tant qu'utilisateur, je veux créer une nouvelle session de chat avec l'assistant afin d'obtenir des réponses juridiques personnalisées.	1	3
17	Question RAG sourcée	En tant qu'utilisateur, je veux envoyer une question en langage naturel et recevoir une réponse fondée sur les textes de loi.	1	5
18	Citation des sources	En tant qu'utilisateur, je veux que les réponses de l'assistant citent les articles et textes juridiques sources.	1	3
19	Historique des sessions	En tant qu'utilisateur, je veux consulter l'historique de mes sessions de chat précédentes.	2	3
20	Évaluation des réponses	En tant qu'utilisateur, je veux évaluer chaque réponse de l'assistant (pouce haut/bas) afin d'améliorer sa qualité.	2	3

Id	Description	User Story	Prio.	Estim. (j)
21	Choix de la langue	En tant qu'utilisateur, je veux choisir la langue de conversation (français ou arabe) afin de dialoguer dans ma langue préférée.	2	2
Sprint 4 — Pipeline d'ingestion documentaire (total : 20 j)				
22	Téléversement de documents	En tant qu'admin, je veux téléverser un ou plusieurs fichiers PDF/Word afin d'enrichir le corpus juridique.	1	3
23	Archivage des originaux	En tant qu'administrateur, je veux que les fichiers téléversés soient archivés en stockage objet (MinIO/S3) afin de conserver les originaux.	1	2
24	Extraction de texte	En tant qu'administrateur, je veux que le texte des PDF numériques soit extrait (Apache Tika) afin de préparer l'indexation.	1	3
25	OCR des PDF scannés	En tant qu'administrateur, je veux que les PDF scannés soient océrisés (Tesseract OCR, fr+ar) afin d'en récupérer le texte.	1	3
26	Découpage en segments	En tant qu'administrateur, je veux que le texte extrait soit découpé en segments (chunks) afin d'améliorer la pertinence des résultats.	1	2
27	Génération d'embeddings	En tant qu'administrateur, je veux que des embeddings soient générés pour chaque segment afin d'activer la recherche sémantique.	1	3
28	Indexation OpenSearch	En tant qu'administrateur, je veux que les segments soient indexés dans OpenSearch afin de rendre les documents recherchables.	1	2
29	Suivi de l'ingestion	En tant qu'admin, je veux suivre l'état d'avancement du pipeline (en cours, réussi, échoué) par document.	2	2
Sprint 5 — Notifications, alertes et administration (total : 21 j)				
30	Création d'alerte	En tant qu'utilisateur, je veux créer une alerte thématique avec un ou plusieurs mots-clés afin d'être notifié des nouvelles lois pertinentes.	2	3
31	Activation d'alerte	En tant qu'utilisateur, je veux activer une alerte existante afin de réactiver mes notifications.	2	2

Id	Description	User Story	Prio.	Estim. (j)
32	Désactivation d'alerte	En tant qu'utilisateur, je veux désactiver une alerte sans la supprimer afin de suspendre mes notifications.	2	2
33	Suppression d'alerte	En tant qu'utilisateur, je veux supprimer une alerte dont je n'ai plus besoin.	2	2
34	Déclenchement d'alerte	En tant qu'utilisateur abonné, je veux que mes alertes soient déclenchées dès qu'un nouveau document correspond à mes mots-clés afin d'être tenu informé.	1	3
35	Email de notification	En tant qu'utilisateur abonné, je veux recevoir un email de notification personnalisé à chaque nouveau document correspondant à mon alerte.	1	3
36	Liste des utilisateurs	En tant qu'admin, je veux consulter la liste de tous les utilisateurs avec leur rôle et statut.	2	2
37	Activation de compte	En tant qu'admin, je veux activer un compte utilisateur afin d'autoriser son accès à la plateforme.	2	2
38	Désactivation de compte	En tant qu'admin, je veux désactiver un compte utilisateur afin de bloquer son accès à la plateforme.	2	2
Sprint 6 — Tableau de bord analytique (total : 19 j)				
39	KPIs globaux	En tant qu'admin, je veux consulter des KPIs globaux (utilisateurs, recherches, documents, sessions chat) afin d'avoir une vue d'ensemble de la plateforme.	1	3
40	Graphique des recherches	En tant qu'admin, je veux consulter un graphique de l'évolution des recherches par jour sur les 30 derniers jours.	2	3
41	Top des requêtes	En tant qu'admin, je veux consulter le top 10 des requêtes les plus fréquentes afin d'identifier les besoins des utilisateurs.	2	2
42	Retours RAG	En tant qu'admin, je veux consulter les retours RAG (positifs/négatifs) avec les commentaires afin d'améliorer l'assistant.	2	3

Id	Description	User Story	Prio.	Estim. (j)
43	Enregistrement historique	En tant qu'utilisateur, je veux que mes recherches soient enregistrées dans mon historique afin de pouvoir les retrouver et les relancer.	1	2
44	Consultation historique	En tant qu'utilisateur, je veux consulter mon historique de recherches afin de retrouver mes requêtes passées.	2	2
45	Relance de recherche	En tant qu'utilisateur, je veux relancer une recherche depuis mon historique afin de gagner du temps.	2	2
46	Journaux d'audit	En tant qu'admin, je veux consulter les journaux d'audit des actions administratives pour la traçabilité.	3	2
Total projet				123

Au total, le projet compte **46 *user stories*** pour une charge globale de **123 jours-homme**, soit une moyenne de **19 à 22 jours par sprint** (environ trois semaines de travail chacun). Le détail de chaque sprint est repris à l'identique dans le backlog du chapitre correspondant.

2.4.3 Planification des releases

Le tableau ci-dessous synthétise la planification des trois releases du projet, avec les sprints qui les composent et les fonctionnalités livrées.

Tableau 2.4 : Planification des releases du projet

Release	Sprints	Thème	Livrables
Release 1	Sprint 1 + Sprint 2	Authentification & Recherche	auth-service, search-service, interface de connexion et de recherche
Release 2	Sprint 3 + Sprint 4	Assistant RAG & Ingestion	chat-service, ingest-service, interface de chat et d'upload
Release 3	Sprint 5 + Sprint 6	Notifications & Analytics	notification-service, analytics-service, dashboard admin

2.5 Architecture du système

2.5.1 Vue d'ensemble de l'architecture microservices

Le portail juridique intelligent repose sur une architecture microservices [B3] composée de **huit services indépendants**, chacun responsable d'un domaine fonctionnel précis. Tous les services communiquent via le **Gateway** (Spring Cloud Gateway) [W2] qui sert de point d'entrée unique pour les clients.

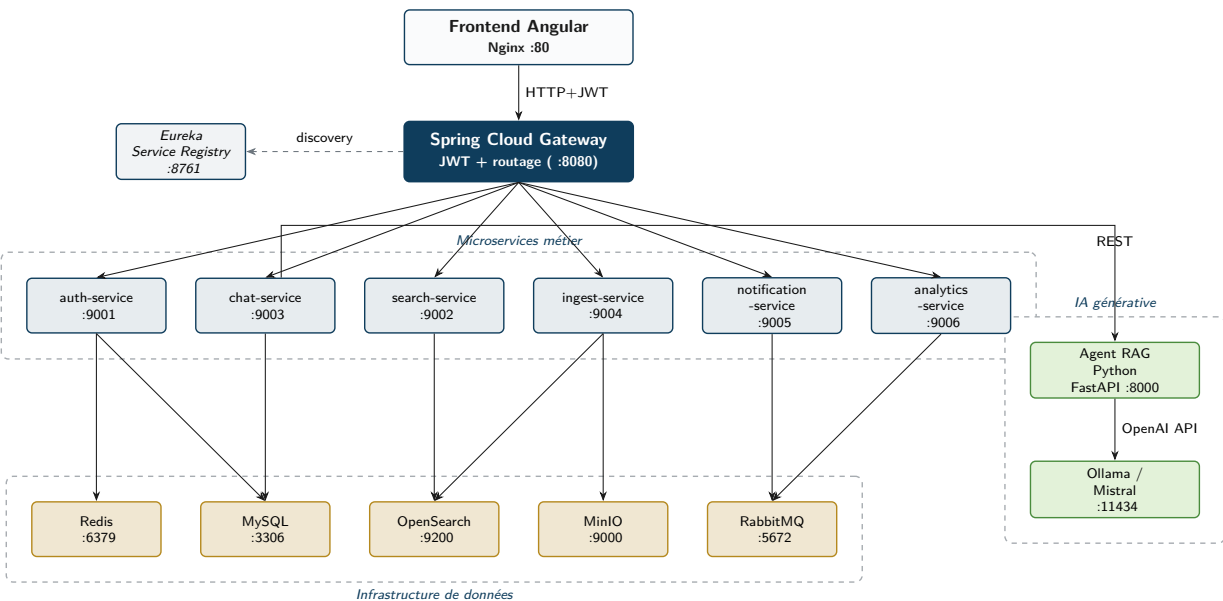


Figure 2.3 : Architecture microservices du portail juridique intelligent

2.5.2 Description des microservices

Le tableau ci-dessous décrit les huit microservices de la plateforme, avec leur port d'écoute et leurs principales responsabilités.

Tableau 2.5 : Description des microservices du portail juridique

Service	Port	Responsabilité
eureka-server	8761	Registre de services (Spring Cloud Netflix Eureka). Tous les services s'y enregistrent et s'y découvrent mutuellement.
gateway	8080	Point d'entrée unique (Spring Cloud Gateway). Valide les JWT, injecte X-User-Id dans les headers, route vers les services cibles, gère le CORS.

Service	Port	Responsabilité
auth-service	9001	Gestion des comptes utilisateurs, authentification OTP par email, génération et révocation des JWT, gestion des rôles.
search-service	9002	Moteur de recherche hybride BM25+kNN via OpenSearch, suggestions de complétion, signets, export JSON/XML.
chat-service	9003	Sessions et messages de chat, intégration RAG avec Ollama/Mistral, historique des conversations.
ingest-service	9004	Pipeline d'ingestion : extraction de texte (Tika+Tesseract), découpage en chunks, génération d'embeddings, indexation OpenSearch, stockage MinIO.
notification-service	9005	Gestion des alertes thématiques, envoi d'emails via JavaMail, écoute des événements RabbitMQ.
analytics-service	9006	Historique des recherches, retours RAG, journaux d'audit, tableaux de bord statistiques, top requêtes.

2.5.3 Flux de communication inter-services

La communication entre les microservices suit deux patterns :

- **Communication synchrone (REST)** : Le Gateway route les requêtes HTTP vers les services via le load balancer Eureka (`lb://service-name`). Les services communiquent entre eux via RestClient ou WebClient.
- **Communication asynchrone (RabbitMQ)** : L'`ingest-service` publie un message `document.indexed` dans l'échange RabbitMQ après chaque ingestion réussie. Le `notification-service` consomme ce message pour déclencher les alertes thématiques et les emails.

2.5.4 Infrastructure

L'ensemble du système est déployé via Docker Compose sur un réseau bridge dédié `pjnet` :

- **MySQL 8.4** : Base de données relationnelle pour auth, chat, analytics.
- **Redis 7** : Cache distribué, blacklist JWT, rate limiting.
- **OpenSearch 2.13.0** : Moteur de recherche hybride (BM25 + kNN) [\[W4\]](#)[\[B7\]](#).
- **MinIO** : Stockage objet S3-compatible pour les fichiers originaux [\[W7\]](#).
- **RabbitMQ 3** : Broker de messages pour la communication asynchrone [\[W8\]](#).

- **Ollama** : Serveur LLM local pour le modèle Mistral (RAG) [W5][W15].

2.6 Environnement de travail

Au fil du projet, nous avons assemblé une pile technique cohérente avec trois lignes directrices qui se sont imposées rapidement : héberger localement les modèles d'intelligence artificielle pour préserver la souveraineté des données juridiques traitées, retenir uniquement des versions stables et activement maintenues afin de limiter la dette technique à long terme, et privilégier dès que possible les composants disposant nativement d'un support du bilinguisme arabe-français propre au corpus tunisien.

2.6.1 Notre environnement matériel

Le développement a été conduit sur un poste équipé d'un processeur Intel Core i7 de 12^e génération offrant huit cœurs et seize *threads*, accompagné de 16 Go de RAM DDR5. Cette mémoire vive correspond, à notre expérience, au minimum nécessaire pour faire cohabiter simultanément nos huit microservices Spring Boot, le cluster OpenSearch et le modèle de langage local sans pénaliser la réactivité de la machine. Le stockage repose sur un SSD NVMe de 512 Go, partagé entre nos artefacts Docker (qui cumulent une trentaine de gigaoctets une fois toutes les images construites), notre corpus de PDFs (environ deux gigaoctets) et l'index OpenSearch. Sur le plan logiciel, nous avons fait cohabiter Windows 11 avec une distribution Ubuntu 22.04 LTS via WSL2, pour bénéficier simultanément des outils Windows habituels et des utilitaires Unix.

2.6.2 Langages de programmation

Java 21 (LTS)

Langage du *backend* : les huit microservices Spring Boot, la persistance JPA/Hibernate, la signature des jetons et la communication avec OpenSearch.



Figure 2.4 : Logo de Java 21 (LTS)

TypeScript 5.9

Langage typé du *frontend* Angular : services HTTP, *store* d'état, intercepteurs et *guards* de navigation.



Figure 2.5 : Logo de TypeScript 5.9

Python 3.11

Langage de l'agent conversationnel (FastAPI, LangChain) et des *pipelines* d'extraction VLM et d'indexation.



Figure 2.6 : Logo de Python 3.11

2.6.3 Pile backend

Spring Boot 3

Framework des huit microservices Java : API REST, injection de dépendances, configuration et démarrage autonome. [\[B4\]](#)[\[W1\]](#)



Figure 2.7 : Logo de Spring Boot 3

Spring Cloud (Gateway + Eureka)

Passerelle unique d'entrée portant le filtre JWT, et registre de découverte des services par nom logique. [W2]



Figure 2.8 : Logo de Spring Cloud (Gateway + Eureka)

Hibernate / Spring Data JPA

ORM assurant le *mapping* objet-relationnel des entités vers les bases MySQL.



Figure 2.9 : Logo de Hibernate / Spring Data JPA

Spring Security + JJWT

Sécurisation des *endpoints*, hachage bcrypt des mots de passe et émission/validation des jetons JWT (HMAC-SHA256).



Figure 2.10 : Logo de Spring Security + JJWT

Apache Tika 2.9

Extraction de texte des documents (PDF, DOCX) lors de l'ingestion par l'`ingest-service`.

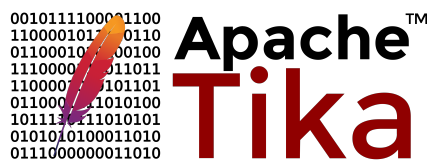


Figure 2.11 : Logo de Apache Tika 2.9

2.6.4 Pile frontend

Angular 21

Framework de l'application monopage (composants *standalone*, routing, formulaires, intercepteurs). [\[B5\]](#)[\[W3\]](#)



Figure 2.12 : Logo de Angular 21

Angular Material + PrimeNG

Bibliothèques de composants graphiques : champs Material Design et *widgets* riches (tableaux paginés, calendriers, graphiques).



Figure 2.13 : Logo de Angular Material + PrimeNG

NgRx Signals

Gestion d'état réactive fondée sur les *signals* natifs d'Angular. [\[W11\]](#)



Figure 2.14 : Logo de NgRx Signals

RxJS 7.8

Programmation réactive : flux asynchrones et composition d'opérateurs.



Figure 2.15 : Logo de RxJS 7.8

2.6.5 Intelligence artificielle

LM Studio

Serveur local exposant une API compatible OpenAI pour les modèles Mistral et Qwen2.5-VL ; garantit la souveraineté des données.



Figure 2.16 : Logo de LM Studio

Ollama

Runtime local de modèles de langage, utilisé pour la génération et le calcul des *embeddings*. [W5]



Figure 2.17 : Logo de Ollama

Mistral

LLM de génération conversationnelle de l'assistant RAG (exécuté en quantification Q4). [W15]

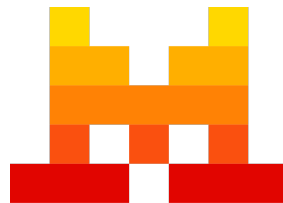


Figure 2.18 : Logo de Mistral

Qwen2.5-VL-7B

Modèle vision-langage du *pipeline* d'extraction : il « lit » les pages du JORT et restitue un JSON structuré.



Figure 2.19 : Logo de Qwen2.5-VL-7B

LangChain 0.3

Abstraction d'orchestration des appels au modèle de langage dans l'agent Python.



Figure 2.20 : Logo de LangChain 0.3

FastAPI 0.115

API de l'agent RAG : *endpoints* /*query* et *streaming* SSE des réponses.



Figure 2.21 : Logo de FastAPI 0.115

OpenCV 4.13

Prétraitement des images des PDF scannés (débruitage, binarisation, redressement).



Figure 2.22 : Logo de OpenCV 4.13

Tesseract OCR 5

Reconnaissance optique de caractères des PDF scannés, en français et en arabe.



Figure 2.23 : Logo de Tesseract OCR 5

2.6.6 Bases de données, moteurs et infrastructure

MySQL 8.4

Base relationnelle des services métier (auth, chat, analytics, notification).



Figure 2.24 : Logo de MySQL 8.4

OpenSearch 2.13

Moteur de recherche bilingue : BM25, kNN et *analyzers* dédiés `ar_legal` / `fr_legal`. [\[W4\]](#)



Figure 2.25 : Logo de OpenSearch 2.13

Redis 7

Cache distribué et *blacklist* des jetons JWT révoqués.



Figure 2.26 : Logo de Redis 7

MinIO

Stockage objet compatible S3 pour les PDF originaux du corpus. [\[W7\]](#)



Figure 2.27 : Logo de MinIO

RabbitMQ 3

Bus de messages asynchrones transportant les événements `document.indexed` entre services. [\[W8\]](#)



Figure 2.28 : Logo de RabbitMQ 3

Docker + Compose

Conteneurisation et orchestration locale de l'ensemble des services. [\[W9\]](#)



Figure 2.29 : Logo de Docker + Compose

GitLab CI/CD

Dépôt Git, intégration continue (*build* sélectif), *registry* d'images et déploiement. [\[W10\]](#)



Figure 2.30 : Logo de GitLab CI/CD

Nginx

Serveur des fichiers statiques Angular et *reverse proxy* vers la passerelle.



Figure 2.31 : Logo de Nginx

2.6.7 Outils de développement

IntelliJ IDEA

IDE principal du *backend* Java et du débogage des microservices.



Figure 2.32 : Logo de IntelliJ IDEA

Visual Studio Code

IDE du *frontend* Angular, des scripts Python et des fichiers de configuration.



Figure 2.33 : Logo de Visual Studio Code

PyCharm

IDE dédié au développement de l'agent RAG en Python.



Figure 2.34 : Logo de PyCharm

Postman

Tests manuels des API REST, en complément de Swagger UI auto-généré.



Figure 2.35 : Logo de Postman

Git + Maven + npm

Gestion de versions et outils de *build* (Maven pour Java, npm pour le *frontend*).



Figure 2.36 : Logo de Git + Maven + npm

Conclusion

Dans ce chapitre, nous avons établi la spécification complète des besoins du portail juridique intelligent, identifié les acteurs, modélisé les données et l'architecture microservices, planifié les six sprints répartis en trois releases, et présenté notre environnement de travail. Nous passons maintenant à la réalisation concrète de la Release 1.

Release 1 : Authentification et Moteur de Recherche

Introduction

Cette première release constitue le socle fondamental du portail juridique intelligent. Elle comprend deux sprints : le **Sprint 1** consacré à la mise en place du service d'authentification avec OTP par email et gestion des rôles, et le **Sprint 2** dédié au développement du moteur de recherche hybride BM25+kNN via OpenSearch, ainsi qu'aux fonctionnalités de signets et de suggestions de complétion.

3.1 Sprint 1 : Service d'authentification et gestion des utilisateurs

3.1.1 Objectifs du Sprint 1

L'objectif du premier sprint est de développer un système d'authentification robuste reposant sur le couple email + mot de passe, le mot de passe étant haché avec **bcrypt** (jamais stocké en clair). Ce socle est renforcé par un code OTP (One-Time Password) envoyé par email en **deuxième facteur optionnel**, activable selon le niveau de sécurité souhaité. Le sprint couvre également la gestion des rôles (CITIZEN, JURIST, MANAGER, ANALYSTE, ADMIN) et la mise en place de l'infrastructure JWT (jeton d'accès + refresh token).

3.1.2 Backlog du Sprint 1

Le tableau ci-dessous détaille le *Product Backlog* du Sprint 1, consacré à l'authentification et à la gestion des utilisateurs. Chaque besoin y est exprimé sous forme de *user story* priorisée et estimée en jours-homme.

Tableau 3.1 : Backlog du Sprint 1 — Authentification

Id	User Story	Priorité	Estim. (j)
1	En tant que visiteur, je veux m'inscrire avec mon email et mon nom afin de créer un compte sur le portail.	1	3
2	En tant que visiteur, je veux recevoir un code OTP à 6 chiffres par email afin de valider mon identité.	1	3
3	En tant que visiteur, je veux soumettre le code OTP reçu afin de finaliser ma connexion et recevoir un token JWT.	1	4
4	En tant qu'utilisateur connecté, je veux consulter mon profil afin de vérifier mes informations.	2	2

Id	User Story	Priorité	Estim. (j)
5	En tant qu'utilisateur connecté, je veux modifier mon profil (nom, langue préférée) afin de personnaliser mon expérience.	2	3
6	En tant qu'administrateur, je veux attribuer le rôle JURIST à un utilisateur afin de lui accorder des permissions étendues.	2	3
7	En tant qu'utilisateur, je veux que mon jeton soit révoqué à la déconnexion (blacklist Redis) afin de sécuriser ma session.	1	4
Total Sprint 1			22

3.1.3 Planification des tâches du Sprint 1

Le tableau 3.2 détaille les tâches de conception et de développement réalisées durant ce sprint.

Tableau 3.2 : Tâches du Sprint 1

Numéro	Tâches
1	Élaboration du diagramme de cas d'utilisation du Sprint 1
2	Élaboration du diagramme de séquence : authentification OTP
3	Élaboration du diagramme de classes de conception du Sprint 1
4	Codage des entités <code>UserAccount</code> , <code>Role</code> et <code>Permission</code>
5	Codage des <i>repositories</i> (<code>UserRepository</code> , <code>RoleRepository</code> , <code>PermissionRepository</code>)
6	Codage du service d'authentification <code>AuthService</code> (génération/validation OTP, JWT, hachage bcrypt)
7	Codage du service RBAC <code>RbacService</code> (rôles et permissions)
8	Codage de la sécurité : <code>JwtAuthFilter</code> , <code>JwtAuthenticationEntryPoint</code> , <code>JwtAccessDeniedHandler</code>
9	Codage des contrôleurs <code>AuthController</code> , <code>RoleController</code> , <code>UserAdminController</code>
10	Développement des interfaces de connexion et de vérification OTP (Angular)
11	Tests unitaires et d'intégration du Sprint 1

3.1.4 Conception du Sprint 1

3.1.4.1 Diagramme de séquence : Authentification OTP

Le diagramme de séquence ci-dessous illustre le flux d'authentification : après vérification du mot de passe, un second facteur par code OTP n'est demandé que *si* la double authentification est activée pour le compte ; dans le cas contraire, la connexion aboutit directement et le jeton JWT est renvoyé.

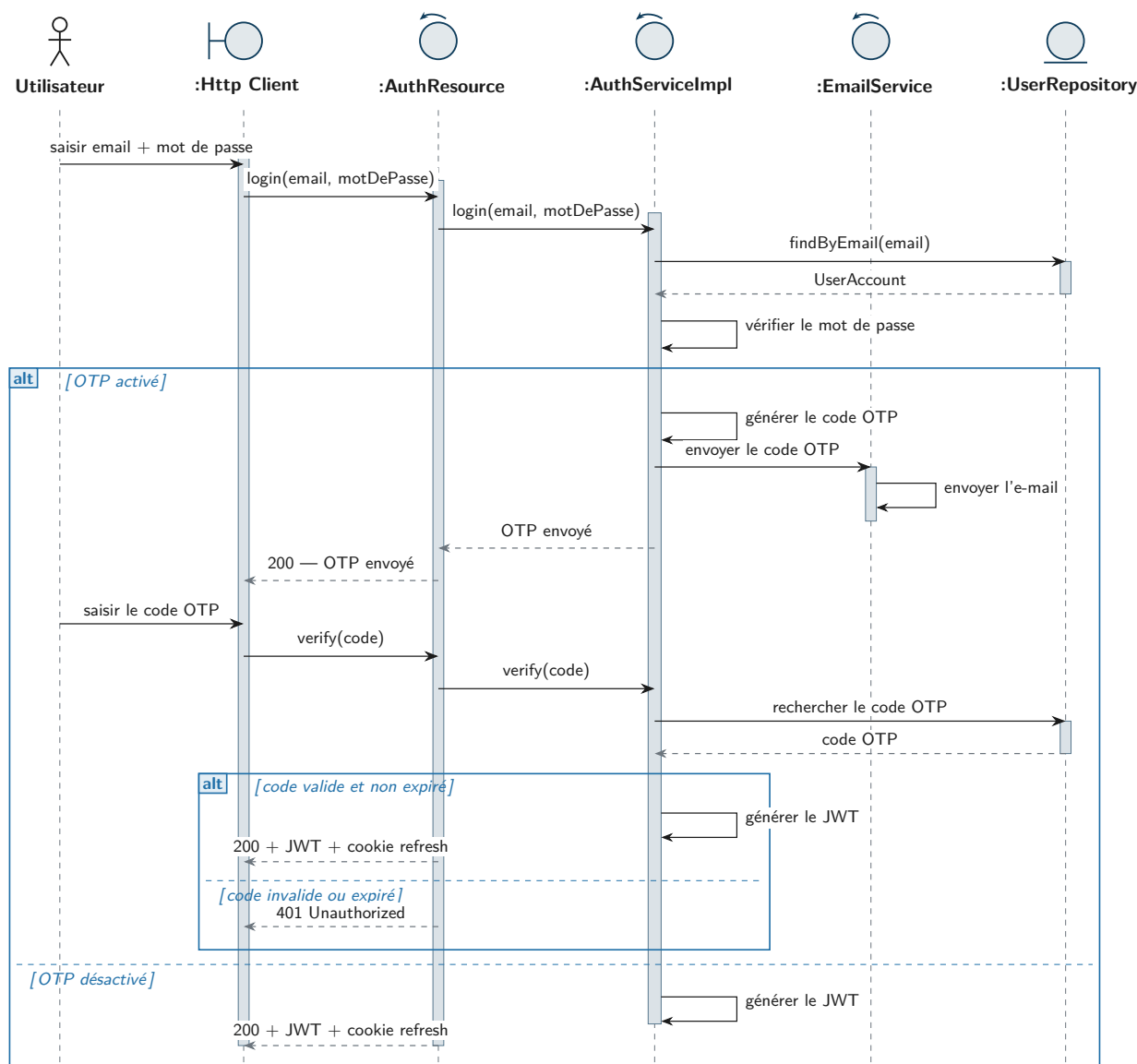


Figure 3.1 : Diagramme de séquence d'objet — Authentification OTP.

Le flux se déroule comme suit :

1. Le client envoie `POST /api/auth/login` avec l'email.
2. L'`auth-service` vérifie si un compte existe (ou le crée pour une inscription).
3. Un code OTP à 6 chiffres est généré et stocké chiffré en base de données avec une expiration

de 10 minutes.

4. L'email est envoyé via `JavaMailSender` (serveur SMTP configuré).
5. Le client soumet le code OTP via `POST /api/auth/verify`.
6. L'`auth-service` valide le code (existence, non-utilisé, non-expiré).
7. En cas de succès, un JWT signé avec HMAC-SHA256 (JJWT 0.11.5) est généré et retourné.
8. Le token est stocké côté client (`localStorage`) et joint à chaque requête via le header `Authorization: Bearer`.

3.1.5 Réalisation du Sprint 1

3.1.5.1 Modèle de données

L'`auth-service` gère deux entités principales via JPA/Hibernate 7 avec MySQL :

- **UserAccount** : Représente le compte utilisateur. L'entité stocke l'email, le nom d'affichage, le mot de passe haché en bcrypt (`passwordHash`), le rôle (enum `UserRole`) et le statut d'activation.
- **OtpToken** : Représente un code OTP à usage unique. Stocke l'email cible, le code haché, la date d'expiration et un flag `used`.

3.1.5.2 Validation JWT dans le Gateway

Le `JwtAuthFilter` du service Gateway est un `GlobalFilter` Spring Cloud Gateway qui :

1. Laisse passer toutes les requêtes vers `/api/auth/**` sans validation.
2. Extrait le token JWT du header `Authorization: Bearer`.
3. Valide la signature et l'expiration via `JwtTokenProvider`.
4. Vérifie que le token n'est pas dans la blacklist Redis.
5. Injecte les headers `X-User-Id`, `X-User-Role` et `X-User-Email` pour les services en aval.
6. Rejette avec HTTP 401 si la validation échoue.

3.1.5.3 Interface utilisateur — Authentification

La figure 3.2 présente la première étape de l'authentification : l'écran de connexion du portail. L'utilisateur y saisit son adresse e-mail pour demander l'envoi d'un code à usage unique, l'interface adoptant une présentation épurée et bilingue conforme à la charte graphique du portail.

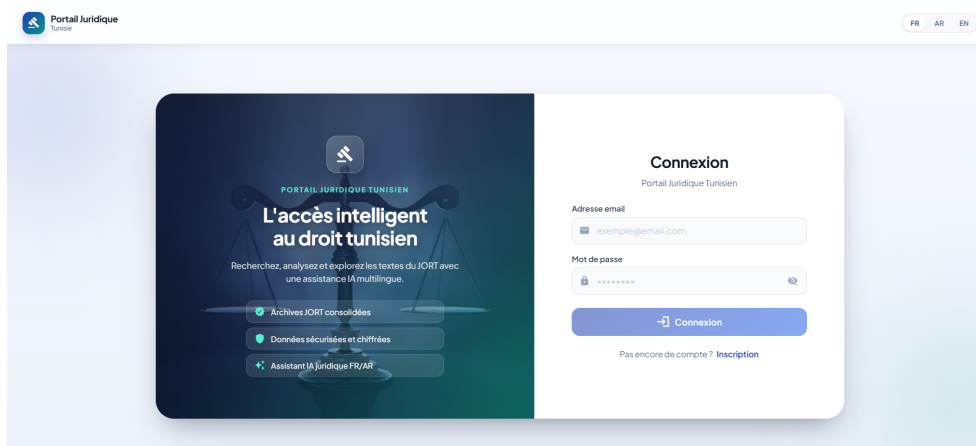


Figure 3.2 : Interface de connexion — Saisie de l'email

La figure 3.3 illustre la seconde étape : après réception de l'e-mail, l'utilisateur saisit le code OTP à six chiffres pour valider son identité et accéder au portail. Un minuteur indique la durée de validité du code et un lien permet d'en demander un nouveau en cas d'expiration.

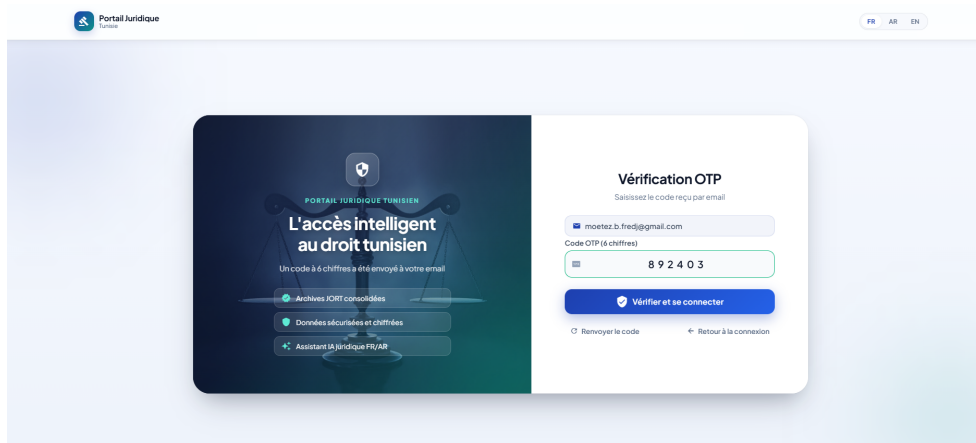


Figure 3.3 : Interface de vérification OTP — Saisie du code à 6 chiffres

La figure 3.4 présente l'e-mail effectivement reçu par l'utilisateur, vu depuis sa boîte de réception Gmail. Envoyé automatiquement par le service de messagerie du portail, il contient le code à usage unique (ici 521849), sa durée de validité de cinq minutes ainsi qu'un avertissement de sécurité invitant à ignorer le message en cas de demande non sollicitée.

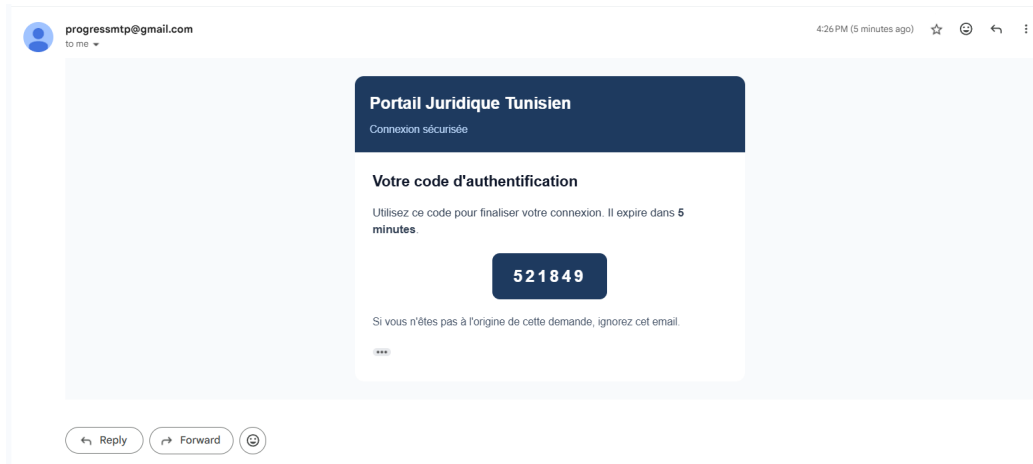


Figure 3.4 : E-mail de réception du code OTP — vue depuis la boîte de réception

3.1.6 Tests du Sprint 1

Le tableau ci-dessous récapitule les principaux cas de test exécutés pour valider le Sprint 1, avec leur scénario et le résultat attendu.

Tableau 3.3 : Cas de test — Sprint 1 : Authentification

Cas de test	Étapes	Résultat attendu
Inscription réussie	Soumettre un email valide non existant	Compte créé, OTP envoyé, HTTP 200
OTP valide	Soumettre le bon code dans les 10 min	JWT retourné, HTTP 200
OTP expiré	Soumettre un code après 10 minutes	HTTP 400, message d'erreur
OTP déjà utilisé	Soumettre le même code une deuxième fois	HTTP 400, accès refusé
Token invalide	Appel API avec token altéré	HTTP 401, Unauthorized
Token blacklisté	Appel après déconnexion	HTTP 401, Unauthorized

3.1.7 Revue et rétrospective du Sprint 1

À l'issue du Sprint 1, l'ensemble des *user stories* planifiées ont été réalisées avec succès. Le service d'authentification est opérationnel, sécurisé et testé. Les points d'amélioration identifiés lors de la rétrospective sont :

- Ajouter un mécanisme de limitation du taux de demandes OTP (rate limiting via Redis) pour prévenir les abus.
- Améliorer les messages d'erreur retournés au client pour une meilleure UX.

3.2 Sprint 2 : Moteur de recherche hybride et signets

3.2.1 Objectifs du Sprint 2

Le Sprint 2 vise à développer le cœur fonctionnel du portail : le moteur de recherche hybride. Ce moteur combine la recherche lexicale BM25 (efficace pour la correspondance exacte de termes juridiques) et la recherche sémantique par vecteurs denses kNN (efficace pour retrouver des textes conceptuellement similaires même sans correspondance de mots-clés). Le sprint inclut également les fonctionnalités de suggestions de complétion automatique et de gestion des signets.

3.2.2 Backlog du Sprint 2

Le tableau ci-dessous présente le *Product Backlog* du Sprint 2, dédié au moteur de recherche hybride. Les *user stories* y sont priorisées et estimées en jours-homme.

Tableau 3.4 : Backlog du Sprint 2 — Moteur de recherche

Id	User Story	Priorité	Estim. (j)
8	En tant qu'utilisateur, je veux rechercher des textes par mots-clés (mode BM25) afin de trouver des lois contenant des termes précis.	1	4
9	En tant qu'utilisateur, je veux effectuer une recherche sémantique (mode kNN) afin de retrouver des textes de même signification.	1	4
10	En tant qu'utilisateur, je veux effectuer une recherche hybride (BM25+kNN) afin d'obtenir les résultats les plus pertinents.	1	3
11	En tant qu'utilisateur, je veux voir des suggestions de complétion automatique en tapant ma requête.	2	3
12	En tant qu'utilisateur, je veux ajouter un texte juridique en signet afin de le retrouver facilement.	2	2
13	En tant qu'utilisateur, je veux consulter mes signets depuis mon tableau de bord.	2	2
14	En tant qu'utilisateur, je veux supprimer un signet dont je n'ai plus besoin.	2	2
15	En tant que juriste, je veux exporter les résultats de recherche en JSON ou XML afin de les utiliser dans mes outils.	3	2
Total Sprint 2			22

3.2.3 Planification des tâches du Sprint 2

Le tableau 3.5 détaille les tâches de conception et de développement réalisées durant ce sprint.

Tableau 3.5 : Tâches du Sprint 2

Numéro	Tâches
1	Élaboration du diagramme de séquence : recherche hybride
2	Configuration du client OpenSearch (<code>OpenSearchConfig</code> , mapping <code>kNN</code> , index <code>legal-documents</code>)
3	Codage des <i>analyzers</i> bilingues <code>ar_legal</code> / <code>fr_legal</code>

Numéro	Tâches
4	Codage du service de recherche <code>SearchService</code> (BM25, kNN, fusion RRF)
5	Codage du service de suggestions (auto-complétion)
6	Codage de l'entité <code>Bookmark</code> et de son <i>repository</i>
7	Codage du service de signets <code>BookmarkService</code>
8	Codage du service d'export des résultats (JSON / XML)
9	Codage du contrôleur <code>SearchController</code>
10	Développement de l'interface de recherche (modes, facettes, signets)
11	Tests du Sprint 2

3.2.4 Conception du Sprint 2

3.2.4.1 Architecture du moteur de recherche hybride

Le moteur de recherche hybride repose sur **OpenSearch 2.13.0** et combine deux stratégies complémentaires :

- **BM25 (Best Match 25)** : Algorithme de ranking probabiliste basé sur la fréquence des termes (TF) et la fréquence inverse dans les documents (IDF) [B10]. Particulièrement efficace pour retrouver des textes contenant des termes juridiques exacts (numéro de loi, article spécifique, terme technique).
- **kNN sémantique (k-Nearest Neighbors)** : Recherche par similarité cosinus dans l'espace vectoriel des embeddings. Les textes sont représentés par des vecteurs denses de 768 dimensions générés par le modèle **multilingual-e5-base** (Microsoft) [B9][W14]. Cette approche retrouve des textes sémantiquement proches même sans correspondance lexicale.
- **Recherche hybride** : Les scores BM25 et kNN sont combinés via la formule de normalisation RRF (*Reciprocal Rank Fusion*) [B11] :

$$S_{hybride}(d) = \alpha \times S_{BM25}(d) + (1 - \alpha) \times S_{kNN}(d)$$

avec $\alpha = 0.5$ par défaut, ajustable par l'utilisateur.

3.2.4.2 Diagramme de séquence : Recherche hybride

La figure 3.5 détaille, étape par étape, le déroulement d'une recherche hybride combinant BM25 et kNN.

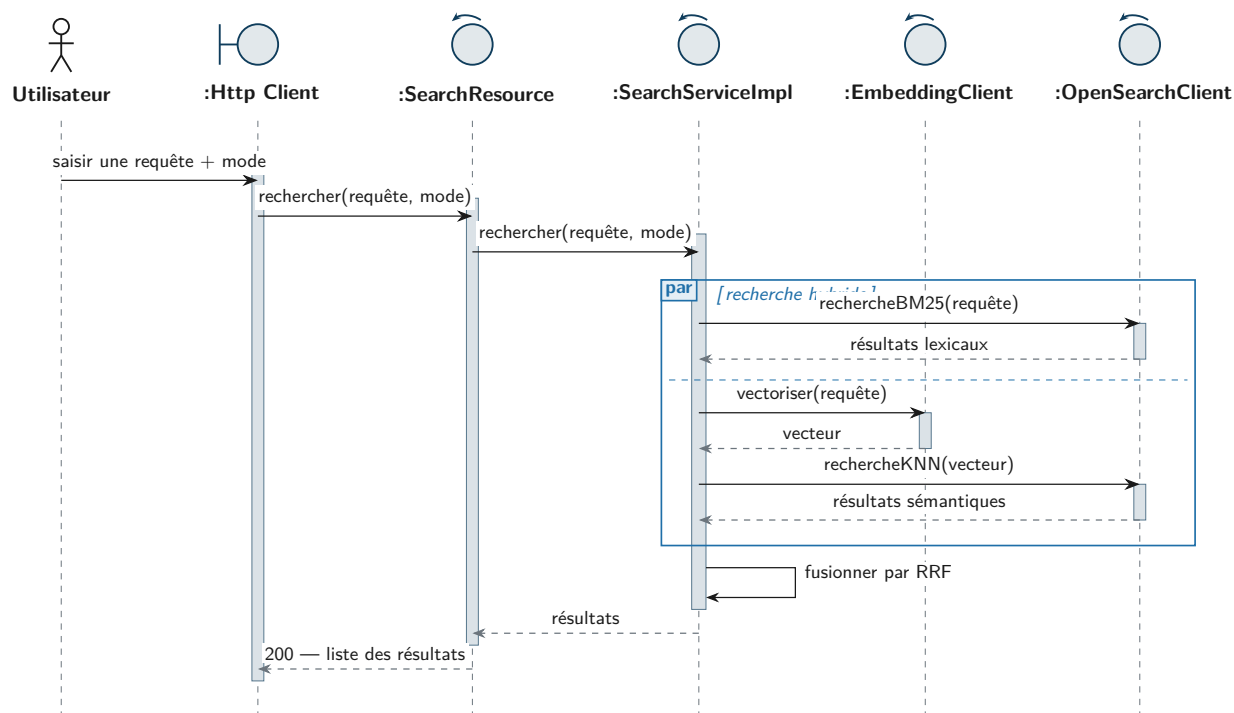


Figure 3.5 : Diagramme de séquence d'objet — Recherche hybride.

3.2.5 Réalisation du Sprint 2

3.2.5.1 Configuration OpenSearch

La classe `OpenSearchConfig` initialise le client `OpenSearch` avec :

- Connexion sécurisée via HTTPS avec `TrustStore` configuré.
- Authentification par utilisateur/mot de passe.
- Pool de connexions HTTP avec timeout configurables.
- Création automatique de l'index `legal-documents` avec le mapping kNN au démarrage.

3.2.5.2 Service de recherche

La classe `SearchService` expose trois modes de recherche :

- **Mode LEXICAL** : Construction d'une requête `multi_match` `OpenSearch` sur les champs `title` (boost 3) et `content` (boost 1) avec l'algorithme BM25 natif d'`OpenSearch`.
- **Mode SEMANTIC** : Génération de l'embedding de la requête utilisateur via l'API Ollama / `api/embeddings` (modèle `multilingual-e5-base`), puis requête `knn` `OpenSearch` avec $k = 10$.

- **Mode HYBRID** : Exécution parallèle des deux requêtes précédentes, fusion des résultats par RRF, déduplication par ID de document.

3.2.5.3 API du search-service

Le tableau ci-dessous récapitule les principaux *endpoints* REST exposés par le **search-service**.

Tableau 3.6 : API REST du search-service

Méthode	Endpoint	Description
GET	/api/search	Recherche hybride/lexicale/sémantique. Params : q, mode, page, size
GET	/api/suggest	Suggestions de complétion. Param : q
GET	/api/search/{id}	Détail d'un document par son ID OpenSearch
POST	/api/export	Export JSON/XML des résultats de recherche
GET	/api/bookmarks	Liste des signets de l'utilisateur courant
POST	/api/bookmarks	Ajout d'un signet
DELETE	/api/bookmarks/{id}	Suppression d'un signet

3.2.5.4 Interface utilisateur — Recherche de textes juridiques

La figure 3.6 présente l'interface de recherche de textes juridiques du portail, qui illustre le mode hybride et l'affichage des résultats.

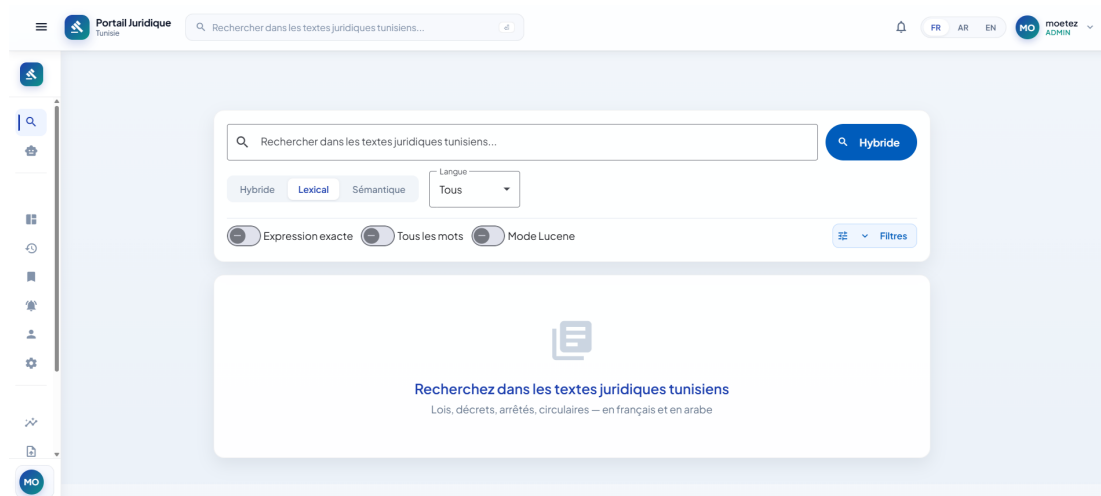


Figure 3.6 : Interface de recherche

L'interface de recherche Angular 21 offre :

- Une barre de recherche avec suggestions de complétion automatique (debounce 300ms).
- Un sélecteur de mode : Lexical / Sémantique / Hybride.
- Un panneau de facettes (type de texte, langue, plage de dates).
- Des cartes de résultats avec titre, extrait pertinent, type, date et bouton de signet.
- Un panneau latéral glissant pour afficher le texte intégral du document sélectionné.
- Un bouton d'export (JSON/XML) visible pour les juristes et administrateurs.

3.2.6 Tests du Sprint 2

Le tableau ci-dessous récapitule les principaux cas de test exécutés pour valider le Sprint 2, avec leur scénario et le résultat attendu.

Tableau 3.7 : Cas de test — Sprint 2 : Moteur de recherche

Cas de test	Entrée	Résultat attendu
Recherche BM25 simple	Query : “décret présidentiel 2023”	Documents contenant ces termes, triés par score BM25
Recherche sémantique	Query : “protection des données personnelles”	Documents sur la vie privée/RGPD même sans ces mots exacts
Recherche hybride	Query : “code du travail heures supplémentaires”	Meilleure pertinence combinée
Complétion automatique	Saisie de “code de”	Suggestions : “code de commerce”, “code du travail”...
Ajout signet	Click bouton signet sur un résultat	Signet créé, confirmé par icône remplie
Suppression signet	Click supprimer dans la liste des signets	Signet supprimé, liste mise à jour
Export JSON	Clic export avec mode JSON	Fichier JSON téléchargé avec les résultats

3.2.7 Revue et rétrospective du Sprint 2

La Release 1 est complète : le portail dispose d'un système d'authentification OTP sécurisé et d'un moteur de recherche hybride fonctionnel. Les performances mesurées sont satisfaisantes : temps de réponse médian de 850ms pour la recherche hybride sur un corpus de 10 000 documents. Les pistes d'amélioration identifiées sont :

- Optimisation du cache Redis pour les requêtes fréquentes.
- Ajout de filtres facettés plus granulaires (par article, par chapitre).

Conclusion

La Release 1 a permis de poser les bases essentielles du portail : une authentification sécurisée (email + mot de passe haché bcrypt, OTP en second facteur optionnel, JWT) et un moteur de recherche hybride innovant. Ces deux composants constituent le cœur de la valeur métier du portail. Nous abordons maintenant la Release 2 avec l'assistant conversationnel RAG et le pipeline d'ingestion documentaire.

Release 2 : Assistant RAG et Pipeline d'Ingestion

Plan

Introduction	4
1 Cadre général du projet	4
2 Étude de l'existant	6
3 Problématique	9
4 Solution proposée	10
5 Méthodologie de gestion de projet	11
Conclusion	12

Introduction

La Release 2 enrichit le portail de deux fonctionnalités à haute valeur ajoutée : le **Sprint 3** consacré au développement de l'assistant conversationnel RAG (*Retrieval-Augmented Generation*) basé sur le modèle Mistral, et le **Sprint 4** dédié au pipeline complet d'ingestion documentaire intégrant l'OCR Apache Tika/Tesseract et le stockage MinIO.

4.1 Sprint 3 : Assistant conversationnel RAG

4.1.1 Objectifs du Sprint 3

L'objectif du Sprint 3 est de développer un assistant conversationnel intelligent capable de répondre à des questions juridiques en langage naturel, en s'appuyant sur le corpus de textes indexé dans OpenSearch. L'assistant utilise le paradigme **RAG** qui garantit que toutes les réponses sont fondées sur des sources documentaires réelles, avec citation systématique des textes juridiques consultés.

4.1.2 Backlog du Sprint 3

Le tableau ci-dessous détaille le *Product Backlog* du Sprint 3, consacré à l'assistant conversationnel RAG. Chaque besoin y est exprimé sous forme de *user story* priorisée et estimée en jours-homme.

Tableau 4.1 : Backlog du Sprint 3 — Assistant RAG

Id	User Story	Priorité	Estim. (j)
16	En tant qu'utilisateur, je veux créer une nouvelle session de chat avec l'assistant afin d'obtenir des réponses juridiques personnalisées.	1	3
17	En tant qu'utilisateur, je veux envoyer une question en langage naturel et recevoir une réponse fondée sur les textes de loi.	1	5
18	En tant qu'utilisateur, je veux que les réponses de l'assistant citent les articles et textes juridiques sources.	1	3
19	En tant qu'utilisateur, je veux consulter l'historique de mes sessions de chat précédentes.	2	3
20	En tant qu'utilisateur, je veux évaluer chaque réponse de l'assistant (pouce haut/bas) afin d'améliorer sa qualité.	2	3

Id	User Story	Priorité	Estim. (j)
21	En tant qu'utilisateur, je veux choisir la langue de conversation (français ou arabe) afin de dialoguer dans ma langue préférée.	2	2
Total Sprint 3			19

4.1.3 Planification des tâches du Sprint 3

Le tableau 4.2 détaille les tâches de conception et de développement réalisées durant ce sprint.

Tableau 4.2 : Tâches du Sprint 3

Numéro	Tâches
1	Élaboration du diagramme de séquence : traitement d'une question RAG
2	Conception du <i>prompt</i> RAG (<i>system prompt</i> , injection du contexte)
3	Codage de l'agent RAG en Python (FastAPI)
4	Intégration du modèle Mistral via Ollama
5	Codage de la récupération kNN (top-5) et de la génération de réponse
6	Codage des entités <code>ChatSession</code> et <code>ChatMessage</code>
7	Codage du service de chat <code>ChatService</code>
8	Codage de l'évaluation des réponses (<code>RagFeedback</code>)
9	Codage du contrôleur <code>ChatController</code>
10	Développement de l'interface de l'assistant conversationnel (Angular)
11	Tests du Sprint 3

4.1.4 Conception du Sprint 3

4.1.4.1 Architecture RAG

Le paradigme **RAG** (*Retrieval-Augmented Generation*) [B8][B6] combine deux étapes fondamentales :

1. **Retrieval (Récupération)** : La question de l'utilisateur est encodée en vecteur d'embedding via le modèle `multilingual-e5-base`. Une recherche kNN dans OpenSearch récupère les $k = 5$ chunks de texte juridique les plus pertinents.

2. **Generation (Génération)** : Les chunks récupérés sont injectés dans un prompt structuré envoyé au LLM Mistral (via Ollama). Le modèle génère une réponse en langage naturel en se basant exclusivement sur ce contexte, garantissant ainsi la traçabilité et limitant les hallucinations.

Le prompt RAG est structuré comme suit :

Tu es un assistant juridique expert en droit tunisien.
Réponds UNIQUEMENT en te basant sur les textes juridiques fournis.
Si la réponse ne figure pas dans les textes, dis-le clairement.

TEXTES JURIDIQUES PERTINENTS :

[CHUNK 1] - {titre du texte 1} :
{contenu du chunk 1}

[CHUNK 2] - {titre du texte 2} :
{contenu du chunk 2}

...

QUESTION : {question de l'utilisateur}

RÉPONSE (cite les sources entre [Texte N]) :

4.1.4.2 Diagramme de séquence : Traitement d'une question RAG

La figure 4.1 illustre le traitement complet d'une question par l'assistant RAG, de la vectorisation à la réponse sourcée.

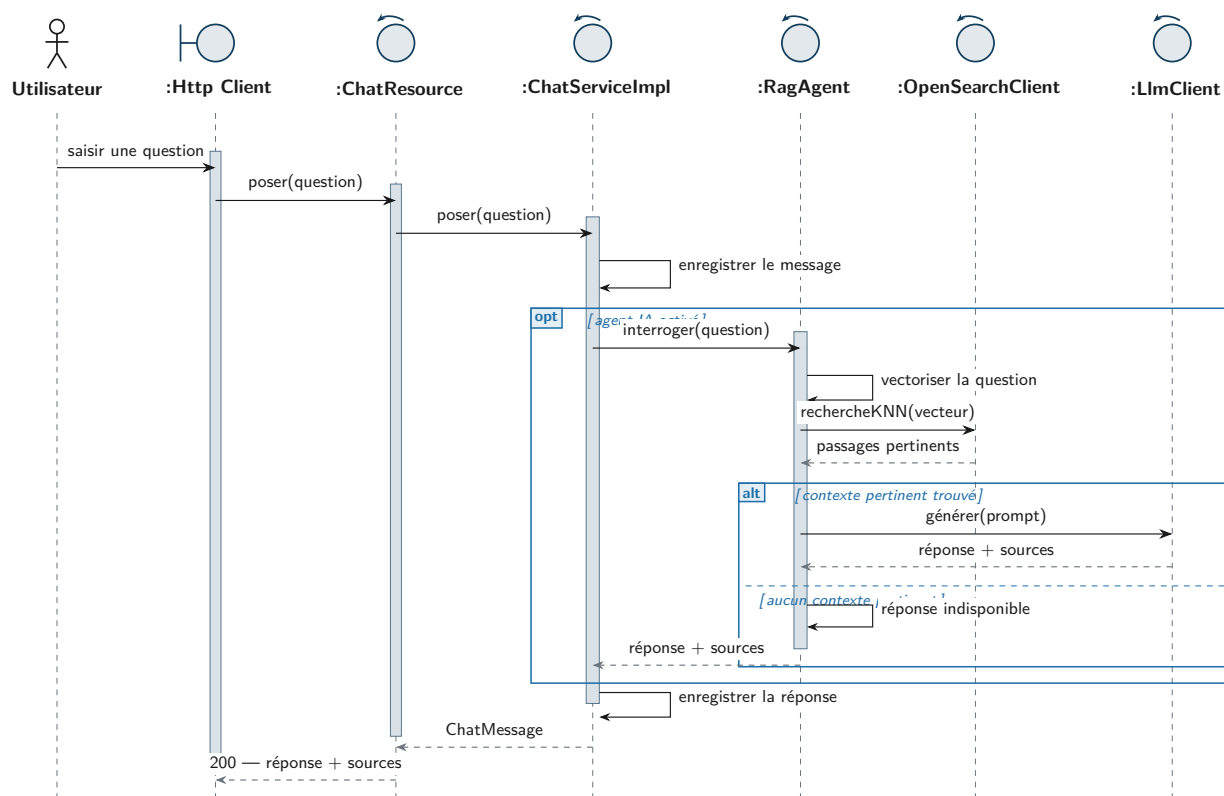


Figure 4.1 : Diagramme de séquence d'objet — Traitement d'une question RAG.

Le flux complet d'une question RAG est le suivant :

1. L'utilisateur envoie sa question via POST `/api/chat/sessions/{id}/messages`.
2. Le `chat-service` appelle l'API Ollama `/api/embeddings` pour vectoriser la question.
3. Une requête kNN est envoyée à OpenSearch pour récupérer les 5 chunks les plus pertinents.
4. Le prompt RAG est construit avec la question et les chunks récupérés.
5. Le prompt est envoyé à l'API Ollama `/api/generate` (modèle : `mistral`).
6. La réponse générée est reçue en streaming et assemblée.
7. Le message et les sources sont persistés en base de données.
8. La réponse avec les métadonnées des sources est retournée au client.

4.1.5 Réalisation du Sprint 3

4.1.5.1 Modèle de données

Le `chat-service` repose sur trois entités principales :

- **ChatSession** : Représente une session de conversation. Champs : `id` (UUID), `userId` (depuis header X-User-Id), `title` (généré automatiquement à partir de la première question), `language` (fr/ar), `createdAt`.
- **ChatMessage** : Représente un message dans une session. Champs : `id` (UUID), `sessionId`,

`role` (enum USER/ASSISTANT), `content`, `sources` (JSON sérialisé des références), `createdAt`.

- **RagFeedback** : Feedback sur un message assistant. Champs : `id`, `messageId`, `userId`, `rating` (enum UP/DOWN), `comment`, `createdAt`.

4.1.5.2 API du chat-service

Le tableau ci-dessous récapitule les principaux *endpoints* REST exposés par le `chat-service`.

Tableau 4.3 : API REST du chat-service

Méthode	Endpoint	Description
GET	<code>/api/chat/sessions</code>	Liste des sessions de l'utilisateur courant
POST	<code>/api/chat/sessions</code>	Créer une nouvelle session
GET	<code>/api/chat/sessions/{id}</code>	Détail d'une session avec ses messages
DELETE	<code>/api/chat/sessions/{id}</code>	Supprimer une session et ses messages
POST	<code>/api/chat/sessions/{id}/messages</code>	Envoyer une question et obtenir la réponse RAG
POST	<code>/api/chat/feedback</code>	Soumettre un feedback (pouce haut/bas)

4.1.5.3 Interface utilisateur — Chat RAG

La figure 4.2 présente l'interface de l'assistant conversationnel et l'affichage des sources citées.

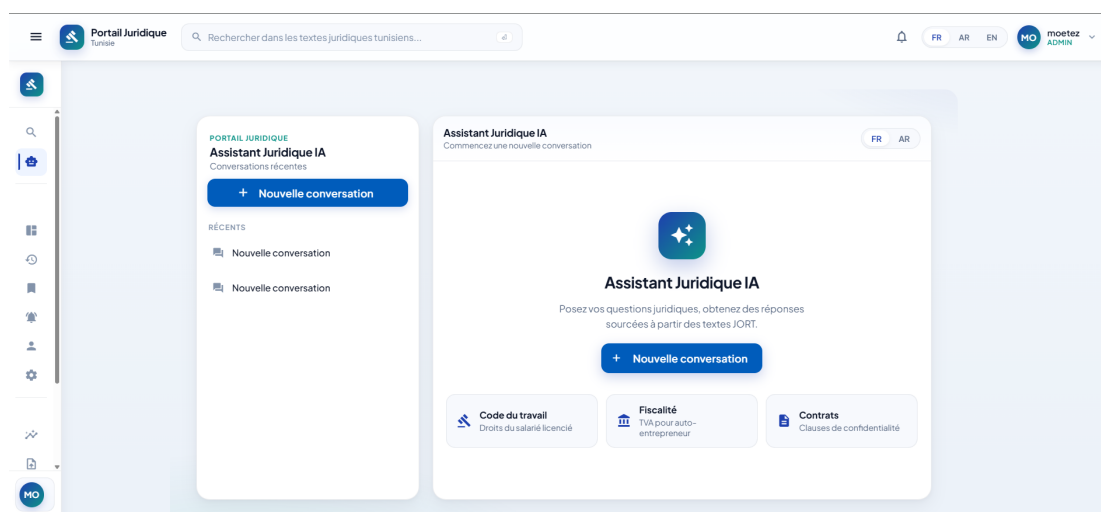


Figure 4.2 : Interface de l'assistant conversationnel RAG

L'interface de chat propose :

- Un panneau latéral listant toutes les sessions de l'utilisateur, avec création de nouvelle session.
- Un panneau principal de conversation avec bulles de messages distinctes (utilisateur/assistant).
- Les réponses de l'assistant affichent les sources citées sous forme de badges cliquables, permettant de naviguer vers le document complet.
- Un sélecteur de langue (français/arabe) avec adaptation RTL dynamique.
- Des boutons de feedback (pouce haut/bas) sous chaque réponse de l'assistant.
- Un indicateur de chargement pendant la génération de la réponse.

4.1.6 Tests du Sprint 3

Le tableau ci-dessous récapitule les principaux cas de test exécutés pour valider le Sprint 3, avec leur scénario et le résultat attendu.

Tableau 4.4 : Cas de test — Sprint 3 : Assistant RAG

Cas de test	Entrée	Résultat attendu
Création de session	POST /api/chat/sessions	Session créée, id retourné
Question avec réponse sourcée	“Quelles sont les obligations de l'employeur en matière de congés payés?”	Réponse avec citations du Code du Travail
Question hors corpus	“Quel est le résultat du match de football?”	Réponse avouant l'ignorance, pas d'hallucination
Feedback positif	POST /api/chat/feedback avec rating=UP	Feedback enregistré, HTTP 201
Historique sessions	GET /api/chat/sessions	Liste des sessions avec titre et date
Suppression session	DELETE /api/chat/sessions/{id}	Session et messages supprimés, HTTP 204

4.1.7 Revue et rétrospective du Sprint 3

Le Sprint 3 livre l'assistant conversationnel RAG, fonctionnalité phare du portail. Toutes les *user stories* ont été réalisées : création de sessions de chat, réponses sourcées en langage naturel et évaluation des réponses. Les premières mesures de qualité (RAGAS) confirment une bonne fidélité aux sources. Les pistes d'amélioration identifiées :

- Réduction de la latence de génération par mise en cache des *embeddings* des questions fréquentes.
- Enrichissement du *prompt* pour mieux gérer les questions hors corpus.

4.2 Sprint 4 : Pipeline d'ingestion documentaire

4.2.1 Objectifs du Sprint 4

Le Sprint 4 développe le pipeline d'ingestion documentaire permettant aux administrateurs d'enrichir le corpus juridique du portail. Ce pipeline traite automatiquement les fichiers PDF (y compris les scannés) et Word, en extrayant leur contenu textuel, le découpant en chunks, générant les embeddings vectoriels et les indexant dans OpenSearch.

4.2.2 Backlog du Sprint 4

Le tableau ci-dessous présente le *Product Backlog* du Sprint 4, dédié au pipeline d'ingestion documentaire. Les *user stories* y sont priorisées et estimées en jours-homme.

Tableau 4.5 : Backlog du Sprint 4 — Pipeline d'ingestion

Id	User Story	Priorité	Estim. (j)
22	En tant qu'admin, je veux téléverser un ou plusieurs fichiers PDF/Word afin d'enrichir le corpus juridique.	1	3
23	En tant qu'administrateur, je veux que les fichiers téléversés soient archivés en stockage objet (MinIO/S3) afin de conserver les originaux.	1	2
24	En tant qu'administrateur, je veux que le texte des PDF numériques soit extrait (Apache Tika) afin de préparer l'indexation.	1	3
25	En tant qu'administrateur, je veux que les PDF scannés soient océrisés (Tesseract OCR, fr+ar) afin d'en récupérer le texte.	1	3

Id	User Story	Priorité	Estim. (j)
26	En tant qu'administrateur, je veux que le texte extrait soit découpé en segments (chunks) afin d'améliorer la pertinence des résultats.	1	2
27	En tant qu'administrateur, je veux que des embeddings soient générés pour chaque segment afin d'activer la recherche sémantique.	1	3
28	En tant qu'administrateur, je veux que les segments soient indexés dans OpenSearch afin de rendre les documents cherchables.	1	2
29	En tant qu'admin, je veux suivre l'état d'avancement du pipeline (en cours, réussi, échoué) par document.	2	2
Total Sprint 4			20

4.2.3 Planification des tâches du Sprint 4

Le tableau 4.6 détaille les tâches de conception et de développement réalisées durant ce sprint.

Tableau 4.6 : Tâches du Sprint 4

Numéro	Tâches
1	Élaboration de l'architecture du pipeline d'ingestion
2	Élaboration du diagramme de séquence : pipeline d'ingestion
3	Configuration du stockage objet MinIO
4	Codage de l'extraction de texte via Apache Tika
5	Intégration de Tesseract OCR (fr+ar) pour les PDF scannés
6	Codage du découpage en <i>chunks</i> (500 tokens, chevauchement 50)
7	Codage de la génération d'embeddings via Ollama
8	Codage de l'indexation OpenSearch (<i>Bulk API</i>)
9	Codage de la publication de l'événement RabbitMQ <code>document.indexed</code>
10	Codage de la gestion des erreurs et du <i>retry</i>
11	Codage du contrôleur <code>IngestController</code>
12	Développement de l'interface d'ingestion (glisser-déposer, suivi d'état)
13	Tests du Sprint 4

4.2.4 Conception du Sprint 4

4.2.4.1 Architecture du pipeline d'ingestion

Le pipeline d'ingestion documentaire se déroule en cinq étapes séquentielles :

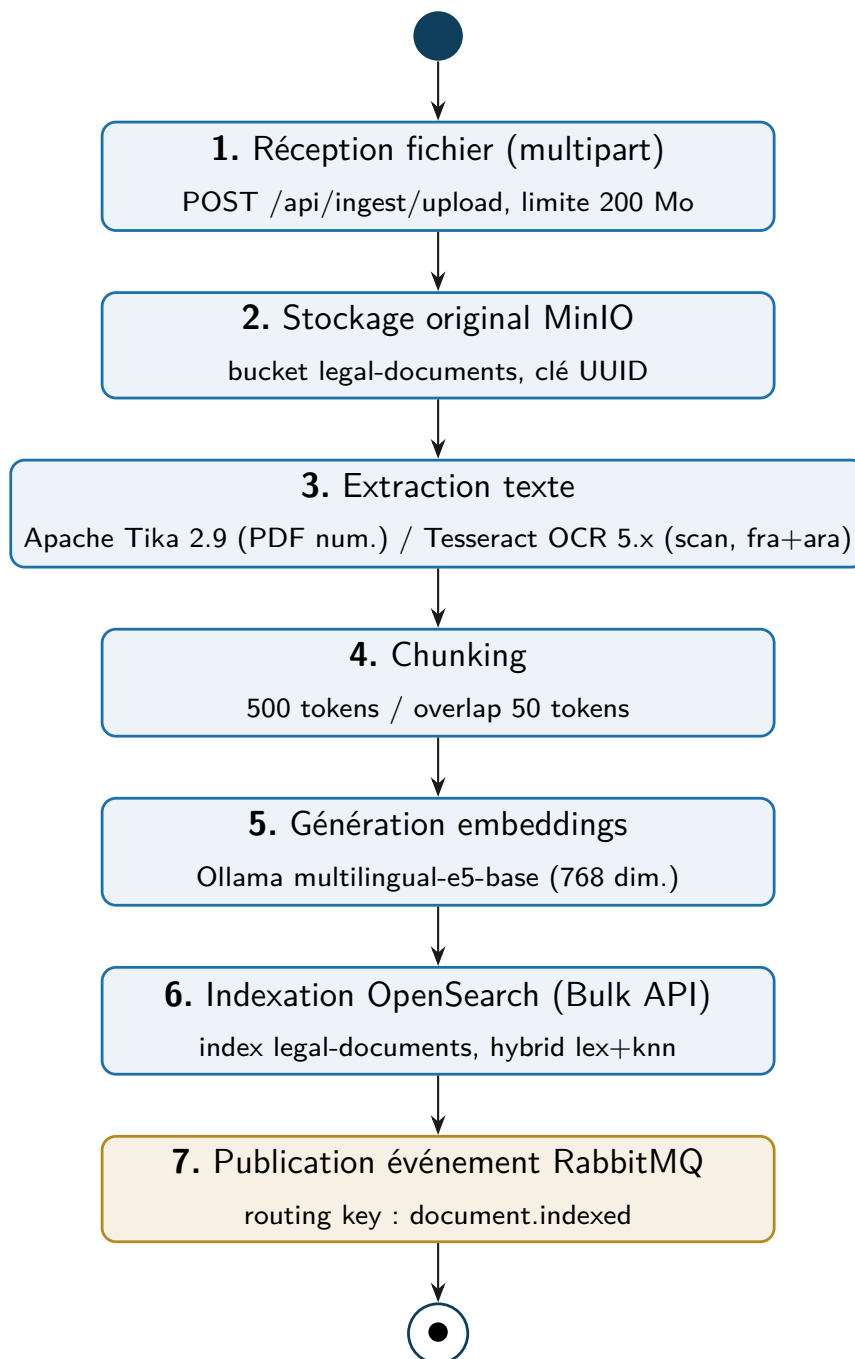


Figure 4.3 : Architecture du pipeline d'ingestion documentaire

1. **Réception et stockage :** Le fichier est reçu via une requête multipart (limite 200 Mo). Il est immédiatement stocké dans **MinIO** (bucket `legal-documents`) avec un identifiant UUID unique comme clé S3. Cette étape garantit la préservation de l'original.

2. Extraction de texte :

- Pour les **PDFs numériques** : Apache Tika 2.9.1 [W6] extrait directement le texte en préservant la structure (paragraphes, listes).
 - Pour les **PDFs scannés** : Tika détecte l'absence de texte et délègue à Tesseract OCR 5.x, configuré pour le français et l'arabe (**fra+ara**). Les images de pages sont converties via PDFBox avant d'être soumises à Tesseract.
 - Pour les **fichiers Word (.docx)** : Tika extrait le texte via Apache POI intégré.
3. **Chunking** : Le texte extrait est découpé en segments (*chunks*) de 500 tokens avec un chevauchement (*overlap*) de 50 tokens. Ce chevauchement assure la cohérence sémantique aux frontières des chunks. La taille de 500 tokens est optimisée pour le contexte du modèle Mistral (fenêtre de 4096 tokens) et pour les passages juridiques tunisiens.
 4. **Génération d'embeddings** : Chaque chunk est envoyé à l'API Ollama `/api/embeddings` avec le modèle `multilingual-e5-base`. Ce modèle multilingue (50+ langues, 768 dimensions) est particulièrement adapté au corpus bilingue français-arabe.
 5. **Indexation OpenSearch** : Les chunks enrichis (texte + embedding + métadonnées) sont indexés en batch via l'API Bulk d'OpenSearch. Un message `document.indexed` est publié dans RabbitMQ pour déclencher les notifications.

4.2.4.2 Diagramme de séquence : Pipeline d'ingestion

La figure 4.4 décrit les étapes du pipeline d'ingestion documentaire, de l'upload à l'indexation.

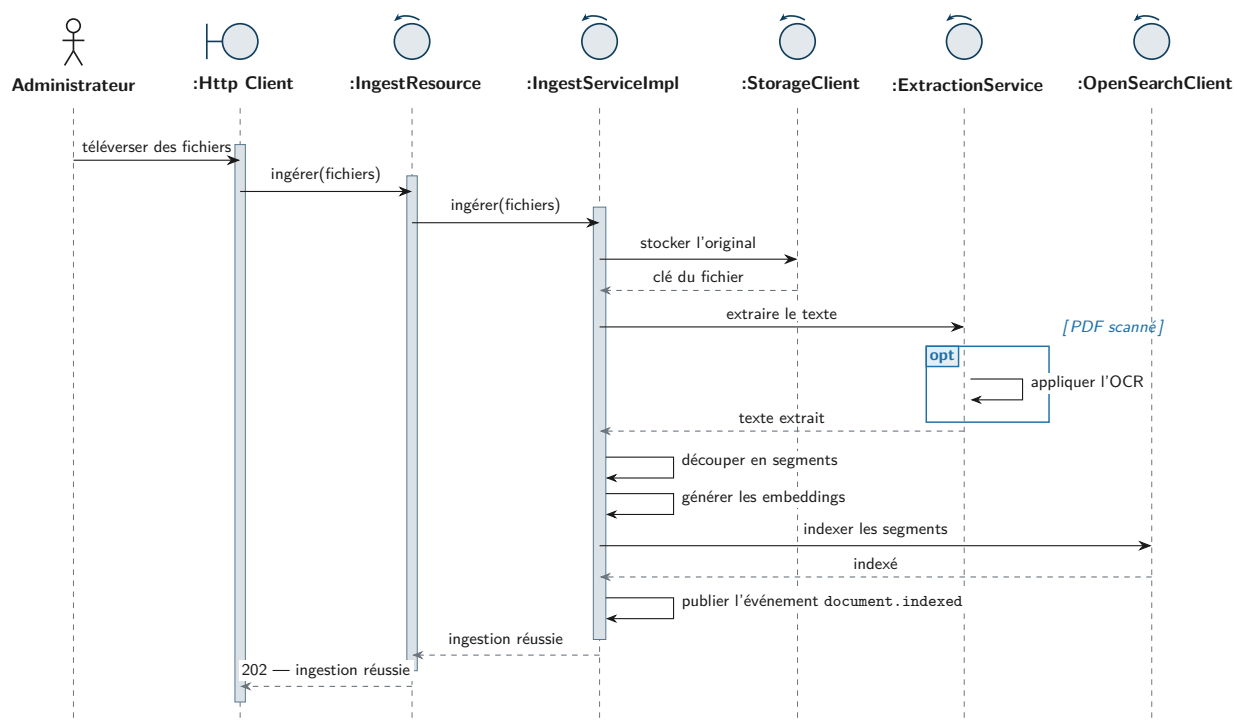


Figure 4.4 : Diagramme de séquence d'objet — Pipeline d'ingestion.

4.2.5 Réalisation du Sprint 4

4.2.5.1 Configuration MinIO

La classe `S3Config` configure le client MinIO (compatible S3) avec :

- URL de l'endpoint MinIO (configurable via `MINIO_URL` dans `.env`).
- Identifiants d'accès (`MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`).
- Création automatique du bucket `legal-documents` si inexistant au démarrage.
- Upload multipart pour les fichiers de grande taille.

4.2.5.2 Service d'ingestion

La classe `DocumentIngestionService` orchestre le pipeline :

- Méthode `ingest(MultipartFile file, DocumentMetadata meta)` appelée par le contrôleur.
- Appel à `S3Service.upload()` pour stocker l'original.
- Appel à `TextExtractionService.extract()` (Tika + détection OCR).
- Appel à `ChunkingService.chunk()` pour le découpage en 500 tokens.
- Pour chaque chunk : appel à `EmbeddingService.embed()` (API Ollama).
- Appel à `OpenSearchIndexService.bulkIndex()` pour l'indexation.
- Publication du message RabbitMQ via `RabbitTemplate`.

4.2.5.3 Gestion des erreurs et retry

Le pipeline implémente un mécanisme de *retry* exponentiel pour les étapes d'embedding et d'indexation. En cas d'échec d'une étape, le statut du document est mis à jour en `FAILED` avec le message d'erreur correspondant, permettant à l'administrateur de relancer l'ingestion.

4.2.5.4 API du ingest-service

Le tableau ci-dessous récapitule les principaux *endpoints* REST exposés par le `ingest-service`.

Tableau 4.7 : API REST du ingest-service

Méthode	Endpoint	Description
POST	<code>/api/ingest/upload</code>	Téléverser un fichier (multipart, max 200Mo)
POST	<code>/api/ingest/batch</code>	Téléverser plusieurs fichiers en lot
GET	<code>/api/ingest/status</code>	Liste des documents avec statut d'ingestion

Méthode	Endpoint	Description
GET	/api/ingest/status/{id}	Statut d'ingestion d'un document spécifique
DELETE	/api/ingest/{id}	Supprimer un document (MinIO + OpenSearch)

4.2.5.5 Interface utilisateur — Ingestion de documents

La figure 4.5 présente l'interface d'ingestion de documents avec sa zone de glisser-déposer.

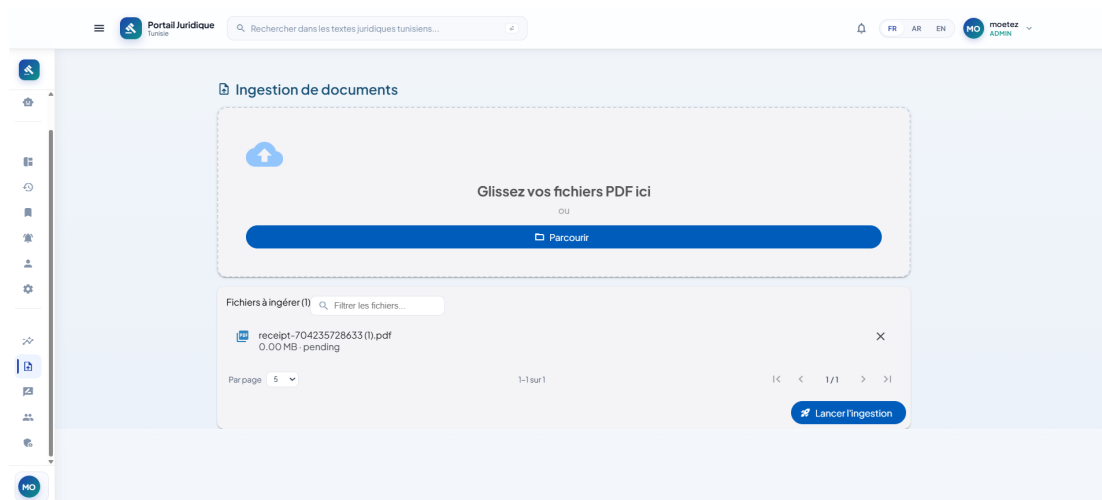


Figure 4.5 : Interface d'ingestion de documents — Zone de glisser-déposer

L'interface d'ingestion propose :

- Une zone de glisser-déposer (*drag & drop*) acceptant les fichiers PDF et Word.
- Un formulaire de métadonnées (type de texte, langue, date de promulgation).
- Une liste des fichiers sélectionnés avec indicateurs de progression individuels.
- Un tableau de statut montrant l'état de chaque document ingéré (EN_ATTENTE, EN_COURS, REUSSI, ECHOUE).
- Un bouton de relancement pour les documents en échec.

4.2.6 Tests du Sprint 4

Le tableau ci-dessous récapitule les principaux cas de test exécutés pour valider le Sprint 4, avec leur scénario et le résultat attendu.

Tableau 4.8 : Cas de test — Sprint 4 : Pipeline d'ingestion

Cas de test	Entrée	Résultat attendu
Upload PDF numérique	Fichier PDF avec texte	Texte extrait, chunks créés, indexé dans OpenSearch
Upload PDF scanné	Fichier PDF image	OCR appliqué, texte extrait, indexé
Upload Word (.docx)	Fichier Word	Texte extrait via POI, indexé
Fichier trop grand	Fichier > 200Mo	HTTP 413, message d'erreur
Type de fichier invalide	Fichier .exe	HTTP 400, type non supporté
Message RabbitMQ	Après ingestion réussie	Message document.indexed publié
Recherche post-ingestion	Recherche sur le nouveau doc	Document retrouvé en BM25 et kNN

4.2.7 Revue et rétrospective du Sprint 4

La Release 2 est complète. Le portail dispose désormais d'un assistant conversationnel RAG performant et d'un pipeline d'ingestion robuste gérant les PDFs scannés. Les performances mesurées pour l'OCR sont de l'ordre de 3–5 secondes par page sur un PDF scanné en 300 DPI. Les pistes d'amélioration identifiées :

- Parallélisation du traitement OCR pour les documents multi-pages.
- Ajout d'une file d'attente RabbitMQ pour l'ingestion asynchrone des gros fichiers.

Conclusion

La Release 2 a permis de développer les deux fonctionnalités les plus innovantes du portail : l'assistant conversationnel RAG basé sur Mistral et le pipeline d'ingestion documentaire avec OCR. Ces composants constituent la différenciation majeure du portail par rapport aux solutions existantes. Nous abordons maintenant la Release 3 avec les services de notifications, d'alertes et le tableau de bord analytique.

Release 3 : Notifications, Alertes et Tableau de Bord Analytique

Plan

Introduction	14
1 Spécification des besoins	14
2 Diagramme de cas d'utilisation global	16
3 Modèle de données	17
4 Pilotage du projet avec Scrum	20
5 Architecture du système	26
6 Environnement de travail	28
Conclusion	38

Introduction

La Release 3 complète le portail juridique intelligent en ajoutant deux dimensions essentielles : le **Sprint 5** consacré au service de notifications et d’alertes thématiques basé sur RabbitMQ et JavaMail, et le **Sprint 6** dédié au tableau de bord analytique et au module d’administration avancé incluant la gestion des retours RAG et les statistiques d’utilisation.

5.1 Sprint 5 : Notifications et alertes thématiques

5.1.1 Objectifs du Sprint 5

L’objectif du Sprint 5 est de développer un système de notifications proactives permettant aux utilisateurs de rester informés des nouvelles publications juridiques correspondant à leurs centres d’intérêt. Le service écoute les événements d’ingestion publiés par RabbitMQ et envoie des emails personnalisés aux utilisateurs abonnés aux thèmes concernés. Le sprint inclut également la gestion des comptes utilisateurs par les administrateurs.

5.1.2 Backlog du Sprint 5

Le tableau ci-dessous détaille le *Product Backlog* du Sprint 5, consacré aux notifications et aux alertes thématiques. Chaque besoin y est exprimé sous forme de *user story* priorisée et estimée en jours-homme.

Tableau 5.1 : Backlog du Sprint 5 — Notifications et alertes

Id	User Story	Priorité	Estim. (j)
30	En tant qu’utilisateur, je veux créer une alerte thématique avec un ou plusieurs mots-clés afin d’être notifié des nouvelles lois pertinentes.	2	3
31	En tant qu’utilisateur, je veux activer une alerte existante afin de réactiver mes notifications.	2	2
32	En tant qu’utilisateur, je veux désactiver une alerte sans la supprimer afin de suspendre mes notifications.	2	2
33	En tant qu’utilisateur, je veux supprimer une alerte dont je n’ai plus besoin.	2	2
34	En tant qu’utilisateur abonné, je veux que mes alertes soient déclenchées dès qu’un nouveau document correspond à mes mots-clés afin d’être tenu informé.	1	3

Id	User Story	Priorité	Estim. (j)
35	En tant qu'utilisateur abonné, je veux recevoir un email de notification personnalisé à chaque nouveau document correspondant à mon alerte.	1	3
36	En tant qu'admin, je veux consulter la liste de tous les utilisateurs avec leur rôle et statut.	2	2
37	En tant qu'admin, je veux activer un compte utilisateur afin d'autoriser son accès à la plateforme.	2	2
38	En tant qu'admin, je veux désactiver un compte utilisateur afin de bloquer son accès à la plateforme.	2	2
Total Sprint 5			21

5.1.3 Planification des tâches du Sprint 5

Le tableau 5.2 détaille les tâches de conception et de développement réalisées durant ce sprint.

Tableau 5.2 : Tâches du Sprint 5

Numéro	Tâches
1	Élaboration de l'architecture du service de notifications
2	Élaboration du diagramme de séquence : déclenchement d'une alerte
3	Configuration de RabbitMQ (échange, file, <i>routing key</i>)
4	Codage de l'entité Alert et de son <i>repository</i>
5	Codage du <i>listener</i> RabbitMQ de l'événement <code>document.indexed</code>
6	Codage du service d'alertes AlertService
7	Codage du service d'email EmailService (<i>template</i> , SMTP)
8	Codage de l'administration des comptes (UserAdminController)
9	Développement des interfaces d'alertes et d'administration (Angular)
10	Tests du Sprint 5

5.1.4 Conception du Sprint 5

5.1.4.1 Architecture du service de notifications

Le **notification-service** est un service réactif qui n'expose que très peu d'endpoints REST publics ; il est principalement piloté par les messages RabbitMQ. Son architecture est la suivante :

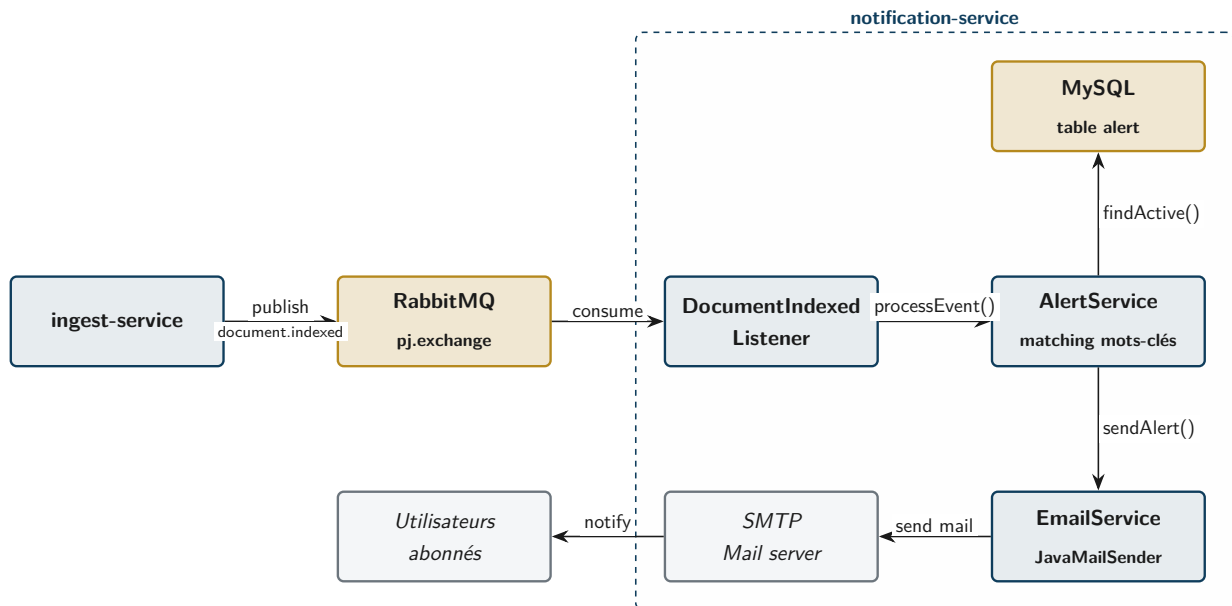


Figure 5.1 : Architecture du notification-service

1. Lors de l'ingestion d'un nouveau document, l'**ingest-service** publie un message JSON dans l'échange RabbitMQ **document-exchange** avec la routing key **document.indexed**. Le message contient le titre, le type, la langue et les mots-clés extraits du document.
2. Le **DocumentIndexedListener** du **notification-service** reçoit ce message via l'annotation **@RabbitListener**.
3. L'**AlertService** requête la base de données pour trouver toutes les alertes actives dont les mots-clés correspondent au document ingéré (correspondance insensible à la casse).
4. Pour chaque alerte correspondante, l'**EmailService** construit et envoie un email HTML personnalisé via **JavaMailSender**.

5.1.4.2 Diagramme de séquence : Déclenchement d’une alerte

La figure 5.2 illustre le déclenchement d’une alerte thématique à l’arrivée d’un nouveau document.

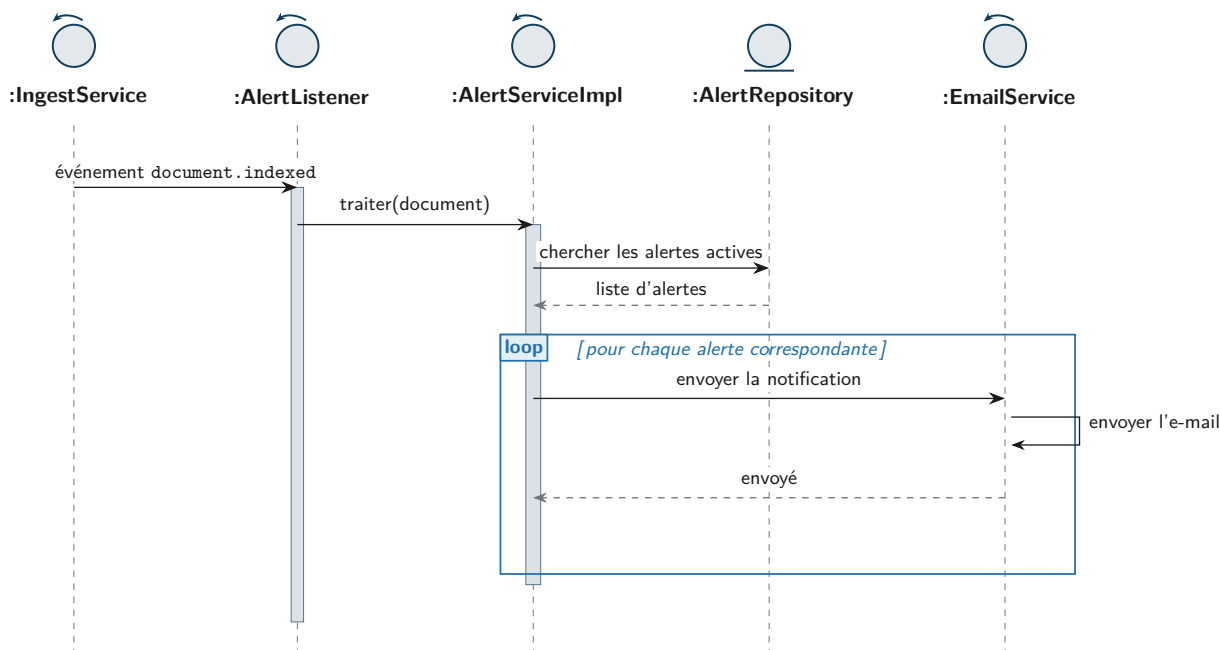


Figure 5.2 : Diagramme de séquence d’objet — Déclenchement d’une alerte thématique.

5.1.5 Réalisation du Sprint 5

5.1.5.1 Configuration RabbitMQ

La classe `RabbitConfig` configure :

- Un échange de type `TopicExchange` nommé `document-exchange`.
- Une file de messages (*queue*) `alert-queue` avec durabilité activée.
- Un binding entre la queue et l’échange avec la routing key `document.indexed`.
- Un `MessageConverter` Jackson pour la sérialisation/désérialisation JSON.

5.1.5.2 Template d’email de notification

Les emails de notification sont générés avec un template HTML incluant :

- Le titre du nouveau document juridique.
- Le type (Loi, Décret, Arrêté, Circulaire).
- La date de promulgation.
- Un bouton “Consulter” liant directement au document dans le portail.
- Le mot-clé de l’alerte ayant déclenché la notification.

- Un lien de désabonnement en un clic.

5.1.5.3 Modèle de données — Alertes

L'entité **Alert** stocke :

- **id** : identifiant auto-généré.
- **userId** : identifiant de l'utilisateur propriétaire.
- **email** : adresse email de destination (peut différer du compte).
- **keyword** : le mot-clé ou l'expression de surveillance.
- **active** : flag permettant d'activer/désactiver sans supprimer.
- **createdAt** : date de création.
- **lastTriggeredAt** : date du dernier déclenchement.

5.1.5.4 API du notification-service

Le tableau ci-dessous récapitule les principaux *endpoints* REST exposés par le **notification-service**.

Tableau 5.3 : API REST du notification-service

Méthode	Endpoint	Description
GET	/api/alerts	Liste des alertes de l'utilisateur courant
POST	/api/alerts	Créer une nouvelle alerte
PATCH	/api/alerts/{id}/toggle	Activer/désactiver une alerte
DELETE	/api/alerts/{id}	Supprimer une alerte
GET	/api/admin/users	[ADMIN] Liste de tous les utilisateurs
PATCH	/api/admin/users/{id}/toggle	[ADMIN] Activer/désactiver un compte

5.1.5.5 Interface utilisateur — Alertes et administration

La figure 5.3 présente l’interface de gestion des alertes thématiques.

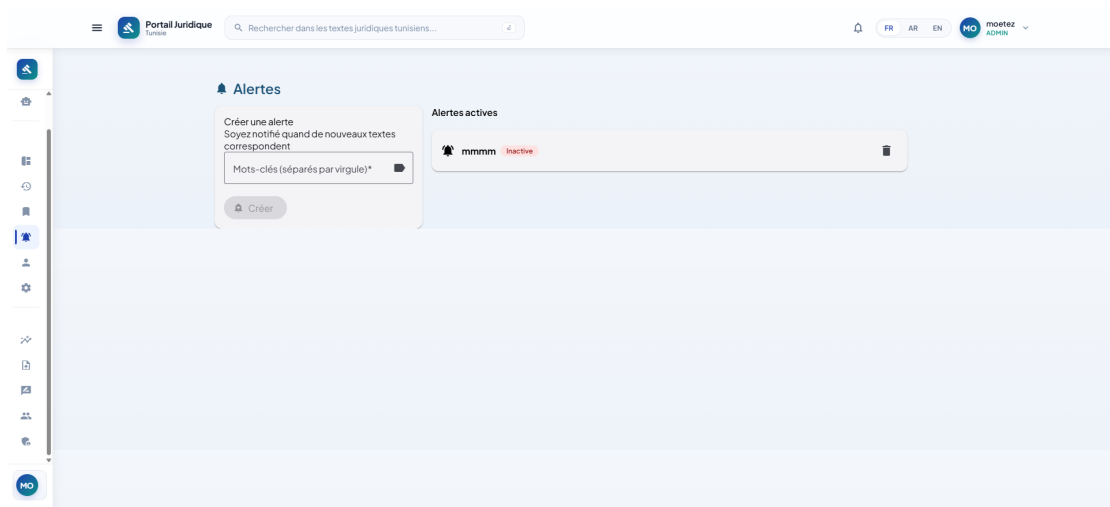


Figure 5.3 : Interface de gestion des alertes thématiques

L’interface de gestion des alertes propose :

- Un formulaire de création d’alerte avec champ mot-clé et adresse email de destination.
- La liste des alertes actives avec toggle ON/OFF et bouton de suppression.
- Un indicateur visuel de la date du dernier déclenchement.

5.1.6 Tests du Sprint 5

Le tableau ci-dessous récapitule les principaux cas de test exécutés pour valider le Sprint 5, avec leur scénario et le résultat attendu.

Tableau 5.4 : Cas de test — Sprint 5 : Notifications et alertes

Cas de test	Entrée	Résultat attendu
Création alerte	POST avec keyword=“décret”	Alerte créée, HTTP 201
Déclenchement alerte	Ingestion d’un document avec “décret”	Email envoyé à l’utilisateur abonné
Alerte désactivée	Toggle OFF puis ingestion	Aucun email envoyé
Suppression alerte	DELETE /api/alerts/{id}	Alerte supprimée, HTTP 204
Gestion utilisateur	PATCH /api/admin/users/{id}/toggle	Compte désactivé, HTTP 200

Cas de test	Entrée	Résultat attendu
Accès refusé	Appel /api/admin par CITIZEN	HTTP 403, Forbidden

5.1.7 Revue et rétrospective du Sprint 5

Le Sprint 5 met en place le système d’alertes thématiques et de notifications par e-mail, piloté par les événements RabbitMQ. Toutes les *user stories* ont été livrées, y compris la gestion fine des alertes (activation, désactivation et suppression) et l’administration des comptes utilisateurs. Les pistes d’amélioration identifiées :

- Regroupement (*batching*) des notifications pour éviter la sur-sollicitation des utilisateurs.
- Ajout d’un canal de notification complémentaire (notification *in-app*).

5.2 Sprint 6 : Tableau de bord analytique et administration

5.2.1 Objectifs du Sprint 6

Le Sprint 6 finalise le portail avec un module analytique complet donnant à l’administrateur une vision globale de l’utilisation de la plateforme : statistiques de recherche, analyse des retours RAG, top requêtes, journaux d’audit et graphiques d’évolution temporelle. Ce sprint complète également les fonctionnalités du dashboard utilisateur (historique de recherches, signets).

5.2.2 Backlog du Sprint 6

Le tableau ci-dessous présente le *Product Backlog* du Sprint 6, dédié au tableau de bord analytique et à l’administration avancée. Les *user stories* y sont priorisées et estimées en jours-homme.

Tableau 5.5 : Backlog du Sprint 6 — Tableau de bord analytique

Id	User Story	Priorité	Estim. (j)
39	En tant qu’admin, je veux consulter des KPIs globaux (nombre d’utilisateurs, recherches, documents, sessions chat) afin d’avoir une vue d’ensemble de la plateforme.	1	3
40	En tant qu’admin, je veux consulter un graphique de l’évolution des recherches par jour sur les 30 derniers jours.	2	3

Id	User Story	Priorité	Estim. (j)
41	En tant qu'admin, je veux consulter le top 10 des requêtes les plus fréquentes afin d'identifier les besoins des utilisateurs.	2	2
42	En tant qu'admin, je veux consulter les retours RAG (positifs/négatifs) avec les commentaires des utilisateurs afin d'améliorer l'assistant.	2	3
43	En tant qu'utilisateur, je veux que mes recherches soient enregistrées dans mon historique afin de pouvoir les retrouver et les relancer.	1	2
44	En tant qu'utilisateur, je veux consulter mon historique de recherches afin de retrouver mes requêtes passées.	2	2
45	En tant qu'utilisateur, je veux relancer une recherche depuis mon historique afin de gagner du temps.	2	2
46	En tant qu'admin, je veux consulter les journaux d'audit des actions administratives pour la traçabilité.	3	2
Total Sprint 6			19

5.2.3 Planification des tâches du Sprint 6

Le tableau 5.6 détaille les tâches de conception et de développement réalisées durant ce sprint.

Tableau 5.6 : Tâches du Sprint 6

Numéro	Tâches
1	Élaboration de l'architecture de l' analytics-service
2	Codage des entités SearchHistory et AuditLog
3	Codage du calcul des KPIs (utilisateurs, recherches, documents, sessions)
4	Codage du service analytique AnalyticsService (statistiques, top requêtes, feedback)
5	Codage de l'enregistrement de l'historique et des journaux d'audit
6	Codage du contrôleur AnalyticsController

Numéro	Tâches
7	Finalisation du module RBAC (matrice des permissions)
8	Développement du tableau de bord administrateur (KPIs, graphiques)
9	Développement du tableau de bord utilisateur (historique, signets)
10	Tests du Sprint 6

5.2.4 Conception du Sprint 6

5.2.4.1 Architecture de l'analytics-service

L'**analytics-service** joue un rôle transversal : il reçoit des données de tous les autres services et les agrège pour produire des métriques exploitables. Il expose des endpoints REST protégés par rôle (ADMIN uniquement pour les stats globales, authentifié pour l'historique personnel).

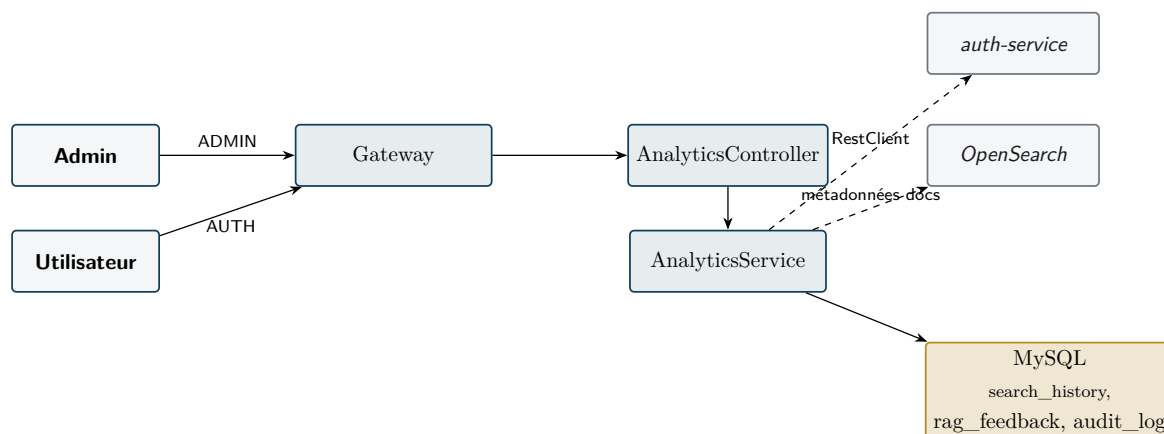


Figure 5.4 : Architecture de l'analytics-service

5.2.5 Réalisation du Sprint 6

5.2.5.1 Modèle de données

L'**analytics-service** gère trois entités :

- **SearchHistory** : Enregistrement de chaque recherche effectuée. Champs : `id`, `userId`, `query`, `searchMode` (LEXICAL/SEMANTIC/HYBRID), `resultCount`, `durationMs`, `createdAt`.
- **RagFeedback** : Retour utilisateur sur une réponse RAG. Champs : `id`, `messageId`, `userId`, `rating` (UP/DOWN), `comment`, `createdAt`.
- **AuditLog** : Journal des actions administratives. Champs : `id`, `userId`, `action` (INGEST, USER_DISABLE, USER_PROMOTE...), `resource`, `details` (JSON), `ipAddress`, `createdAt`.

5.2.5.2 Calcul des KPIs

Le service calcule les indicateurs clés de performance suivants :

- **Utilisateurs totaux/actifs** : Requête sur `user_accounts` via RestClient vers l'`auth-service`.
- **Documents indexés** : Requête sur l'API stats d'OpenSearch.
- **Recherches (30j)** : Agrégation `COUNT(*)` sur `search_history` avec filtre de date.
- **Sessions chat (30j)** : Requête vers le `chat-service`.
- **Taux de satisfaction RAG** : `COUNT(rating=UP) / COUNT(*)` sur `rag_feedback`.
- **Top 10 requêtes** : `GROUP BY query ORDER BY COUNT DESC LIMIT 10`.
- **Évolution temporelle** : `GROUP BY DATE(created_at)` pour les 30 derniers jours.

5.2.5.3 API de l'analytics-service

Le tableau ci-dessous récapitule les principaux *endpoints* REST exposés par l'`analytics-service`.

Tableau 5.7 : API REST de l'analytics-service

Méthode	Endpoint	Description
GET	<code>/api/analytics/kpis</code>	[ADMIN] KPIs globaux de la plateforme
GET	<code>/api/analytics/daily</code>	[ADMIN] Recherches par jour (30 derniers jours)
GET	<code>/api/analytics/top-queries</code>	[ADMIN] Top 10 des requêtes les plus fréquentes
GET	<code>/api/analytics/feedback</code>	[ADMIN] Liste des feedbacks RAG avec stats
GET	<code>/api/analytics/audit-logs</code>	[ADMIN] Journal d'audit avec pagination
GET	<code>/api/analytics/history</code>	[AUTH] Historique de recherches de l'utilisateur
POST	<code>/api/analytics/history</code>	[INTERNE] Enregistrer une entrée d'historique
POST	<code>/api/analytics/feedback</code>	[AUTH] Soumettre un feedback RAG

5.2.5.4 Interface utilisateur — Tableau de bord admin

La figure 5.5 présente le tableau de bord administrateur (KPIs et graphiques).

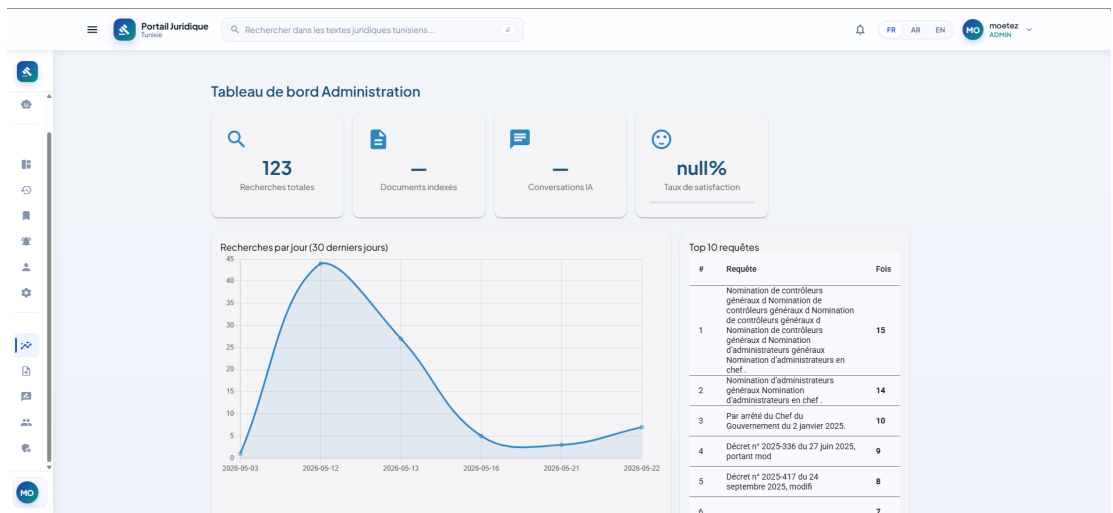


Figure 5.5 : Tableau de bord administrateur — KPIs et graphiques

Le tableau de bord administrateur propose :

- **Cartes KPI** : 4 cartes affichant le nombre total d'utilisateurs, de documents indexés, de recherches sur 30 jours, et le taux de satisfaction RAG.
- **Graphique en ligne** : Évolution du nombre de recherches par jour sur les 30 derniers jours, réalisé avec *ng2-charts* (Chart.js) via *BaseChartDirective*.
- **Tableau Top Requêtes** : Top 10 des requêtes les plus fréquentes avec compteur et mode de recherche prédominant.
- **Tableau Feedbacks** : Liste des retours RAG avec notation, commentaire, et lien vers la session de chat concernée.

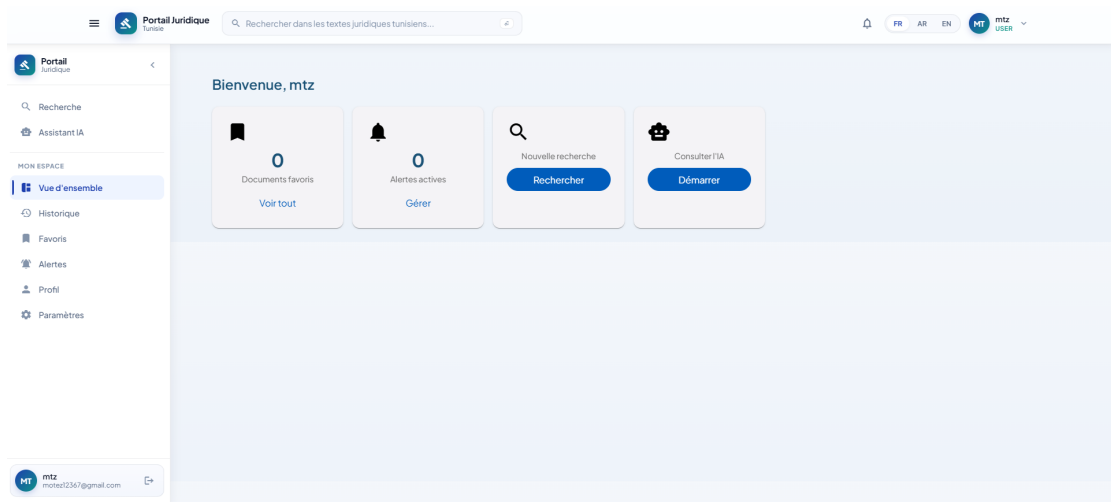


Figure 5.6 : Tableau de bord utilisateur — Historique et signets

Le tableau de bord utilisateur propose :

- Un résumé des activités récentes (dernières recherches, derniers signets).
- L'historique complet des recherches avec mode, nombre de résultats et date.
- La grille des signets avec titre, type et bouton de suppression.
- La liste des alertes actives avec état ON/OFF.

5.2.6 Contrôle d'accès basé sur les rôles

Le module d'administration avancé du Sprint 6 s'appuie sur une architecture de contrôle d'accès basé sur les rôles (*Role-Based Access Control*, RBAC), entièrement **persistée en base de données** et **configurable**. Cette approche se substitue à un système de rôles codé en dur : l'ajout d'un rôle ou la modification des permissions d'un rôle existant se ramène à une simple opération sur les données, sans recompilation ni redéploiement de l'application.

5.2.6.1 Modèle de données

Le modèle repose sur trois tables (**roles**, **permissions** et la table d'association **role_permissions** matérialisant la relation *plusieurs-à-plusieurs* entre rôles et permissions), ainsi que sur l'enrichissement de la table **users** par trois colonnes de profil (**first_name**, **last_name**, **avatar**). L'ensemble de ces évolutions de schéma est porté par une migration **Flyway** additive (V3), qui crée les tables, insère les données de référence et préserve les données existantes. Le diagramme de classes global présenté à la figure 2.2 intègre les entités **Role** et **Permission** ainsi que les nouveaux champs de profil.

5.2.6.2 Rôles, permissions et matrice d'affectation

Cinq rôles de référence sont fournis (**ADMIN**, **MANAGER**, **JURIST**, **ANALYSTE** et **USER**) et dix permissions atomiques couvrent la gestion des utilisateurs, la gestion documentaire, la gestion des retours et l'accès à l'espace d'administration. La matrice du tableau 5.8 synthétise l'affectation des permissions aux rôles : une coche indique que le rôle (en ligne) dispose de la permission (en colonne).

Tableau 5.8 : Matrice d'affectation des permissions par rôle.

Permission	ADMIN	MANAGER	JURIST	ANALYSTE	USER
Gestion des utilisateurs					
USER_READ	✓	✓	✗	✗	✗
USER_CREATE	✓	✗	✗	✗	✗
USER_UPDATE	✓	✗	✗	✗	✗
USER_DELETE	✓	✗	✗	✗	✗
Documents juridiques					
DOCUMENT_READ	✓	✓	✓	✓	✓
DOCUMENT_UPLOAD	✓	✓	✓	✗	✗
DOCUMENT_DELETE	✓	✓	✗	✗	✗
Retours RAG (feedback)					
FEEDBACK_READ	✓	✓	✓	✓	✗
FEEDBACK_MANAGE	✓	✓	✗	✗	✗
Administration					
ADMIN_ACCESS	✓	✓	✗	✓	✗

✓ = permission accordée ✗ = permission refusée

5.2.6.3 Points d'accès et vérification des permissions

L'administration des utilisateurs et des rôles est exposée par le **auth-service** au travers d'*endpoints* protégés, chacun exigeant une permission précise (tableau 5.9). La vérification est centralisée dans le composant **RbacService**, qui lève une réponse HTTP 403 lorsque le rôle de l'appelant — lu depuis l'en-tête **X-User-Role** injecté par la passerelle — ne dispose pas de la permission requise.

Tableau 5.9 : Points d'accès d'administration et permissions requises.

Méthode	Route	Permission requise
GET	/api/users	USER_READ
POST	/api/users	USER_CREATE
PUT	/api/users/{id}	USER_UPDATE
DELETE	/api/users/{id}	USER_DELETE
GET	/api/roles	ADMIN_ACCESS
POST	/api/roles	ADMIN_ACCESS
PUT	/api/roles/{id}/permissions	ADMIN_ACCESS
DELETE	/api/roles/{id}	ADMIN_ACCESS

5.2.6.4 Interfaces d'administration

Côté frontend Angular, deux écrans matérialisent la configurabilité du RBAC. L'écran de **gestion des utilisateurs** présente un tableau paginé doté d'une recherche et du tri des colonnes ; l'administrateur y modifie le rôle en ligne, active ou désactive un compte, supprime un utilisateur ou en crée un nouveau. L'écran de **gestion des rôles** affiche, pour chaque rôle, l'ensemble des permissions sous forme de cases à cocher, et permet de créer ou supprimer des rôles. La sécurité de l'interface repose sur une garde de route dédiée (`permissionGuard`) qui masque automatiquement les menus et actions non autorisés à partir des permissions de l'utilisateur, résolues côté serveur lors de l'authentification.

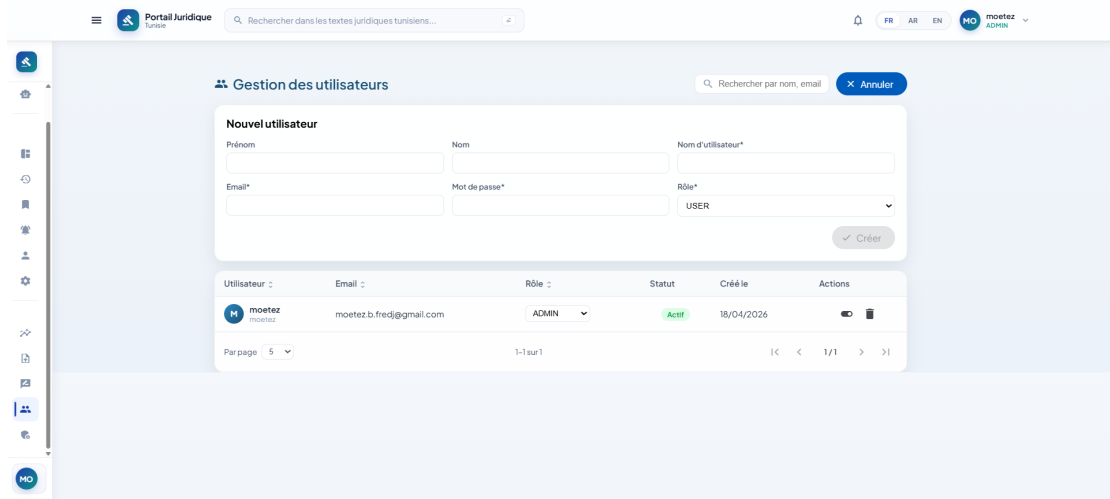


Figure 5.7 : Interface de gestion des utilisateurs

La figure 5.8 montre l'écran de gestion des rôles et des permissions. Pour chaque rôle, l'administrateur visualise et ajuste l'ensemble des permissions associées sous forme de cases à cocher, et peut créer ou supprimer des rôles, ce qui matérialise concrètement le contrôle d'accès

basé sur les rôles.

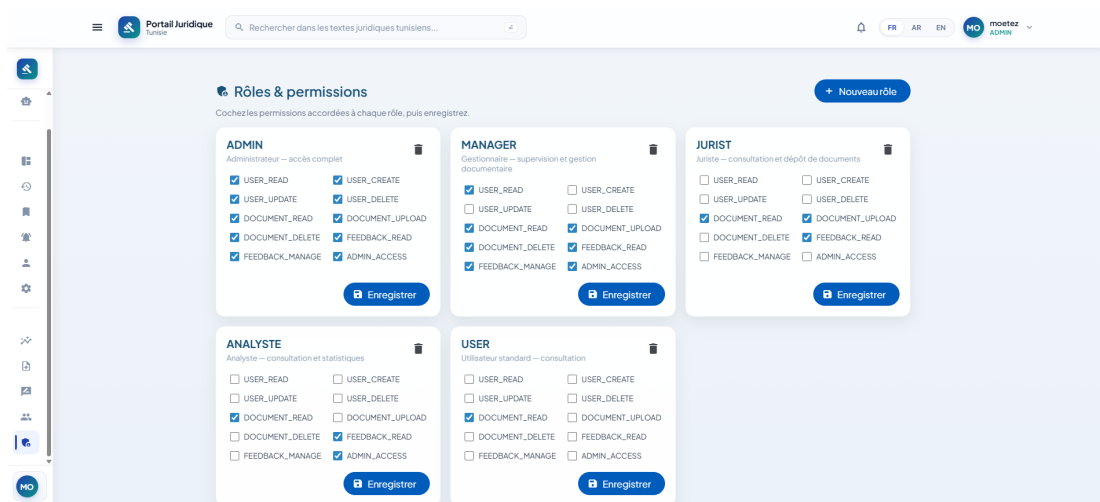


Figure 5.8 : Interface de gestion des rôles et permissions

5.2.6.5 Gestion du profil utilisateur

En complément du RBAC, la page de profil permet désormais à chaque utilisateur de mettre à jour ses informations (nom, prénom, e-mail) et de téléverser une photo d'**avatar**. Le fichier sélectionné est validé (type image, taille maximale de 2 Mo), encodé en base64, puis envoyé via la route protégée PUT `/api/profile` et persisté dans la colonne **avatar** de la table **users**. À la réception de la réponse, le *store* local est mis à jour, ce qui rafraîchit instantanément l'avatar affiché sans rechargement de page. La mise à jour de l'e-mail gère par ailleurs le conflit éventuel (réponse HTTP 409 si l'adresse est déjà utilisée).

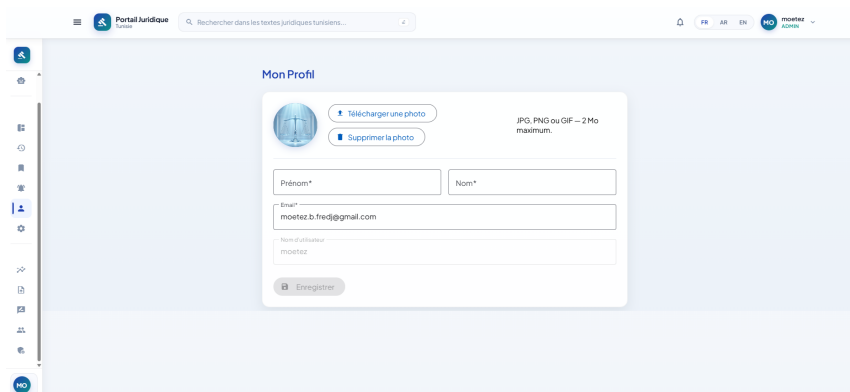


Figure 5.9 : Interface de gestion du profil utilisateur

5.2.7 Tests du Sprint 6

Le tableau ci-dessous récapitule les principaux cas de test exécutés pour valider le Sprint 6, avec leur scénario et le résultat attendu.

Tableau 5.10 : Cas de test — Sprint 6 : Tableau de bord analytique

Cas de test	Entrée	Résultat attendu
KPIs admin	GET /api/analytics/kpis	JSON avec tous les indicateurs
Données graphique	GET /api/analytics/daily	Tableau de 30 entrées date/count
Top requêtes	GET /api/analytics/top-queries	Top 10 avec query et count
Historique utilisateur	GET /api/analytics/history	Liste paginée des recherches
Enregistrement auto	Recherche via search-service	Entrée créée dans search_history
Accès admin refusé	Appel KPIs par CITIZEN	HTTP 403, Forbidden
Journal d'audit	GET /api/analytics/audit-logs	Liste paginée des actions admin

5.2.8 Revue et rétrospective du Sprint 6

Le Sprint 6 finalise le tableau de bord analytique et le contrôle d'accès basé sur les rôles (RBAC). Toutes les *user stories* ont été réalisées : KPIs globaux, graphiques d'évolution, top des requêtes, analyse des retours RAG, historique de recherches et journaux d'audit. Les pistes d'amélioration identifiées :

- Mise en cache des agrégations analytiques les plus coûteuses.
- Export des rapports analytiques au format PDF.

5.3 Revue et rétrospective globale

La Release 3 et l'ensemble du projet sont désormais complets. Le portail juridique intelligent est opérationnel avec ses 8 microservices, son interface bilingue Angular 21 et son infrastructure Docker Compose. Le bilan global des sprints est le suivant :

Tableau 5.11 : Bilan global des sprints

Sprint	Durée	Stories réalisées	Stories reportées
Sprint 1	3 semaines	7/7 (100%)	0
Sprint 2	3 semaines	8/8 (100%)	0
Sprint 3	3 semaines	6/6 (100%)	0
Sprint 4	3 semaines	8/8 (100%)	0
Sprint 5	3 semaines	9/9 (100%)	0
Sprint 6	3 semaines	8/8 (100%)	0
TOTAL	18 semaines	46/46 (100%)	0

Conclusion

La Release 3 achève le développement du portail juridique intelligent. Grâce aux services de notifications intelligentes et au tableau de bord analytique, la plateforme offre désormais une expérience complète : de l'ingestion documentaire à l'analyse de l'usage, en passant par la recherche hybride, l'assistant RAG et les alertes thématiques. Nous présentons dans la conclusion générale le bilan du projet et les perspectives d'évolution.

Pipeline d'extraction et d'indexation du corpus juridique

Plan

Introduction	40
1 Sprint 1 : Service d'authentification et gestion des utilisateurs	40
2 Sprint 2 : Moteur de recherche hybride et signets	46
Conclusion	52

Introduction

Notre système conversationnel n'est rien sans la qualité du corpus qu'il exploite. Pour le portail juridique tunisien, ce corpus est constitué de **261 fichiers PDF** issus du *Journal Officiel de la République Tunisienne*, rédigés en français et en arabe à parts presque égales. Obtenir de ces documents un contenu propre, structuré et exploitable par un moteur de recherche s'est avéré l'un des défis les plus exigeants que nous ayons rencontrés tout au long du projet.

Dans ce chapitre, nous décrivons l'évolution de notre approche, qui s'est faite en trois itérations successives. Chaque itération nous a appris quelque chose de fondamental sur les limites de la précédente. La première a reposé sur des outils classiques d'extraction texte avec PyMuPDF et Apache Tika. La deuxième a introduit un grand modèle de langage textuel (Ministral) pour tenter de nettoyer la sortie. La troisième, qui constitue aujourd'hui notre solution opérationnelle, mobilise un modèle vision-langage (Qwen2.5-VL) capable de « lire » les pages comme un humain. Nous présentons pour chaque itération l'architecture, les obstacles rencontrés, les correctifs appliqués et les chiffres mesurés sur notre propre corpus.

La dernière partie du chapitre est consacrée à la stratégie d'indexation OpenSearch qui consomme les documents extraits.

6.1 Audit préalable du corpus JORT

Avant de concevoir le moindre *pipeline*, nous avons jugé indispensable de caractériser objectivement le corpus. Notre script *audit-corpus.py* ouvre chaque PDF, mesure la densité textuelle extractible et détermine la présence éventuelle de caractères arabes en parcourant les pages Unicode U+0600–U+06FF. Cette routine prend environ 45 secondes pour traiter les 261 fichiers.

Les résultats que nous avons obtenus sont synthétisés dans le tableau 6.1. Ils traduisent immédiatement plusieurs points clés pour notre conception.

Tableau 6.1 : Indicateurs d'audit que nous avons mesurés sur le corpus JORT 2025.

Indicateur	Valeur	Observation
Total PDFs	261	131 arabes + 130 français
Natifs textuels	222 (85%)	PyMuPDF lit directement
Scannés (OCR requis)	39 (15%)	moins de 50 caractères / page
Avec caractères arabes	109 (42%)	présence détectée par regex
Avec caractères français	152 (58%)	—
Moyenne pages / fichier	20	cohérent avec un JORT classique
Problèmes d'encodage	0	UTF-8 systématique en 2025

Trois constats ont orienté nos choix par la suite. D'abord, le fait que 85 % des fichiers soient déjà textuels nous a évité de devoir généraliser une chaîne OCR à tout le corpus, ce qui aurait pesé lourdement sur les performances. Ensuite, le caractère parfaitement bilingue du corpus a imposé un traitement spécifique du sens de lecture droite-à-gauche pour l'arabe. Enfin, les 39 fichiers scannés ont nécessité une chaîne dédiée que nous décrirons plus loin, sachant que ces documents proviennent essentiellement des archives plus anciennes du JORT.

6.2 Première itération : la chaîne texte classique

6.2.1 Vue d'ensemble du *pipeline*

Notre première version, implémentée dans `pipeline_complet.py`, combine quatre traitements successifs. PyMuPDF se charge d'abord d'extraire le texte page par page. Pour les documents scannés, nous intercalons une étape de préparation des images via OpenCV (`preprocess.py`). Le texte ainsi obtenu est ensuite découpé en segments sémantiques au niveau de l'article par `chunker.py`. Chaque segment est enfin vectorisé via le modèle `nomic-embed-text-v1.5` (`embeddings.py`) et stocké dans un index vectoriel local.



Figure 6.1 : Notre première chaîne de traitement, fondée sur PyMuPDF et OpenCV.

6.2.2 Quatre obstacles que nous avons documentés

Cette approche, séduisante par sa rapidité, nous a confrontés à des limites successives que nous avons analysées et tentées de corriger les unes après les autres.

Obstacle 1 : l'inversion du sens de lecture en arabe

PyMuPDF restitue les chaînes arabes dans l'ordre *visual*, où les caractères apparaissent de gauche à droite à l'écran. La sémantique exige pourtant l'ordre *logical* (de droite à gauche). Le texte brut que nous récupérions était donc inversé caractère par caractère, ce qui rendait toute exploitation impossible. Nous avons résolu ce problème en intégrant la bibliothèque `python-bidi` qui reconstruit l'ordre logique avant tout traitement ultérieur. Sur les trois PDFs arabes que nous avons utilisés comme corpus de tests, 23 pages ont été réordonnées avec succès.

Obstacle 2 : la pollution par les en-têtes répétitifs

Le JORT répète à chaque page le même bandeau de tête : titre du journal, date de parution, mention de la section. Sur 10 pages consécutives, nous voyions ainsi *Journal Officiel de la République Tunisienne — 3 janvier 2025* apparaître à chaque extraction. La pollution était double : ce texte rentrait dans les *chunks* et dégradait les vecteurs sémantiques, et la pertinence des résultats de recherche en souffrait. Nous avons mis au point un algorithme de détection par fréquence : toute ligne apparaissant dans au moins un tiers des pages est considérée comme un en-tête et retirée avant le *chunking*.

Obstacle 3 : les 39 fichiers sans texte exploitable

Pour les fichiers scannés que notre audit avait identifiés, nous avons construit une chaîne complète : préparation des images via OpenCV (débruitage, binarisation adaptative, correction d'inclinaison si l'angle dépasse 0,5°), puis reconnaissance optique via Tesseract 5 avec les modèles français et arabe. Les 81 images PNG correspondant à nos trois fichiers de référence ont toutes été traitées sans erreur. Le *Word Error Rate* théorique attendu sur ce type de documents restait cependant compris entre 0,15 et 0,25, soit environ dix fois plus que sur les fichiers natifs.

Obstacle 4 : l'effacement total de la structure

L'obstacle le plus pénalisant à nos yeux a été la perte de toute information de mise en page. Les tableaux de nominations, les colonnes parallèles du JORT français, la hiérarchie des articles, le sommaire en deux colonnes avec ses titres reliés à leur numéro de page par des points de suite : tous ces éléments visuels portent du sens, et ils disparaissaient entièrement après extraction. Le pire cas concerne les sommaires, où les numéros de page se retrouvaient agrégés en bloc à la fin (« 4 4 4 6 7 8 »), totalement inutilisables.

6.2.3 Bilan de la première itération

Notre chaîne V1 produit un index vectoriel fonctionnel, mais la qualité des réponses du RAG en pâtit clairement. Les *chunks* contiennent du bruit en-tête, perdent les liaisons croisées que portaient les tableaux et n'honorent jamais la structure légale des documents.

6.3 Deuxième itération : nettoyage par modèle de langage textuel

6.3.1 Notre motivation

Pour atténuer le bruit et compenser la perte de structure, nous avons décidé d'insérer un grand modèle de langage chargé de **nettoyer le texte brut** produit par PyMuPDF. Le script *minstral_extraction.py* a porté ces traitements. Notre idée de départ était simple : demander à

un LLM de retirer les en-têtes redondants, de remettre dans l'ordre les mots arabes inversés et de préserver scrupuleusement les références officielles (numéros de décret, dates, noms propres).

6.3.2 Choix du modèle et instrumentation

Notre choix s'est porté sur `ministral-3-3b-instruct-2512`, en quantification Q4_K_M (2,15 Go), servi localement par LM Studio sur `http://127.0.0.1:1234`. Nous avons rédigé un *prompt* spécifique par langue, en arabe pour les fichiers arabes et en français pour les fichiers français. Chaque *prompt* insiste sur trois consignes que nous jugions critiques : retirer les éléments répétitifs en tête et pied de page, conserver intacts tous les numéros de référence officiels et ne jamais ajouter d'information absente du texte source.

6.3.3 Résultats RAGAS observés

Pour évaluer la qualité résultante, nous avons écrit notre propre script d'évaluation *Evaluationrag.py* inspiré du *framework* RAGAS. Il s'appuie sur le LLM lui-même comme juge et mesure quatre métriques sur un échantillon de deux questions représentatives : une en français portant sur les obligations fiscales, une en arabe portant sur les droits du travailleur.

Tableau 6.2 : Métriques RAGAS que nous avons obtenues sur notre version V2.

Question	Faithfulness	Relevancy	Context Recall	Context Precision
Obligations fiscales [FR]	0,950	0,950	0,588	0,361
Droits du travailleur [AR]	0,800	0,850	0,267	0,010
Moyenne	0,875	0,900	0,427	0,185
Cible RAGAS	> 0,85 ✓	> 0,80 ✓	> 0,75 ✗	> 0,70 ✗

6.3.4 Trois limites que nous avons identifiées

Au-delà des chiffres, l'analyse fine des sorties a fait émerger trois problèmes structurels.

D'abord, nous avons constaté que les réponses arabes étaient fréquemment fournies en français. Lorsque nous demandions « *quelles sont les conditions de validité d'un contrat ?* » en arabe, le modèle Ministral nous répondait en français en citant le Code civil tunisien. Notre analyse a établi que le *system prompt* initial ne contraignait pas la langue de réponse. Le correctif a consisté à introduire un *prompt* conditionnel qui impose explicitement la langue de sortie selon la langue détectée côté entrée.

Ensuite, le score de *Context Recall* restait bloqué à 0,427, nettement en deçà du seuil cible de 0,75 fixé par les bonnes pratiques. Notre analyse a fait ressortir deux causes : les *chunks* demeuraient contaminés par les en-têtes répétitifs, et l'absence de séparation entre index arabes et français provoquait des retrievals croisés peu pertinents.

Enfin, l'obstacle de fond restait entier : le LLM textuel ne voit pas la page, il reçoit un flux

de mots sans aucune information visuelle. Les nominations groupées en tableau dans le JORT continuaient d'apparaître comme un bloc indistinct dans le texte de sortie.

6.3.5 Bilan de la deuxième itération

Si les scores de *Faithfulness* (0,875) et *Answer Relevancy* (0,900) atteignent les seuils visés, le *Context Recall* et le *Context Precision* bloquent la qualité globale. Notre analyse nous a conduits à conclure que l'obstacle structurel ne pouvait être levé qu'en sortant du paradigme textuel : aucun *prompt* de nettoyage, aussi raffiné soit-il, ne restituera la mise en page perdue à l'extraction.

6.4 Troisième itération : modèle vision-langage Qwen2.5-VL

6.4.1 Pourquoi l'extraction d'actes juridiques est un problème difficile

Avant d'exposer notre solution, il importe de mesurer la difficulté réelle de la tâche, car c'est elle qui justifie l'ampleur de notre travail d'ingénierie. Une page du *Journal Officiel* n'est pas un texte linéaire : c'est une mise en page dense, à **deux colonnes**, qui condense plusieurs régimes documentaires hétérogènes. Nous avons recensé sept caractéristiques qui, combinées, rendent l'extraction fiable particulièrement ardue.

- **Ordre de lecture à deux colonnes** : le texte d'un même acte court sur la colonne de gauche puis se poursuit en haut de la colonne de droite. Toute lecture qui mélange les colonnes brise la continuité juridique du texte.
- **Pluralité d'actes par page** : une page contient souvent la fin d'un acte, puis le début d'un autre. Il faut détecter la frontière (bloc de signature « Tunis, le ... », puis nouveau titre) et séparer correctement les actes.
- **Actes à cheval sur plusieurs pages** : un décret peut commencer page 4 et se terminer page 6. La page de continuation ne porte alors aucun titre : la confondre avec un nouvel acte crée des doublons et des titres fantômes.
- **Confusion autorité / titre** : les bandeaux en capitales (« PRÉSIDENCE DE LA RÉPUBLIQUE », « MINISTÈRE DE LA SANTÉ ») sont des *rubriques d'autorité*, et non des titres d'actes ; les prendre pour des titres fausse toute la classification.
- **Sommaires trompeurs** : une page de sommaire liste des intitulés « Décret n°... », « Arrêté du... » suivis de points de conduite et d'un numéro de page. Ce ne sont pas des actes mais des renvois ; les extraire comme des actes pollue gravement le corpus.
- **Tableaux et nominations** : les nominations multiples se présentent en tableaux (prénom, grade, emploi fonctionnel) qu'une extraction linéaire aplatit et rend illisibles.
- **Bilinguisme et arabe RTL** : la moitié du corpus est en arabe, lu de droite à gauche, avec des chiffres et des dates dans un ordre que les extracteurs classiques inversent.

À cela s'ajoute le **bruit de page** omniprésent (en-têtes « Journal Officiel... », pieds de page « Page 24 », mentions « Signature numérique », ISSN), qu'il ne faut jamais confondre avec le contenu juridique — notamment la date de pied de page, qu'un extracteur naïf prend pour la date de signature de l'acte.

6.4.2 Notre choix : un modèle vision-langage exécuté en local

Pour franchir le verrou identifié aux itérations précédentes, nous avons basculé vers une architecture radicalement différente : un modèle *vision-langage* (VLM) reçoit directement l'**image** de chaque page et en restitue une description structurée. L'intérêt est conceptuel autant que pratique : en « voyant » la page comme un lecteur humain, le modèle préserve par construction la structure bidimensionnelle (colonnes, tableaux, sommaires) que l'extraction texte détruisait.

Nous avons retenu `qwen2.5-vl-7b-instruct` pour quatre raisons convergentes. Sa qualité de lecture de l'arabe, y compris manuscrit ou calligraphié, est reconnue. Sa compréhension des structures tabulaires et des sommaires en colonnes s'est avérée, à l'usage, au niveau requis. Le modèle s'exécute **localement** via LM Studio en quantification Q4, ce qui respecte notre contrainte de souveraineté des données juridiques. Enfin, son API compatible OpenAI s'intègre sans friction à notre agent Python existant.

6.4.3 Notre nouveau *pipeline*

Le script `extract_with_vlm.py` (environ 2 900 lignes) remplace toute la chaîne PyMuPDF + Tika + OCR + LLM textuel par une séquence maîtrisée. Pour chaque page, `pypdfium2` produit une image PNG ; cette image est, le cas échéant, **découpée en colonnes** (tranches verticales) pour garantir l'ordre de lecture ; le modèle est ensuite sollicité en **deux passes** — une lecture fidèle (OCR vision) puis une structuration en JSON — dont la sortie traverse une couche de **validation et de réparation** avant d'être écrite, à raison d'un acte par ligne, dans le fichier JSONL. Loin d'être un simple appel d'API, cette chaîne est le fruit d'itérations successives que nous détaillons plus bas.

Le réglage de la résolution de rendu a été empirique. À 96 DPI, les diacritiques arabes et les petits chiffres (numéros d'actes, dates) devenaient illisibles ; nous avons retenu **150 DPI** comme compromis par défaut, en montant à 170 DPI et au-delà pour les pages arabes denses où la finesse des caractères l'exige. La largeur de l'image envoyée au modèle est par ailleurs bornée (2 400 pixels) pour éviter de saturer le contexte visuel du VLM.

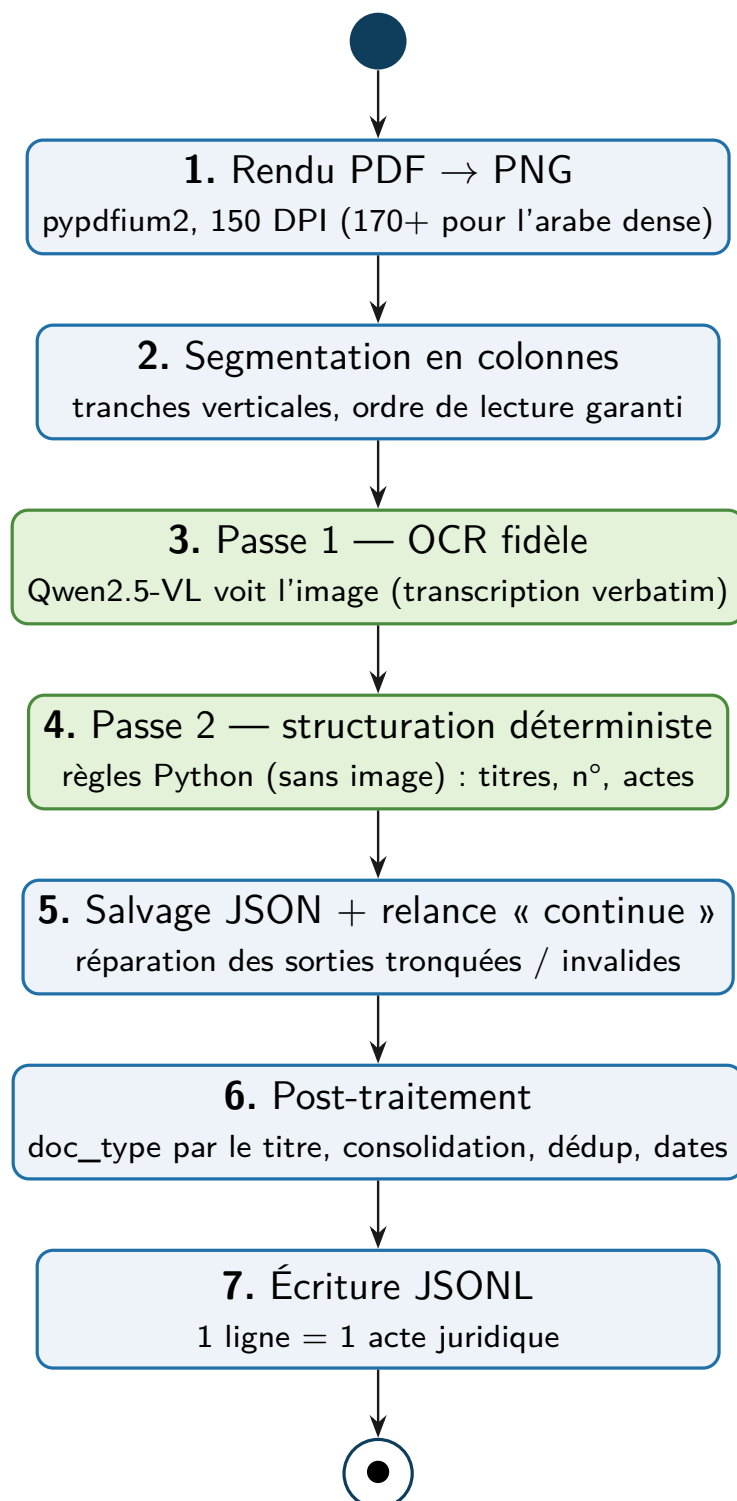


Figure 6.2 : Notre chaîne V3 : extraction par modèle vision-langage.

6.4.4 Une démarche expérimentale : des erreurs observées aux règles du *prompt*

Obtenir une extraction exploitable n'a rien eu d'immédiat. Notre première version se contentait d'envoyer l'image avec une consigne générale du type « extrais les actes juridiques de cette page en JSON ». Les résultats, systématiquement relus page à page contre le PDF source, ont révélé une série de défaillances récurrentes. Chacune d'elles a donné lieu à une **règle explicite** ajoutée au *prompt* : le *prompt* final n'est pas une consigne rédigée d'un trait, mais la sédimentation d'une dizaine de corrections expérimentales. Le tableau 6.3 retrace cette progression.

Tableau 6.3 : Erreurs observées et règles ajoutées au fil de nos tests d'extraction.

Problème observé sur les premiers résultats	Règle introduite dans le <i>prompt</i>
Le modèle inventait des « Vu », des articles, des dates ou des signataires absents de la page.	Règle n°0 — n'inventer jamais. Tout champ non visible vaut <code>null</code> ; on transcrit ce que l'on voit, on ne reconstitue pas.
Les bandeaux d'autorité en capitales (« MINISTÈRE... ») étaient pris pour des titres d'actes.	Règle n°1 — autorité $\neq$ titre. Ces bandeaux vont dans <code>authority</code> ; un titre commence par « Décret », « Arrêté », « Par arrêté », « Loi » ou « Décision ».
Le texte des actes était résumé ou tronqué après « Art. 2 », avec des « ... ».	Règle n°3 — contenu intégral verbatim , du préambule (« Vu... ») jusqu'à la clôture « Tunis, le... », sans reformuler ni abréger.
Les colonnes étaient mélangées ; le haut de la colonne de droite était sauté.	Règle d'ordre de lecture : lire entièrement la colonne de gauche, puis entièrement celle de droite, sans rien sauter ni tronquer.
Les tableaux de nominations étaient perdus ou aplatis.	Règle n°6 — tableaux : une ligne du tableau → un objet, une colonne → une clé (l'en-tête), toutes les lignes capturées.
Deux actes d'une même page étaient fusionnés ; une page de continuation répétait le titre précédent.	Règles n°7 et n°8 — séparation et continuité : séparer les actes à chaque bloc de signature ; une page commençant en plein acte est une continuation (<code>is_continuation</code> , titre vide).
Des signataires d'un autre acte étaient recopiés.	Règle n°9 — signataires uniquement si visibles dans le bloc de signature de <i>cet</i> acte ; sinon liste vide.
La date du pied de page (« ... 3 janvier 2025 ») était prise pour la date de signature.	Règle n°10 — ignorer le bruit (en-têtes, pieds de page, ISSN, « Signature numérique ») ; la date de clôture est celle signée en bas de l'acte.

Problème observé	Règle introduite
Les pages de sommaire étaient extraites comme des actes (faux décrets, faux « Vu »).	Contrat « toc » : sur une page de sommaire, produire un seul objet <code>doc_type = "toc"</code> et remplir <code>toc_entries</code> , sans créer d'acte.

Au-delà de ces règles, nous avons constaté que *la manière* de formuler les consignes influençait fortement l'obéissance du modèle. Les instructions **impératives, numérotées et accompagnées d'exemples concrets** (« Décret n° 2024-724 du... ») étaient bien mieux suivies que des descriptions abstraites. Distinguer typographiquement les **interdictions** (« NE mets JAMAIS... ») des attendus a également réduit les confusions autorité/titre. Enfin, imposer un **schéma de sortie JSON explicite**, avec l'énumération fermée des `doc_type` admissibles, a stabilisé la forme des réponses. Nous maintenons par ailleurs deux *prompts* parallèles, français et arabe, ce dernier intégrant les règles propres à la lecture de droite à gauche.

6.4.5 Le tournant : séparer la lecture de la structuration

Le durcissement du *prompt* a beaucoup amélioré les résultats, sans toutefois éliminer les hallucinations : tant que le modèle produisait *en un seul appel* le JSON final à partir de l'image, il lui restait loisible de « compléter » un acte par des éléments plausibles mais inexistantes. Nous avons alors opéré un changement d'architecture décisif : scinder l'extraction en **deux passes**.

1. **Passé 1 — lecture fidèle (OCR vision)**. Le VLM ne reçoit qu'une consigne de transcription : restituer, en texte brut, tout ce qui est écrit sur l'image, sans interpréter ni structurer. L'image n'est utilisée que pour *lire*.
2. **Passé 2 — structuration (sans image)**. Le découpage du texte transcrit en actes structurés se fait *à partir du texte seul*. Privé de l'image, le système *ne peut plus inventer* un décret qu'il n'a pas lu : l'hallucination est éliminée par construction.

Or nous avons observé qu'en confiant aussi la passe 2 à un modèle de langage, celui-ci continuait à halluciner — il prenait par exemple un fragment de bruit isolé (« **Décret n° 2024-723** » sans contexte) pour un titre, ou inventait un numéro. Nous avons donc poussé la logique jusqu'au bout en remplaçant la passe 2 par une **structuration déterministe en Python**, fondée sur des règles. Ce choix, contre-intuitif à l'ère du tout-LLM, a été pour nous le plus rentable :

- le **titre** est repris *verbatim* dans le texte lu : il n'est ni inventé, ni manquant ;
- le **numéro d'acte** est lu *dans le titre* et non dans le corps — ce qui évite de capturer un numéro cité dans un « Vu la loi n° 67-20 » et la fausse déduplication qui s'ensuivrait ; il n'est d'ailleurs extrait que pour les types qui en portent un réellement (les arrêtés et nominations n'ont pas de numéro propre) ;
- le **type d'acte** est déduit du titre par des motifs robustes (« Décret présidentiel », « Par

arrêté... » → nomination, etc.), le type renvoyé par le modèle n'étant qu'un repli ;

- les **signataires** sont reconstitués par adjacence « rôle (Le ministre...) puis nom », et seulement s'ils figurent dans le texte.

6.4.6 Robustesse face aux réponses dégradées du modèle

Un modèle de 7 milliards de paramètres en quantification Q4, sollicité sur des pages denses, produit régulièrement des sorties imparfaites. Une part importante de notre code est précisément consacrée à *recupérer le maximum d'information* de ces réponses dégradées, plutôt que de rejeter la page entière.

- **JSON tolérant.** Les virgules traînantes (,}, première cause de JSON invalide, sont corrigées ; un JSON **tronqué** (budget de *tokens* atteint) est réparé en refermant les structures ouvertes, ce qui sauve les actes déjà complets de la page au lieu de tout perdre.
- **Relance « continue ».** Lorsque la réponse est coupée (`finish_reason = length`), nous relançons le modèle en lui demandant de poursuivre exactement où il s'est arrêté, avec une **détection de boucle** qui interrompt le processus si le modèle se met à répéter son propre *prompt*.
- **Récupération de charge utile imbriquée.** Le VLM empaquetait parfois toute sa réponse JSON dans un champ texte (`content`) d'un acte ; nous détectons ce cas et ré-analysons ce contenu pour en récupérer les actes.

6.4.7 Post-traitement : fiabiliser et consolider les champs

Une dernière couche de traitement consolide les données avant l'écriture. Le VLM répartit souvent le même texte entre `content`, `articles[]` et `table_rows[]` ; comme la recherche n'interroge que le titre et le texte intégral, nous **réunissons** ces fragments dans un unique champ indexable, en **dé-dupliquant** les paragraphes et articles répétés. Le bloc de signatures, lu en ordre brouillé sur les sous-colonnes du JORT, est retiré du corps (les signataires demeurent dans le champ structuré dédié). La queue parasite de l'acte précédent, parfois collée en tête de `content` sur les pages à deux colonnes, est détectée puis supprimée. Enfin, les dates rédigées en toutes lettres, qu'elles soient françaises (« 31 décembre 2024 ») ou arabes tunisiennes (les mois propres au calendrier tunisien étant pris en charge), sont normalisées au format ISO 2024-12-31, sans jamais inventer de date absente.

6.4.8 Structure des données extraites et analyse des résultats

Le produit de la chaîne est le fichier *extracted_docs.jsonl*, où **chaque ligne est un acte juridique** autonome. Ce grain — un acte par document plutôt qu'un PDF par document — est un choix de conception : il rend chaque décret, arrêté ou nomination directement adressable par la recherche, indépendamment du numéro de JORT qui le contient. Chaque acte reçoit un identifiant lisible et reproductible, construit par concaténation de l'année, du numéro de JORT, de la langue, du type, de la page et d'une empreinte courte :

2025-jort1-fr-decree-p4-3b81599c

soit *année-jort-langue-type-page-hash*. L'empreinte garantit l'unicité (et donc l'idempotence de la ré-indexation), tandis que les autres segments restent interprétables à l'œil et facettables.

Le listing 6.1 présente, sous une forme abrégée, deux actes réellement produits par notre chaîne sur le premier JORT 2025 : un décret présidentiel et une nomination accompagnée de son tableau de bénéficiaires.

```
1 { "doc_id": "2025-jort1-fr-decree-p4-3b81599c", "source_pdf": "Jo0012025.pdf",
2   "page": 4, "language": "fr", "doc_type": "decree",
3   "title": "Decret n 2024-719 du 31 decembre 2024, portant prorogation ...",
4   "authority": "PRESIDENCE DE LA REPUBLIQUE",
5   "act_number": "2024-719", "date_signed": "2024-12-31",
6   "signatories": [ {"role": "Le President de la Republique", "name": "Kais Saied"} ],
7   "content": "Le President de la Republique, Vu la Constitution, Vu la loi n 92-54 ...",
8   "articles": [ ... ], "table_rows": [], "is_continuation": false }
9
10 { "doc_id": "2025-jort1-fr-nomination-p9-7de273f8", "page": 9,
11   "language": "fr", "doc_type": "nomination",
12   "title": "Par arrete du ministre de la sante du 9 decembre 2024.",
13   "authority": "MINISTERE DE LA SANTE", "date_signed": "2024-12-09",
14   "content": "Les cadres dont les noms suivent, sont charges des emplois ...",
15   "table_rows": [ ["Prenom et Nom", "Grade", "Emploi fonctionnel"],
16                   ["Ridha Derbali", "Administrateur en chef de la sante publique",
17                   "Directeur de l'hopital de circonscription de Jelma ..."] ] }
```

Listing 6.1 – Deux actes réels extraits (extrait abrégé de *extracted_docs.jsonl*).

Ces deux exemples illustrent la richesse du schéma. Le décret expose son préambule (« Vu... ») et ses articles, son numéro propre et son signataire ; la nomination, dépourvue de numéro, conserve en revanche son tableau de bénéficiaires ligne par ligne. Le tableau 6.4 récapitule les principaux champs, leur signification et leur utilité directe pour la recherche juridique.

Tableau 6.4 : Principaux champs extraits, leur signification et leur utilité pour la recherche juridique.

Champ	Signification	Utilité pour la recherche
doc_type	Type d'acte (decree, order_minister, nomination, law...)	Facette de filtrage par nature d'acte

Champ	Signification	Utilité
<code>title</code>	Intitulé complet, repris verbatim	Champ de recherche principal (fort <i>boost</i>)
<code>authority</code>	Autorité émettrice (présidence, ministère...)	Facette « émetteur » et filtrage institutionnel
<code>act_number</code> / <code>act_year</code>	Numéro officiel et année dérivée	Recherche exacte d'un acte, tri chronologique
<code>date_signed</code>	Date de signature au format ISO	Filtrage et tri par date
<code>content</code> / <code>raw_text</code>	Texte intégral consolidé de l'acte	Recherche plein texte (lexicale et sémantique)
<code>articles[]</code>	Articles structurés (numéro, contenu)	Recherche fine au niveau de l'article (<i>nested</i>)
<code>table_rows[]</code>	Tableaux (nominations, barèmes)	Rend les nominations cherchables nom par nom
<code>signatories[]</code>	Signataires {rôle, nom}	Facette « signataire »
<code>toc_entries[]</code>	Entrées de sommaire (titre, page)	Index croisé titre ↔ contenu
<code>language</code>	Langue de l'acte (fr / ar)	Aiguillage vers l'analyseur adapté

La comparaison avec nos itérations précédentes est éloquent. Là où la chaîne texte produisait un flux désordonné — arabe inversé, en-têtes intercalés, tableaux dissous, sommaires confondus avec des actes — la chaîne V3 livre des objets propres, typés et structurés, dans lesquels un décret est un décret, une nomination conserve son tableau, et un sommaire n'est jamais pris pour un texte de loi. C'est cette qualité structurelle, bien plus que le simple taux de caractères correctement lus, qui rend le corpus réellement exploitable par la recherche et l'assistant.

6.4.9 Gains et limites de notre chaîne V3

Le bénéfice le plus net est la **disparition des classes d'erreurs** qui plombaient les versions précédentes : l'arabe est lu nativement de droite à gauche, ce qui nous a permis de supprimer toute la chaîne *python-bidi* ; les tableaux sont préservés ; les sommaires sont correctement isolés ; et, surtout, la séparation lecture/structuration associée à la structuration déterministe a tari les hallucinations. Nous restons néanmoins lucides sur les défaillances **résiduelles** que nos relectures ont mises au jour, car aucune chaîne automatique n'est parfaite :

- des **tableaux à cheval sur deux pages** sont parfois rattachés imparfaitement, lorsque la page de continuation ne porte aucun en-tête réimprimé ;

- sur des pages très bruitées, un fragment isolé peut encore être **mal typé** (rangé en **other**) faute de marqueur de titre exploitable ;
- un caractère ou un nom propre peu lisible est transcrit [?] plutôt que deviné — un choix assumé : nous préférons une **donnée incomplète mais honnête** à une donnée inventée.

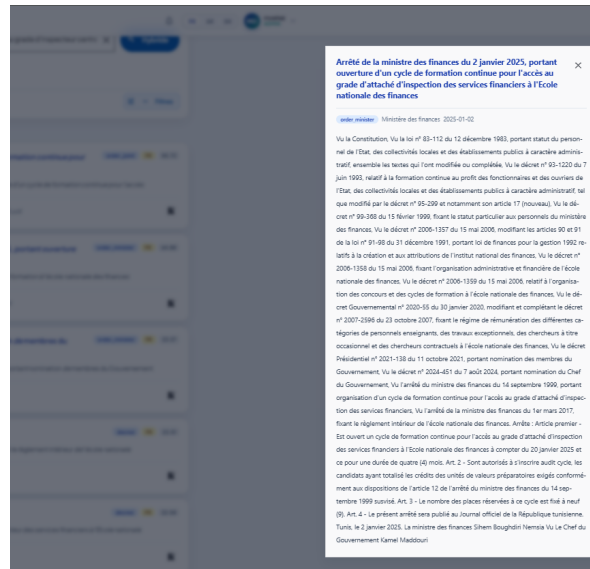
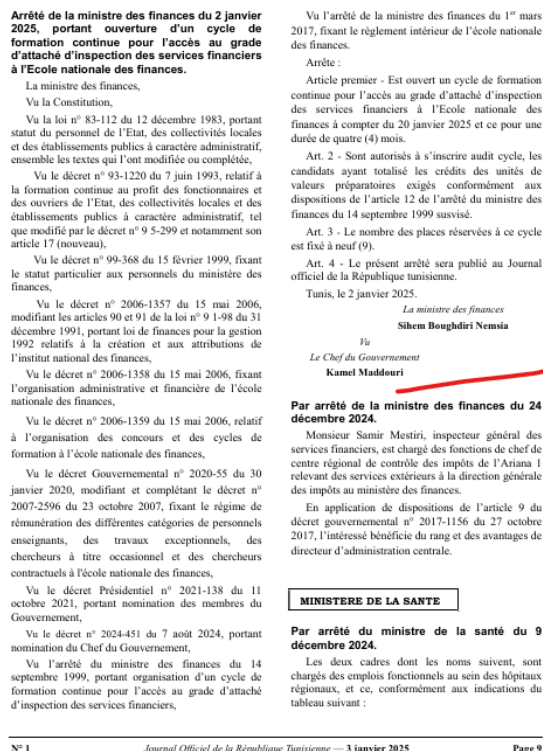
À ces limites de qualité s'ajoutent deux contraintes d'exploitation que nous assumons. La **latence** d'abord : 60 à 180 secondes par page selon la densité, soit plusieurs heures pour le corpus complet ; le coût reste acceptable car l'extraction est un traitement *hors ligne*, réalisé une seule fois par fichier. L'**empreinte mémoire** ensuite : Qwen2.5-VL-7B en quantification Q4 mobilise environ 8 Go de VRAM, ce qui exclut les cartes graphiques d'entrée de gamme — contrainte acceptable sur notre environnement d'exécution maîtrisé.

6.5 Validation sur des cas réels et difficultés résiduelles

Au-delà des métriques agrégées, nous avons validé notre chaîne d'extraction sur des cas concrets du *Journal Officiel*, en confrontant systématiquement la page source du JORT au résultat effectivement indexé puis restitué dans le portail. Cette confrontation visuelle, page par page, est le seul moyen rigoureux de vérifier que chaque acte est extrait avec son titre exact et son contenu intégral. Elle nous a permis d'identifier les structures documentaires les plus délicates et de cerner précisément les difficultés résiduelles.

6.5.1 Acte réparti sur deux pages

La première difficulté tient à la pagination du JORT : un même acte juridique débute fréquemment au bas d'une page et se poursuit sur la suivante. Sans traitement adapté, l'acte serait scindé en deux fragments, dont l'un sans titre. La figure 6.3 illustre le cas d'un arrêté du ministère des finances réparti sur deux pages : à gauche, les deux pages du JORT ; à droite, le document reconstitué dans le portail, dont le contenu a été correctement fusionné en un acte unique et complet.



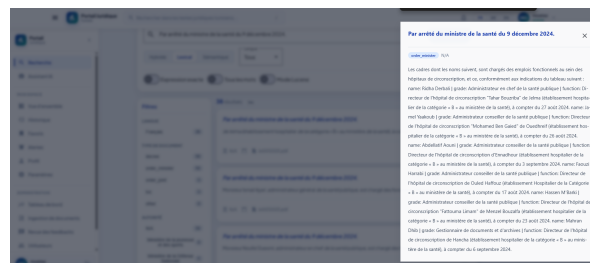
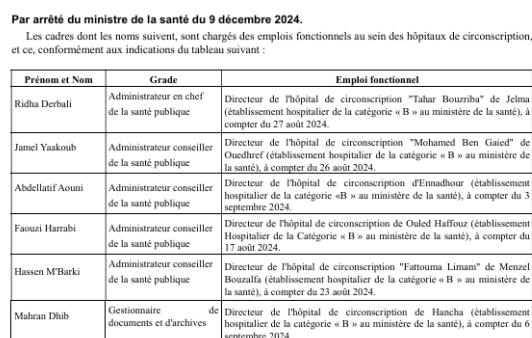
(a) Page source du JORT (acte à cheval sur deux pages)

(b) Acte reconstitué et complet dans le portail

Figure 6.3 : Fusion inter-pages d'un acte réparti sur deux pages du JORT.

6.5.2 Tableau de nominations sur une page entière

Les actes de nomination du JORT se présentent sous forme de tableaux (prénom et nom, grade, emploi fonctionnel). Lorsqu'un tableau occupe une page entière, notre chaîne en restitue l'intégralité du contenu, comme le montre la figure 6.4 : chaque ligne du tableau source est préservée dans le contenu de l'acte indexé, sans perte d'information.



(a) Tableau de nominations dans le JORT

(b) Contenu intégral restitué dans le portail

Figure 6.4 : Extraction complète d'un tableau de nominations occupant une page entière.

6.5.3 Difficulté résiduelle : tableau à cheval sur deux pages

Le cas le plus délicat que nous ayons rencontré est celui d'un tableau qui se prolonge d'une page sur l'autre. La figure 6.5 en présente un exemple : le tableau des emplois fonctionnels au sein des hôpitaux régionaux commence en bas de page et se poursuit au-delà du saut de page. Dans ce cas, la **totalité de la donnée est bien récupérée** — aucun nom, grade ni affectation n'est perdu — mais la structure tabulaire est *aplatie* en texte linéaire de la forme « Prénom et nom : ... | Grade : ... | Emploi fonctionnel : ... » au sein du contenu de l'acte. Nous assumons cette limite : elle préserve l'exhaustivité de l'information — ce qui est l'exigence première pour la recherche et le *RAG* — au prix d'une représentation moins structurée que le champ `table_rows[]` nominal.

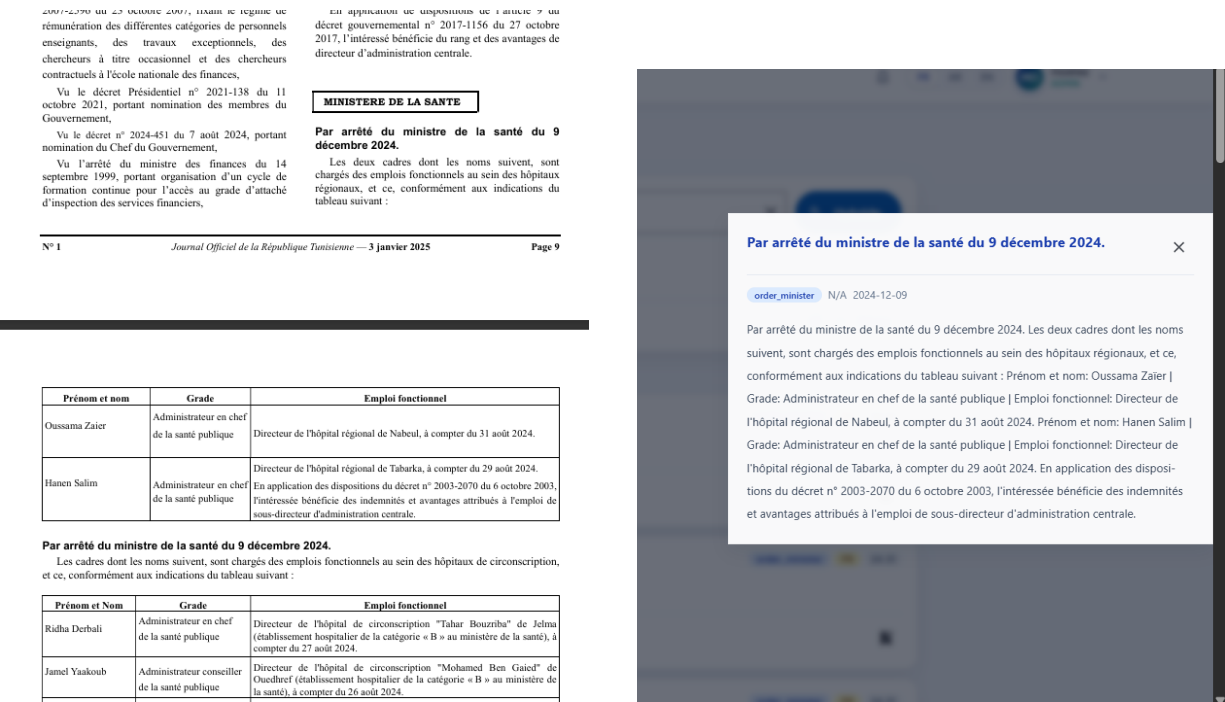


Figure 6.5 : Tableau réparti sur deux pages : donnée intégrale mais structure linéarisée.

6.5.4 Page mixte : décrets, arrêtés et tableau encadré

Une même page du JORT combine souvent plusieurs structures : deux colonnes d'actes distincts, un bandeau d'autorité encadré (« MINISTÈRE... ») et un petit tableau inséré dans le corps d'un décret. La figure 6.6 montre une telle page mixte et le résultat de sa segmentation : chaque acte est isolé, son autorité émettrice est correctement rattachée et le tableau encadré n'est pas confondu avec un titre.

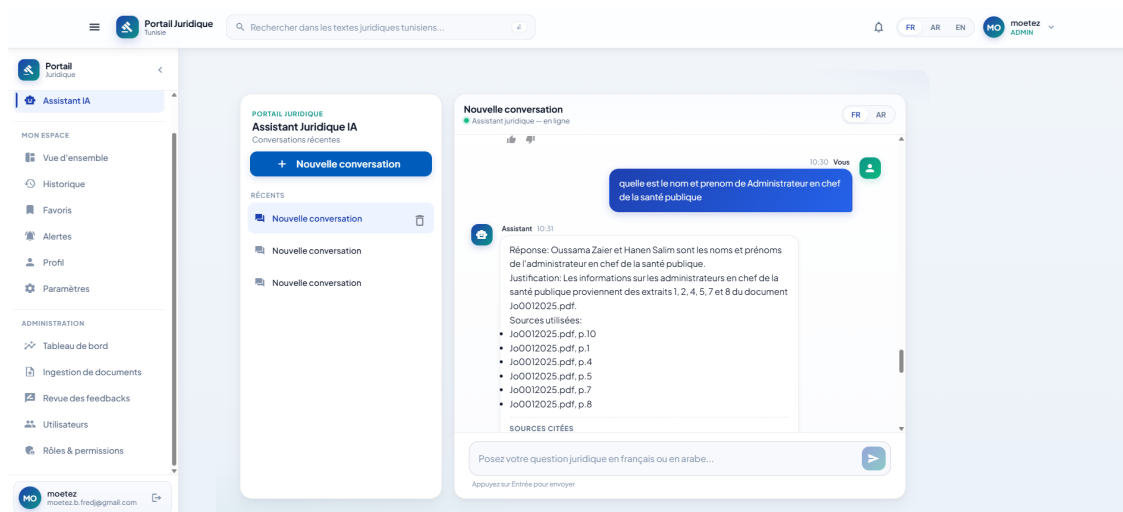


Figure 6.7 : Validation RAG : réponse correcte et sourcée à une question dont la donnée provient d'un tableau du JORT.

6.5.6 Bilan des difficultés d'extraction

Cette campagne de validation sur des cas réels confirme que notre chaîne atteint son objectif premier — **l'exhaustivité de la donnée** — sur l'ensemble des structures rencontrées : actes répartis sur plusieurs pages, tableaux pleine page et pages mixtes. La principale difficulté résiduelle, la linéarisation des tableaux à cheval sur un saut de page, n'entame pas l'exploitabilité de l'information puisqu'elle préserve la totalité du contenu et reste correctement indexée. Ces constats nous ont également guidés vers une simplification ultérieure du *pipeline* pour les éditions françaises du JORT, dont les fichiers comportent une couche texte native exploitable de manière déterministe — piste que nous évoquons parmi nos perspectives.

6.6 Synthèse comparative de nos trois itérations

Le tableau 6.5 récapitule les différences fonctionnelles et qualitatives entre les trois versions de notre *pipeline*.

Tableau 6.5 : Synthèse de nos trois itérations successives.

Critère	V1 PyMuPDF	V2 + Ministral	V3 Qwen2.5-VL
Type d'entrée	Texte brut	Texte brut + LLM	Image rendue
Lecture RTL arabe	python-bidi requis	python-bidi requis	Native
En-têtes répétitifs	Filtre frequence	Prompt LLM nettoie	Ignorés par le modèle
Tableaux	Perdus	Perdus	<code>table_rows[]</code>
Colonnes	Mélangées	Mélangées	Respectées
Sommaire (TOC)	Numéros agrégés	Idem	<code>toc_entries[]</code> structurés
Faithfulness	Non mesuré	0,875	Visé > 0,90
Context Recall	Non mesuré	0,427	Visé > 0,75
Latence par page	< 1 s	1 s + 9 s LLM	60–180 s
Souveraineté données	Locale	Locale (LM Studio)	Locale (LM Studio)

6.7 Indexation OpenSearch

6.7.1 Notre *pipeline* en quatre commandes

La sortie de notre chaîne V3 est un fichier *extracted_docs.jsonl* dans lequel chaque ligne représente un acte juridique unique. Le script *index_opensearch.py* que nous avons écrit consomme ensuite ce fichier et le pousse vers un index dédié nommé `jort_acts_v2`. La séquence opérationnelle se résume à quatre commandes que nous lançons successivement : extraction VLM, création de l'index avec son *mapping*, indexation *bulk* des documents puis vérification finale.

6.7.2 Notre *mapping* bilingue à analyseurs spécialisés

La conception du *mapping* a constitué un travail à part entière. Nous avons opté pour un schéma strict (`"dynamic": "false"`) qui interdit toute insertion d'un champ non déclaré dans le mapping. Cette discipline nous a paru essentielle pour préserver la stabilité du *scoring* dans le temps. Au cœur du mapping figurent deux *analyzers* personnalisés que nous avons baptisés `ar_legal` et `fr_legal`.

L'*analyzer* arabe applique successivement plusieurs traitements que nous avons jugés nécessaires. Un *char filter* retire les kashidas (caractère décoratif U+0640), unifie les variantes d'alef (les trois formes U+0623, U+0625, U+0622 sont toutes ramenées à U+0627), unifie ya et alef maksura (U+0649 → U+064A) et remplace la ta marbuta par un ha (U+0629 → U+0647). Cette normalisation préalable est suivie par l'analyseur arabe intégré d'OpenSearch (`arabic_normalization` qui retire les harakats), le filtrage des mots vides arabes et la racinisation `arabic stemmer`.

L'*analyzer* français, plus classique, applique l'élision des articles courts (l', d', j', etc.), filtre les mots vides français, applique le *stemmer light_french* (moins agressif que le standard pour préserver les radicaux juridiques) et termine par un *folding ASCII* qui insensibilise la recherche aux accents.

6.7.3 Multi-fields et types imbriqués

Pour les champs textuels critiques (`title`, `summary`, `raw_text`), nous avons opté pour une déclaration en *multi-fields* qui expose simultanément quatre vues : la vue `text` par défaut, une vue `.ar` analysée en arabe, une vue `.fr` analysée en français, et une vue `.keyword` pour les agrégations exactes. Cette redondance contrôlée nous permet d'écrire des requêtes adaptatives selon la langue de l'utilisateur.

Les structures internes d'un acte juridique (articles, tables, annexes, entrées de sommaire) sont modélisées comme types `nested`. Cette décision préserve l'indépendance de chaque sous-document lors des requêtes imbriquées : un utilisateur peut chercher dans le titre d'une entrée de sommaire spécifique sans que les autres entrées du même document ne polluent les agrégations.

6.7.4 Du JSONL au *mapping* : notre étape de normalisation

Le choix d'un format intermédiaire JSONL n'est pas anodin : en matérialisant la sortie de l'extraction dans un fichier **inspectable ligne par ligne**, nous **découplons** l'extraction (lente, coûteuse, réalisée une fois) de l'indexation (rapide, rejouable). Nous pouvons ainsi relire, corriger ou rejouer l'indexation sans relancer le VLM, et contrôler la qualité des données *avant* qu'elles n'entrent dans le moteur.

Le schéma « à plat » produit par l'extracteur ne coïncide toutefois pas exactement avec le *mapping* strict d'OpenSearch. Entre les deux, nous avons écrit une fonction `_normalize_doc` qui réconcilie les deux représentations sans perte d'information. Elle convertit `content` en `raw_text`, dérive `page_start` et `page_end` du champ `page` unitaire, extrait l'année (`act_year`) du numéro d'acte par expression régulière, transforme `table_rows` en `tables[]` imbriqué (en-têtes et lignes normalisés en chaînes), et regroupe les entités à plat (`people`, `organizations`, `places`, `laws_referenced`) sous l'objet `entities`. Elle aplatit aussi les signataires {rôle, nom} vers le champ `keyword` attendu (en conservant le nom, signal de facette le plus utile), retire les dates non interprétables — pour ne pas heurter le type `date` de l'index — et écarte les champs hors *mapping* (`date_signed_raw`, `is_continuation`, réponses brutes). Cette **validation systématique** avant indexation, conjuguée au schéma strict (`"dynamic": false`), garantit qu'aucun document mal formé ne vient corrompre l'index ni perturber le *scoring*.

6.7.5 Vérification après indexation

La sous-commande `verify` de notre script interroge OpenSearch pour confirmer la bonne réception du corpus et produire un résumé statistique. Elle compte le nombre total de documents et calcule trois agrégations : par langue, par type de document et par année JORT. Cette routine nous a servi de garde-fou tout au long du développement : elle a détecté plusieurs régressions dans nos *mappings* avant qu'elles ne se propagent en production.

6.7.6 Diagramme d'ensemble de notre indexation

La figure 6.8 donne une vue d'ensemble de notre chaîne d'indexation OpenSearch.

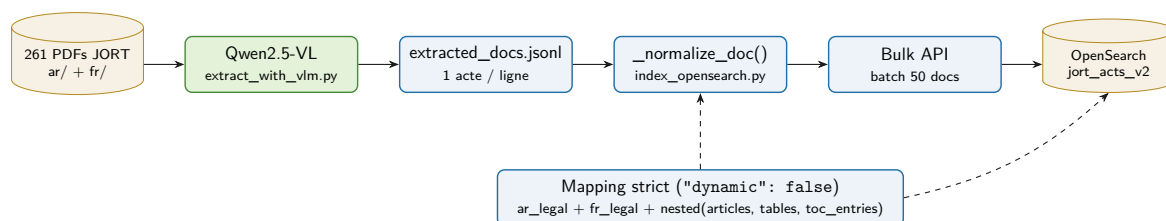


Figure 6.8 : Notre architecture d'indexation OpenSearch.

Conclusion

Le traitement du corpus JORT s'est imposé comme l'un des chantiers les plus formateurs de notre projet. La nature bilingue, la diversité des mises en page et la présence de fichiers scannés nous ont conduits à abandonner deux approches successives avant de stabiliser la troisième. Cette dernière, fondée sur le modèle vision-langage Qwen2.5-VL, restitue la structure documentaire que les techniques classiques d'extraction texte effaçaient. Notre indexation OpenSearch exploite ensuite cette structure au travers d'un *mapping* strict, d'analyseurs bilingues sur mesure et de types imbriqués qui ouvrent la voie à des requêtes sophistiquées par article ou par entrée de sommaire. Le chapitre suivant décrit la stratégie de déploiement Docker et d'intégration continue qui rend l'ensemble du système livrable et maintenable.

Conteneurisation, déploiement et intégration continue

Plan

Introduction	54
1 Sprint 3 : Assistant conversationnel RAG	54
2 Sprint 4 : Pipeline d'ingestion documentaire	60
Conclusion	66

Introduction

Après avoir détaillé la construction fonctionnelle de notre portail dans les chapitres précédents, nous abordons à présent la question opérationnelle : comment livrer ce système d'une machine de développement vers un environnement exécutable de manière reproductible. Notre choix a été de conteneuriser l'intégralité de la solution. Concrètement, chaque microservice Java que nous avons développé, l'application Angular et l'agent Python disposent désormais de leur propre image Docker [W9], bâtie à partir d'un *Dockerfile* qui leur est propre. En local, nous orchestrans ces conteneurs avec *Docker Compose* ; en production, le déclenchement et la mise à jour sont confiés à un *pipeline* GitLab CI [W10] organisé en plusieurs étapes.

Trois bénéfices ont guidé ce travail d'industrialisation. D'abord, la reproductibilité : le même *docker compose up* produit exactement la même topologie quel que soit le poste utilisé. Ensuite, la sélectivité : quand nous modifions un seul service, seules ses images sont reconstruites et redéployées. Enfin, l'automatisation : une fusion sur la branche `main` suffit à mettre la nouvelle version en service.

7.1 Notre stratégie de conteneurisation

7.1.1 Construction multi-étages pour les microservices Java

Nos huit microservices Spring Boot partagent une structure de *Dockerfile* identique, que nous avons conçue en deux étapes distinctes. La première étape, que nous nommons `build`, repose sur l'image officielle `maven:3.9.6-eclipse-temurin-21` ; elle compile le code source et produit un *fat-jar*. La seconde étape repart d'une image `eclipse-temurin:21-jre` considérablement plus légère et ne copie que le JAR final. Cette séparation nous a paru essentielle pour deux raisons : nous évitons d'embarquer Maven et les sources Java dans l'image livrée, et nous réduisons d'environ 70 % le poids final.

Une particularité de notre configuration : nous compilons avec l'argument `-DskipTests`. Cela ne signifie pas que nous délaissions les tests ; ils sont au contraire exécutés systématiquement, mais dans une étape dédiée du *pipeline* d'intégration continue décrit plus loin. La seule différence d'un service à l'autre se résume au numéro de port exposé (`EXPOSE 9001` pour `auth-service`, `9002` pour `search-service`, et ainsi de suite).

7.1.2 Frontend Angular servi par Nginx

Pour l'application Angular, nous avons appliqué le même principe de construction en deux étapes. La première utilise `node:22-alpine` pour produire la version compilée du SPA via `ng build`; la seconde récupère ces fichiers statiques dans une image `nginx:alpine`. Notre `nginx.conf` embarqué dans l'image règle deux problèmes en même temps : la redirection SPA (toute route inconnue retombe sur `index.html`) et le proxy de transmission vers la passerelle Spring Cloud pour les chemins commençant par `/api/`.

7.1.3 Image Python pour l'agent conversationnel

L'agent RAG est plus exigeant en dépendances système, en particulier pour PyMuPDF et le traitement d'images. Nous sommes partis de `python:3.11-slim` et avons ajouté manuellement les bibliothèques indispensables (`libgl1`, `libglu1`, `libsm6`, `libxext6`, `libxrender-dev`, `curl`). L'installation des paquets Python s'effectue depuis notre `requirements.txt` avec l'option `-no-cache-dir` pour ne pas alourdir inutilement l'image.

Une option supplémentaire que nous avons jugée précieuse : la directive `HEALTHCHECK` qui interroge périodiquement `/health` toutes les trente secondes. Nous avons fixé un `start-period` de soixante secondes après le démarrage, ce qui correspond au temps moyen d'initialisation que nous avons mesuré pour notre agent.

7.2 Orchestration locale avec Docker Compose

7.2.1 Topologie déployée

Notre fichier `docker-compose.yml` décrit quatre familles de services reliées par un même réseau bridge baptisé `pjnet`. La première famille rassemble l'infrastructure de support : MySQL 8.4 pour les données relationnelles, Redis 7 en mode `alpine` pour le cache, OpenSearch 2.13 pour le moteur de recherche, MinIO pour le stockage des fichiers, RabbitMQ avec son interface d'administration pour les messages asynchrones. Nous avons ajouté un service compagnon `minio-init` chargé de créer le `bucket` `portail-juridique` au premier démarrage.

La deuxième famille héberge notre agent IA Python, construit à partir du sous-projet `../juridique-py`. Pour ce service, nous avons veillé à exposer `host.docker.internal` grâce à `extra_hosts: ["host.docker.internal:host-gateway"]`. Cette astuce permet à notre agent, bien qu'enfermé dans son conteneur, de communiquer avec LM Studio qui s'exécute directement sur la machine hôte.

La troisième famille couvre nos huit microservices Spring Boot bâtis depuis le répertoire `./backend/`. Enfin, sept volumes persistants conservent les données entre les redémarrages : `mysql_data`, `opensearch_data`, `redis_data`, `minio_data`, `python_agent_index`, `python_agent_docs` et `python_agent_logs`.

La figure 7.1 illustre cette topologie sous la forme d'un diagramme de déploiement UML, où nous distinguons clairement les conteneurs métier, l'infrastructure et les volumes au sein du réseau pjnet.

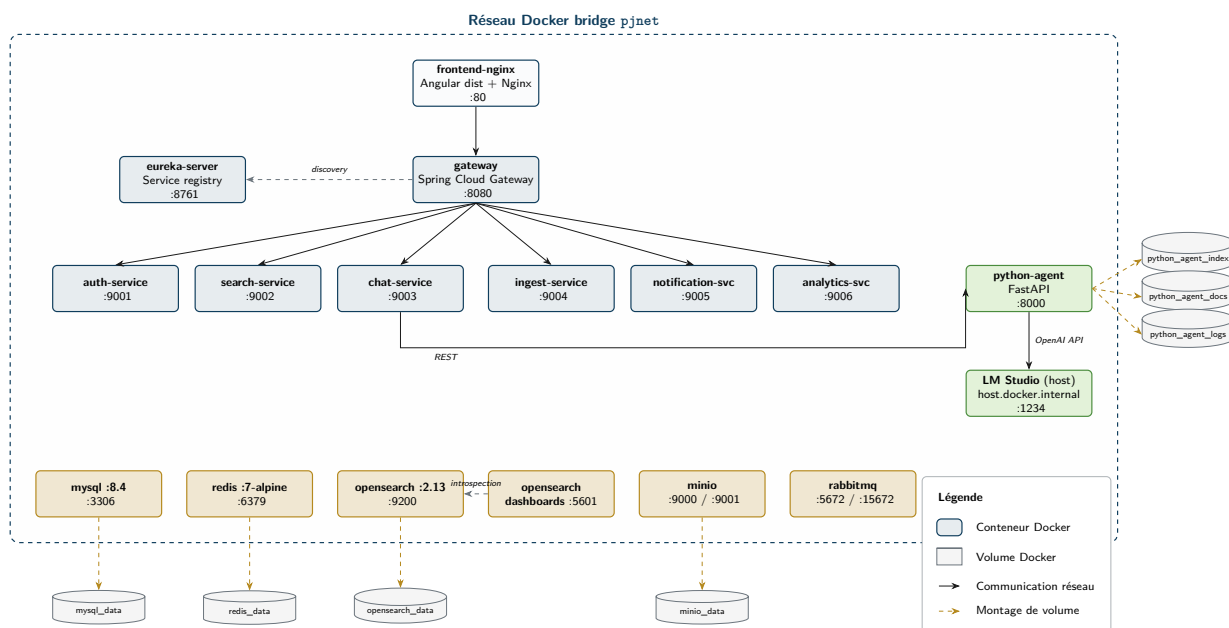


Figure 7.1 : Topologie de déploiement Docker que nous avons mise en place : conteneurs, volumes et réseau pjnet.

7.2.2 Gestion des dépendances de démarrage

L'un des points qui nous a posé difficulté en début de projet est l'ordre de démarrage des conteneurs. Nous l'avons résolu en associant à chaque service métier une condition de dépendance explicite, soit `service_healthy`, soit `service_started`. Concrètement, nos services qui consomment MySQL attendent que la sonde `mysqladmin ping` renvoie une réponse positive ; ceux qui consomment OpenSearch patientent jusqu'à ce que le cluster soit en état *green* ou *yellow*. Cette discipline a éliminé toute une classe d'erreurs intermittentes que nous rencontrions auparavant au démarrage.

7.2.3 Mode développement contre mode production

Nous maintenons un second fichier, `docker-compose.deploy.yml`, qui sert de surcharge pour les déploiements vers nos environnements de qualification et de production. Trois modifications principales sont appliquées par cette surcharge. Premièrement, la directive `build` est neutralisée et remplacée par une référence à une image préalablement poussée dans le registre GitLab (`image: ${CI_REGISTRY_IMAGE}/...:latest`). Deuxièmement, nous activons le profil Spring prod. Troisièmement, nous injectons les valeurs sensibles depuis les variables de CI/CD plutôt que depuis le fichier local : clé de signature des jetons, identifiants des bases, mots de passe SMTP. Cette séparation entre développement et production constitue une bonne pratique que nous appliquons rigoureusement.

7.3 Notre *pipeline* GitLab CI/CD

7.3.1 Trois étapes pour une livraison maîtrisée

Au cœur de notre fichier `.gitlab-ci.yml` nous avons défini trois étapes (*stages*) qui s'exécutent successivement. La première, nommée `build`, est laissée en réserve pour une éventuelle phase de compilation préalable que nous ajouterons à l'avenir si nécessaire. La deuxième, `docker`, regroupe les travaux de construction et de publication des images dans le *Container Registry* de GitLab. La troisième, `deploy`, assure le rapatriement et le redémarrage des conteneurs sur la branche `main` uniquement.

Une décision que nous avons prise dès le départ est de taguer chaque image avec le *short SHA* du commit en cours (`CI_COMMIT_SHORT_SHA`). Cette traçabilité nous permet de remonter sans ambiguïté à la révision exacte du code embarquée dans une image donnée. Nous ajoutons en parallèle un tag `latest` pour simplifier les déploiements par défaut.

7.3.2 Factorisation des travaux Docker

L'écriture initiale du pipeline contenait beaucoup de répétitions : chaque microservice nécessitait des étapes identiques de construction. Nous avons rapidement factorisé cette logique dans un *template* caché que nous avons baptisé `.docker_build_push`. Ce gabarit prépare le service Docker-in-Docker, configure l'authentification sur le registre via `CI_JOB_TOKEN`, puis laisse à chaque travail spécifique le soin de spécifier uniquement le sous-dossier à compiler et le nom de l'image résultante.

Le bénéfice est double : une maintenance simplifiée (modifier le gabarit suffit pour propager le changement à tous les services) et une lisibilité accrue du fichier YAML. Sept travaux dérivés couvrent ainsi `eureka-server`, `gateway`, `auth-service`, `chat-service`, `ingest-service`, `notification-service` et `analytics-service`.

7.3.3 Construction sélective par service

Une autre optimisation à laquelle nous tenons est le *build* conditionnel. Plutôt que de reconstruire les huit images à chaque commit, nous utilisons la clause `rules: changes:` de GitLab CI qui déclenche un travail uniquement quand les fichiers correspondants ont été modifiés. Concrètement, modifier un seul fichier dans `backend/search-service/` ne reconstruira que l'image de `search-service`, laissant les autres intactes. Cette discipline a divisé par cinq notre durée moyenne de pipeline sur les commits courants.

7.3.4 Déploiement automatique sur la branche principale

Le travail `deploy` ne s'exécute que sur la branche `main` et uniquement si le commit touche au moins un fichier sous `backend/`, le fichier `docker-compose.yml`, son surcharge `docker-compose.deploy.yml` ou le `.gitlab-ci.yml` lui-même. Quand ces conditions sont remplies, notre *script* calcule la liste des services réellement modifiés par comparaison avec le commit précédent, puis enchaîne trois opérations : vérification de la présence de l'image `latest` dans le registre (et reconstruction *ad hoc* si elle manque), récupération via `docker compose pull` et redémarrage par `docker compose up -d --remove-orphans` sur la seule liste filtrée.

Cette logique permet, dans la quasi-totalité des cas, un déploiement de moins de deux minutes après la fusion d'une pull request. Si la modification touche un fichier transverse (comme `docker-compose.yml`), nous redéployons l'ensemble par précaution pour garantir la cohérence de la topologie.

7.3.5 Notre exécuteur GitLab

Le travail `deploy` cible un *runner* GitLab que nous avons configuré sur la machine hôte de notre environnement de démonstration, étiqueté `backend-test`. Ce *runner* dispose de l'accès au *socket* Docker local, ce qui lui permet de piloter *Docker Compose* directement. Cette configuration nous a évité de mettre en place un accès SSH, et nous laisse l'avantage que GitLab pilote le déploiement de bout en bout depuis l'interface web.

7.4 Externalisation des paramètres sensibles

L'une des préoccupations qui a guidé notre conception du pipeline est l'absence de toute valeur sensible dans le code source. Toutes ces valeurs sont externalisées via les variables d'environnement de Docker Compose, elles-mêmes alimentées par les variables protégées du projet GitLab. Le tableau 7.1 récapitule les principales.

Tableau 7.1 : Paramètres externalisés que nous avons identifiés.

Variable	Usage dans notre projet
JWT_SECRET	Clé HMAC partagée par les services pour la validation des jetons.
JWT_EXPIRATION_MS	Durée de vie de l' <i>access token</i> .
JWT_REFRESH_EXPIRATION_MS	Durée de vie du <i>refresh token</i> .
DB_USER et DB_PASSWORD	Couple d'identifiants MySQL utilisé par tous les services persistants.
OPENSEARCH_URL	Adresse interne du cluster OpenSearch dans le réseau <code>pjnet</code> .

Variable	Usage dans notre projet
MAIL_USERNAME et MAIL_PASSWORD	Compte SMTP destiné à l'envoi des codes OTP.
MINIO_ROOT_USER et MINIO_ROOT_PASSWORD	Identifiants administrateur de MinIO.
PYTHON_AGENT_URL	URL interne pointant vers notre agent RAG.
PYTHON_AGENT_ENABLED	Bascule activant ou désactivant le chemin RAG.
LM_STUDIO_BASE_URL	Point d'accès à l'API du modèle de langage.
CORS_ALLOWED_ORIGINS	Origines autorisées par la passerelle.

7.5 Cycle de déploiement type d'une modification

Pour illustrer concrètement l'enchaînement automatique permis par notre chaîne, prenons l'exemple typique d'une évolution métier que nous avons réalisée plusieurs fois pendant le projet.

Le développeur produit un commit sur une branche de fonctionnalité. GitLab CI déclenche alors uniquement les travaux `docker_*` correspondant aux dossiers modifiés. Ces travaux fabriquent une nouvelle image étiquetée par le *short SHA* du commit et, en parallèle, réécrivent l'étiquette `latest`.

Après la relecture par un pair et la fusion sur la branche `main`, le travail `deploy` prend le relais. Il vérifie ou reconstruit l'image si nécessaire, déclenche `docker compose pull` pour récupérer la nouvelle version, puis exécute `docker compose up -d` en ne ciblant que les services impactés.

Enfin, une commande `docker image prune -f` nettoie les images intermédiaires qui s'accumuleraient sinon sur le *runner*, libérant ainsi l'espace disque pour les prochains *pipelines*.

Conclusion

Notre stratégie de déploiement repose sur une discipline simple mais efficace : ne reconstruire et ne redémarrer que ce qui a réellement changé. Cette approche a réduit drastiquement notre temps moyen entre un commit et sa mise en service, tout en limitant le risque de régression sur les composants non touchés. L'unification de l'orchestration via Docker Compose, étendue vers la production par un simple fichier de surcharge, garantit la cohérence entre l'environnement de chaque développeur et la cible de production. L'ensemble des chapitres techniques étant désormais couvert, la conclusion générale du rapport synthétise les apports du projet et discute les évolutions que nous envisageons pour la suite.

Conclusion générale et perspectives

Ce projet de fin d'études avait pour ambition de concevoir et de développer un portail juridique intelligent dédié aux textes de loi tunisiens, en réponse à un besoin réel et documenté : l'accès difficile à l'information juridique pour les citoyens, les professionnels du droit et les chercheurs tunisiens. À l'issue des six sprints Scrum répartis en trois releases successives, ce besoin a trouvé une réponse complète sous la forme d'un système fonctionnel, modulaire et déployable via Docker Compose.

Le travail accompli a abouti à une architecture microservices robuste, composée de huit services indépendants (Eureka, Gateway, authentification, recherche, chat, ingestion, notification et analytics) orchestrés par Spring Boot 4.0.5 et Spring Cloud 2025.1.0, communiquant de manière synchrone via REST et asynchrone via RabbitMQ. La plateforme repose sur un mécanisme d'authentification par email et mot de passe (haché en bcrypt), renforcé par un code OTP à usage unique envoyé par courriel en second facteur optionnel, avec génération de jetons JWT signés, refresh token et liste de révocation Redis, garantissant à la fois sécurité et simplicité d'usage. Le cœur fonctionnel du portail est un moteur de recherche hybride combinant la recherche lexicale BM25 et la recherche sémantique par vecteurs denses kNN via OpenSearch 2.13.0, avec fusion des résultats par *Reciprocal Rank Fusion* pour une pertinence optimale sur un corpus juridique bilingue français-arabe.

À cette recherche s'ajoute un assistant conversationnel reposant sur le paradigme RAG et le modèle de langage Mistral exécuté via Ollama, capable de répondre à des questions juridiques en langage naturel tout en citant systématiquement ses sources documentaires, ce qui limite les hallucinations inhérentes aux grands modèles de langage. L'alimentation du système est assurée par un pipeline d'ingestion documentaire complet, intégrant Apache Tika 2.9.1 pour l'extraction de texte, Tesseract OCR 5.x pour les PDF scannés en français et en arabe, un mécanisme de découpage sémantique, la génération d'embeddings par le modèle *multilingual-e5-base* et le stockage des fichiers originaux dans MinIO. La plateforme se complète d'un système de notifications proactives par alertes thématiques, d'un tableau de bord analytique destiné à l'administrateur, et d'une interface utilisateur bilingue développée avec Angular 21.2.6 offrant un support natif de l'écriture de droite à gauche pour la langue arabe.

La réalisation de ce projet a impliqué plusieurs défis techniques. L'utilisation simultanée de versions très récentes — Spring Boot 4.0.5 reposant sur Jakarta EE 11, Spring Cloud 2025.1.0 et Angular 21 — aux interfaces de programmation parfois instables a exigé une veille technologique continue et de nombreux ajustements de configuration. La gestion du bilinguisme arabe-français a constitué une difficulté non triviale, qu'il s'agisse de configurer l'analyse de texte arabe dans OpenSearch (tokenisation, racinisation) ou d'intégrer le support RTL côté Angular. L'optimisation du pipeline OCR pour les documents scannés du JORT, souvent de qualité médiocre, a nécessité un prétraitement d'images (débruitage, correction de l'inclinaison) avant la reconnaissance. Enfin, l'ajustement du *prompt* RAG, afin de limiter les hallucinations du modèle tout en conservant des réponses fluides et correctement sourcées, a demandé de nombreuses itérations expérimentales.

Ce projet ouvre par ailleurs de nombreuses perspectives d'amélioration et d'extension. Un *fine-tuning* d'un modèle de langage spécialisé sur le corpus juridique tunisien permettrait d'améliorer la précision et la terminologie des réponses, tandis que la construction d'un graphe de connaissances des lois tunisiennes (relations entre articles, références croisées) enrichirait le contexte fourni au modèle. L'ajout d'un résumé automatique des textes longs faciliterait la lecture rapide, et une migration de Docker Compose vers Kubernetes, accompagnée de *charts* Helm, offrirait une scalabilité horizontale automatisée en production. À plus long terme, le développement d'une application mobile consommant les mêmes API REST, l'intégration d'un *crawler* récupérant automatiquement chaque nouveau numéro du Journal Officiel, ainsi qu'un module de comparaison des versions successives d'un même texte de loi constitueraient des évolutions naturelles de la plateforme.

En définitive, ce projet de fin d'études a été une occasion exceptionnelle de mobiliser l'ensemble des compétences acquises au fil de trois années de formation en Génie Logiciel et Systèmes d'Information, tout en contribuant à un enjeu de société majeur : rendre le droit accessible à tous les Tunisiens, dans leur langue, sans barrière technique ni financière.

Bibliographie

- [B1] Pierre Pezziardi, *Référentiel des Pratiques Agiles*, édition ebook, 2013.
- [B2] Ken Schwaber, Jeff Sutherland, *The Scrum Guide — The Definitive Guide to Scrum : The Rules of the Game*, Scrum.org, 2020.
- [B3] Sam Newman, *Building Microservices : Designing Fine-Grained Systems*, 2^e édition, O'Reilly Media, 2021.
- [B4] Craig Walls, *Spring in Action*, 6^e édition, Manning Publications, 2022.
- [B5] Sebastián Eschweiler, *Angular : Up and Running*, O'Reilly Media, 2023.
- [B6] Lewis Tunstall, Leandro von Werra, Thomas Wolf, *Natural Language Processing with Transformers*, O'Reilly Media, 2022.
- [B7] Mairead Donahue, *Elasticsearch : The Definitive Guide*, O'Reilly Media, 2015 (principes applicables à OpenSearch).
- [B8] Patrick Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020, arXiv :2005.11401.
- [B9] Junyang Lin et al., *M3E : Multilingual and Multi-Functionality Embedding Model*, 2024.
- [B10] Stephen Robertson, Hugo Zaragoza, *The Probabilistic Relevance Framework : BM25 and Beyond*, Foundations and Trends in Information Retrieval, 2009.
- [B11] Gordon V. Cormack, Charles L.A. Clarke, Stefan Buettcher, *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*, SIGIR 2009.

Webographie

[W1] Spring Boot, *Reference Documentation v4.0*. Disponible sur : <https://docs.spring.io/spring-boot/docs/4.0.x/reference/html/> (consulté le 10 juin 2026).

[W2] Spring Cloud, *Documentation 2025.1.0*. Disponible sur : <https://spring.io/projects/spring-cloud> (consulté le 10 juin 2026).

[W3] Angular, *Documentation v21*. Disponible sur : <https://angular.dev/docs> (consulté le 10 juin 2026).

[W4] OpenSearch, *Documentation 2.13*. Disponible sur : <https://opensearch.org/docs/latest/> (consulté le 8 juin 2026).

[W5] Ollama, *Documentation*. Disponible sur : <https://ollama.com/docs> (consulté le 8 juin 2026).

[W6] Apache Tika, *Documentation 2.9*. Disponible sur : <https://tika.apache.org/2.9.0/> (consulté le 8 juin 2026).

[W7] MinIO, *Documentation*. Disponible sur : <https://min.io/docs/minio/> (consulté le 8 juin 2026).

[W8] RabbitMQ, *Documentation 3.x*. Disponible sur : <https://www.rabbitmq.com/docs> (consulté le 9 juin 2026).

[W9] Docker, *Documentation*. Disponible sur : <https://docs.docker.com/> (consulté le 9 juin 2026).

[W10] GitLab, *CI/CD Documentation*. Disponible sur : <https://docs.gitlab.com/ee/ci/> (consulté le 9 juin 2026).

[W11] NgRx, *Signals Documentation*. Disponible sur : <https://ngrx.io/guide/signals> (consulté le 9 juin 2026).

[W12] Journal Officiel de la République Tunisienne (JORT). Disponible sur : <https://www.iort.gov.tn> (consulté le 11 juin 2026).

[W13] Baeldung, *Spring Boot Tutorials*. Disponible sur : <https://www.baeldung.com> (consulté le 7 juin 2026).

[W14] Hugging Face, *multilingual-e5-base*. Disponible sur : <https://huggingface.co/intfloat/multilingual-e5-base> (consulté le 7 juin 2026).

[W15] Mistral AI, *Documentation*. Disponible sur : <https://docs.mistral.ai> (consulté le 7 juin 2026).



Résumé

Ce rapport présente la conception et le développement d'un *portail juridique intelligent* dédié aux textes de loi tunisiens (JORT), bilingue français-arabe. La plateforme repose sur une architecture microservices de huit services Spring Boot 4.0.5 / Spring Cloud 2025.1.0 (Java 21) orchestrés par Eureka derrière un Spring Cloud Gateway WebFlux, et sur un *agent IA Python* (FastAPI) jouant le rôle d'orchestrateur du pipeline RAG. La recherche documentaire combine recherche lexicale BM25 et recherche sémantique par vecteurs denses (kNN) via OpenSearch 2.13. L'agent intègre un *routeur d'intention* en cascade à trois niveaux (regex / score juridique / LLM zero-shot), un score d'hallucination heuristique, le streaming des réponses en SSE, et l'évaluation continue via les métriques RAGAS (*faithfulness, answer relevancy, context recall*). Le modèle de langage est Ministral 3B servi localement par LM Studio (API OpenAI-compatible), garantissant la souveraineté des données juridiques. L'interface Angular 21 offre une expérience RTL native pour l'arabe. La conduite du projet a suivi la méthodologie Scrum en six sprints sur trois releases.

Mots-clés : Microservices, RAG, OpenSearch, Spring Boot, Angular, Recherche sémantique, LLM local, Ministral, RAGAS, Scrum, JWT, NLP

Abstract

This report describes the design and implementation of an *intelligent legal portal* for Tunisian legislative texts (JORT), with bilingual French-Arabic support. The platform is built upon a microservices architecture composed of eight Spring Boot 4.0.5 / Spring Cloud 2025.1.0 services (Java 21) registered with Eureka behind a Spring Cloud Gateway WebFlux, and a dedicated *Python AI agent* (FastAPI) orchestrating the RAG pipeline. Retrieval combines BM25 lexical search and dense vector kNN through OpenSearch 2.13. The agent integrates a three-tier *intent router* (regex / legal-feature scoring / LLM zero-shot), a heuristic hallucination score, SSE answer streaming, and continuous evaluation via RAGAS metrics (*faithfulness, answer relevancy, context recall*). The language model is Ministral 3B served locally through LM Studio (OpenAI-compatible API), guaranteeing data sovereignty for sensitive legal content. The Angular 21 front-end delivers a native RTL experience for Arabic. The project was conducted using Scrum across six sprints divided into three releases.

Keywords : Microservices, RAG, OpenSearch, Spring Boot, Angular, Semantic search, local LLM, Ministral, RAGAS, Scrum, JWT, NLP

